

Lecture #2: The System R Optimizer

15-721 Advanced Database Systems (Fall 2025)

<https://www.cs.cmu.edu/~15721-f25/>

Carnegie Mellon University

Notes by: Te-Yen (Andy) Chou, Leon Lu

1 Introduction & Background

SQL is a declarative language where users specify what data to retrieve, not how to retrieve it. The database management system (DBMS) must translate SQL queries into imperative execution plans. Query optimization is both one of the hardest (NP-hard) and one of the most important components of DBMS. Even with a very fast execution engine (say, 10x faster), if the optimizer picks a bad plan, the query can still be very slow, potentially 100,000x slower than an optimal plan.

1.1 Backdrop

As a bit of historical context, the Selinger paper [?] was published in the 1970s, the same decade in which the relational model was developed at IBM.

1.2 The Rise of Data Independence

The need for query optimization is a direct consequence of the relational model, which introduced the crucial concept of **data independence**. Before data independence, many databases used to store data based on its physical location on the storage device, which can be a source of inconvenience for users who simply want to query data *declaratively* without needing to know the low-level details of how it is stored. The idea of data independence is that applications should be insulated from the low-level details of how data is stored and organized. There are two types of data independence:

- **Physical Data Independence:** This protects applications from changes in the physical storage structure. For example, an administrator can add an index, change the file organization, or move data to a different disk without requiring any changes to the application code. The application programmers don't need to touch those physical structures. (Instead of telling "get the data on that physical address", the application programmer can now just say "I want that data".)
- **Logical Data Independence:** This protects applications from changes to the logical schema. This is often achieved through **views**, which provide a stable, virtual representation of data even if the underlying base tables are restructured (e.g., a table is split into two, or columns are renamed). The application continues to query the consistent view, while the DBMS handles the translation to the new schema. This is also useful for security. For example, instead of giving one user permission for the whole table, we can give a user permission for a view, which might just include two of the 50 columns of the table.

One of the biggest advantages is that application programmers can focus on business logic without worrying about the specific internal storage layout. One can argue that physical data independence is what makes the relational model work: if a system did not have such physical independence, changes to the storage layout or the indexes can easily require a change to the query, which does not fit SQL's declarative paradigm. At the same time, data independence means that the DBMS must figure out the lower-level details of how to access data efficiently given the user's declarative queries, such as by considering the B-Tree in the above example.

1.3 System R: The Pioneering Optimizer

System R is a classic database system that pioneered cost-based query optimization and influences nearly all modern optimizers [?]. It employs a bottom-up, dynamic programming strategy that systematically constructs optimal plans by combining smaller, optimal subplans.

System R's key innovations include:

- **Cost-Based Optimization:** Comparison of execution plans using I/O and CPU cost models
- **Dynamic Programming:** Building optimal n -way joins from optimal $(n - 1)$ -way joins
- **Interesting Orders:** Retaining plans that produce useful sort orders for subsequent operations
- **Restricted Search Space:** Limiting consideration to left-deep join trees

The topics covered will be as follows:

- System R's Search Space & Enumeration Strategy
- The System R Cost Model & Cardinality Estimation
- System R Join Methods
- Handling Nested Queries
- Modern Advancements in Cardinality Estimation

2 System R's Search Space & Enumeration Strategy

To manage the complexity of optimization (which is NP-hard with respect to the number of joins), the System R optimizer makes critical design choices to prune the search space.

2.1 Search Space: Left-Deep Join Trees

An n -way join can be represented as a tree. Figure ?? shows a few common classes of tree shapes.

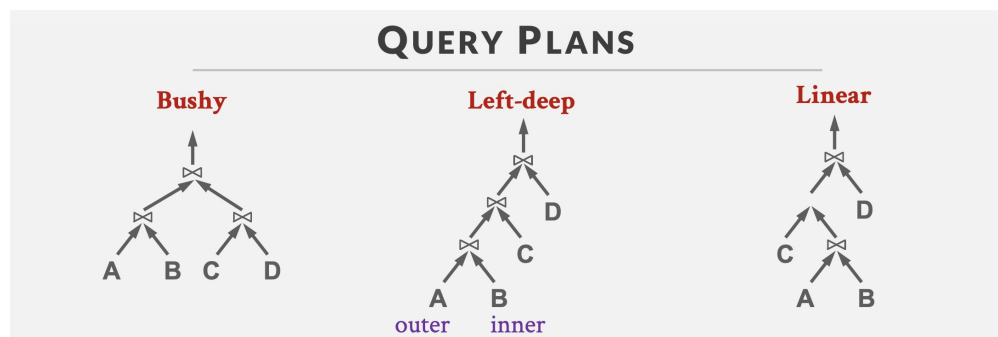


Figure 1: Three types of query plan structures: Bushy, Left-deep, and Linear

- **Bushy Tree:** The most general plan, where the input to a join can be the result of two other joins. This offers the largest search space but is complex to optimize.
- **Left-Deep Tree:** A plan where the inner relation of every join is always a base table. This structure is highly conducive to **pipelining**.
- **Linear Tree:** A plan where at least one child of every join is a base relation.

System R deliberately restricts its search to **left-deep join trees**. By focusing only on left-deep plans, System R reduces the search space for join permutations from $O(n!)$ to a more manageable $O(2^n)$. System R also avoids considering **Cartesian products** unless absolutely necessary, further pruning the search (Cartesian products can be fast in some cases, but are often expensive).

It is possible, in certain cases, that a bushy plan will be better than a left-deep plan, such as illustrated in ?? . In cases like this, System R can miss the optimization opportunity.

Looking back from 2025, even with the $O(n!)$ complexity, modern CPUs are likely able to explore all query plans with a modest number of joins (e.g., 10) with a bushy plan. But when System R came out, the hardware was not that efficient, so it still needed to restrict the search space to left-deep trees in general.

2.2 Enumeration Strategy: Bottom-Up Dynamic Programming

Even with a search space restricted to left-deep trees, naively enumerating all combinations of the join orders is still expensive. Instead, System R uses a dynamic programming approach. It realizes that the optimization result for any given subset of the tables in the join can be saved and reused for optimization passes involving

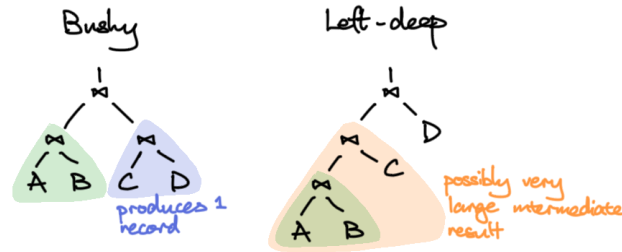


Figure 2: A situation where the joins can be better optimized in a bushy plan. Suppose joining tables C and D produces only 1 record. Then, a bushy plan that joins C and D helps increase the selectivity at the top-most join. The left-deep plan on the right, however, can end up doing lots of wasted work producing a large intermediate result before the top-most join with D .

larger subsets of tables. In practice, it starts with optimizing for size-1 subsets of the tables and iteratively builds up to larger sets of tables until we have a plan for joining all the tables.

This approach is considered a **bottom-up strategy**, in which an executable plan is not obtained until after all passes are complete, whereas a top-down optimizer, such as Cascade, can often produce a valid (though likely suboptimal) working plan much more quickly.

2.2.1 Pass 1: Single-Relation Access Paths

For each individual relation in the query, the optimizer finds the cheapest way to access it. This involves costing out all available **access paths**:

1. **Segment Scan:** A full table scan.
2. **Index Scan:** Using, for example, a B-Tree index to retrieve data, potentially in a specific order or for a specific range of key values.

2.2.2 The “Interesting Orders” Concept

One interesting aspect of the optimizer in System R is that the cheapest access path isn’t always the only plan that one needs to retain to make the dynamic programming search work correctly in this case. Traditional dynamic programming would normally suggest keeping only the lowest-cost plan. However, in query optimization, we can have situations where this is not true.

As a simple example, it is possible that a segment/file scan of table A is the best access method for that table, but if we want to later join table A with B and an index exist on the join key, it may be better to access A via the index to obtain ordered results for a more efficient join, even though it’s not the best access method when A is analyzed in isolation. In the first pass, we would thus want the optimizer to retain both the cheap segment scan and the less-cheap but more usefully-ordered indexed scan for table A .

In the general case, System R retains the cheapest plans for each **“interesting order”**, even if they are more than the cheapest plan where order is ignored. An order is “interesting” if it matches a **GROUP BY** clause, an **ORDER BY** clause, or the join column of a subsequent join.

2.2.3 Pass 2 to N for N -Way Joins

After Pass 1, the optimizer starts to find the best plans for joining two tables, then three, and so on, up to the total number of tables in the query. In each pass, it takes the optimal plans from the previous pass (as the outer relation) and considers joining them with every remaining table (as the inner relation). For each combination, it estimates the cost for different join methods (like Nested Loop and Merge Scan, see Section ??). It again uses dynamic programming and interesting orders: for each subset of relations, it saves

only the cheapest overall plan and the cheapest plan for each interesting order. All other more expensive plans for that subset are pruned, dramatically reducing the search space.

3 The System R Cost Model & Cardinality Estimation

To compare the different plans generated during the enumeration phase, the optimizer needs a quantitative way to estimate their execution cost. System R introduced a cost model that combines I/O and CPU components.

3.1 The Cost Formula

The cost of any plan is estimated using the following weighted formula:

$$\text{COST} = \text{Page Fetches} + W \times (\text{RSI Calls})$$

- **Page Fetches:** This term represents the I/O cost, which is the estimated number of pages read from or written to disk. In the 1970s, I/O was the overwhelming bottleneck, making this the most important part of the formula.
- **RSI Calls:** Used as a proxy for the CPU cost.
- **W:** A configurable weighting factor used to balance the relative importance of I/O versus CPU. This allows the model to be tuned for different hardware configurations.

3.2 Cardinality & Selectivity Estimation

The accuracy of the cost formula relies on correctly estimating the number of tuples (cardinality) processed at each step of a plan. System R's optimizer does this by using statistics stored in the system catalog to calculate a **selectivity factor** for each predicate in the **WHERE** clause. The selectivity factor is the estimated fraction of tuples that will satisfy the predicate.

3.2.1 Catalog Statistics

The optimizer relies on a few key statistics that are maintained for each table and its indexes:

- **NCARD(T):** The number of tuples (cardinality) in relation T.
- **TCARD(T):** The number of pages in the segment that hold tuples of relation T.
- **ICARD(I):** The number of distinct key values in index I.

3.2.2 Selectivity Factor Formulas

System R used a set of heuristics to estimate selectivity, often assuming a uniform distribution of data values and independence in distribution between columns. The formulas used for some common predicate shapes are as follows (where *SF* = selectivity factor):

- **column = value:** The selectivity is estimated as $1 / \text{ICARD}(\text{index})$ if an index exists on the column. If no index exists, a “magic number” like 1/10 is used as a default guess.
- **column > value:** For numeric types, the selectivity is estimated using linear interpolation: $(\text{high_key} - \text{value}) / (\text{high_key} - \text{low_key})$. Otherwise, a default guess like 1/3 is used.
- **predicate1 AND predicate2:** The selectivities are multiplied:
 $\text{SF} = \text{SF}(\text{predicate1}) * \text{SF}(\text{predicate2})$. This assumes the predicates are independent, which can be a significant source of error for correlated columns. For example, `city = 'Pittsburgh' AND state = 'PA'` could be severely under-estimated since these predicates are highly correlated.
- **predicate1 OR predicate2:** The formula follows the principle of inclusion-exclusion:
 $\text{SF} = \text{SF}(\text{predicate1}) + \text{SF}(\text{predicate2}) - (\text{SF}(\text{predicate1}) * \text{SF}(\text{predicate2}))$.
- **not predicate1:** $\text{SF} = 1 - \text{SF}(\text{predicate})$.

The final estimated cardinality for an operation is the product of the input cardinality and the combined selectivity factors of all applicable predicates. This might not be the most comprehensive strategy, but it pioneered cost-based optimization.

4 System R Join Methods

After the optimizer finds the best access paths for single tables, it must determine the most efficient way to join them together. Based on early research into join performance, the System R optimizer primarily considers two main join algorithms.

- **Nested Loop Join:** This is arguably the simplest join method. For each tuple retrieved from the outer relation, the algorithm scans the entire inner relation to find all matching tuples. While straightforward, this can be very expensive.
- **Merge Scan Join:** This method requires both relations to be sorted on the join column(s). The two sorted inputs are then scanned, and the streams are “merged” to find matching tuples. It is fast when one or both inputs are already sorted via an index scan that provides an “interesting order”, thus avoiding the high cost of an explicit sort operation.

The optimizer computes the estimated cost for both methods for each potential join in the plan and chooses the one that is predicted to be cheaper.

The modern hash join algorithm is not in the System R paper. This is because hash joins were not a well-established technique in the 1970s, and their reliance on large in-memory hash tables was impractical given the limited and expensive RAM of that era.

5 Handling Nested Queries

Nested queries are usually slow, as nested evaluation often results in inefficient dependent joins with expensive runtime complexity. It is desirable to flatten any nested queries before sending them to the optimizer, and new methods from Germany [?] handle this problem algebraically.

6 Modern Advancements in Cardinality Estimation

One of the biggest weaknesses of the original System R optimizer was its simple cardinality estimation, which could lead to significant errors and suboptimal plans, especially with skewed data. Query optimization relies on cardinality estimation. If the estimation goes wrong, and one subplan gets wrong information, and the subplan on top of it gets information with more errors, it could lead to a very slow plan at the end of the optimization.

Looking back from 2025, we know that **under-estimation** of cardinality is far more dangerous than over-estimation, as it can lead the optimizer to choose an algorithm like a nested loop join (the optimizer thinks the outer table is small) for what is actually a massive result set.

6.1 Histograms

Instead of assuming uniform data distribution, many modern databases use **histograms** to create a more accurate, non-parametric model of the values in a column¹. The core idea is to partition the values into a set of discrete bins and store statistics for each bin.

Storing histograms can be as simple as storing the frequency and boundary value pairs² for each bucket (see Figure ??). To construct a histogram, one could scan the entire table, but in cases where a full table scan

¹Modern databases often automatically update these histograms in the background.

²we can also store both boundary values, as suggested by the slides

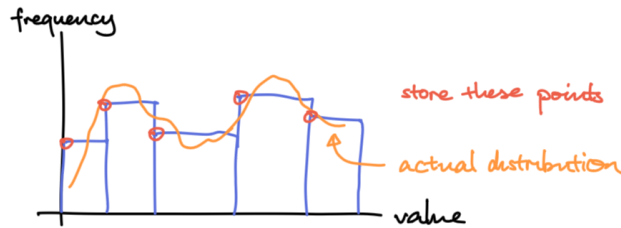


Figure 3: An example histogram.

is expensive, it might be better to sample the data to approximate the histogram. A sample size of about $300 \times \# \text{bins}$ tends to yield good results.

Two common strategies for creating these bins are:

- **Equi-width Histograms:** In this approach, the range of values for a column (from min to max) is divided into buckets of equal width. While relatively simple to construct, this method is not robust to skewed data, as a large number of tuples may fall into just a few buckets while other buckets remain sparse or empty, leading to poor estimates.
- **Equi-depth Histograms (or Equi-height):** Here, the bucket boundaries are adjusted so that each bucket contains approximately the same number of tuples. This method adapts much better to skewed data distributions by effectively allocating more resolution (i.e., more buckets) to denser regions of values. Though more complex to build, it generally produces more accurate estimates.

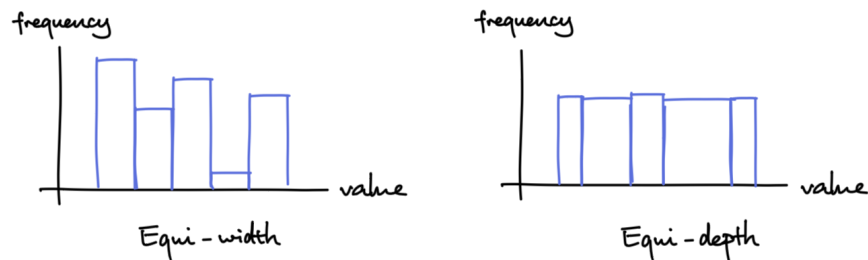


Figure 4: Equi-width vs. equi-depth histograms

A particularly effective and common strategy is to enhance an equi-depth histogram with a list of the **Most Common Values (MCVs)**. The system separately stores the exact frequencies for the most common "heavy-hitter" values, preventing these outliers from distorting the statistics in the histogram's regular bins.

Histograms can also be constructed for intermediate results, such as the output table of a join. This could be done by combining multiple histograms from the catalog.

6.2 Sketches

Sketches are compact, probabilistic data structures that can summarize data properties with very little memory, often in a single pass.

An example of a sketch data structure is the **Count-Min Sketch**. Count-Min Sketch makes use of multiple independent hash functions to hash values into slots and keep track of the count of how many times an element has been hashed into that slot. When estimating the number of times a value appears, we hash the value and take the min count at the slots it hashes to. The idea is that collisions between the hash functions for all the slots the value hashes to will be small, meaning that the minimum count among them is likely to be the true count of that value.

Figure ?? shows a Count-Min Sketch in action. We have three hash functions h_0 , h_1 , and h_2 , and we have added values 1 and 2 to our database and incremented the counts for the corresponding slots accordingly. Notice that a collision took place for h_2 : both $h_2(1)$ and $h_2(2)$ landed in slot 0, resulting in a count of 2

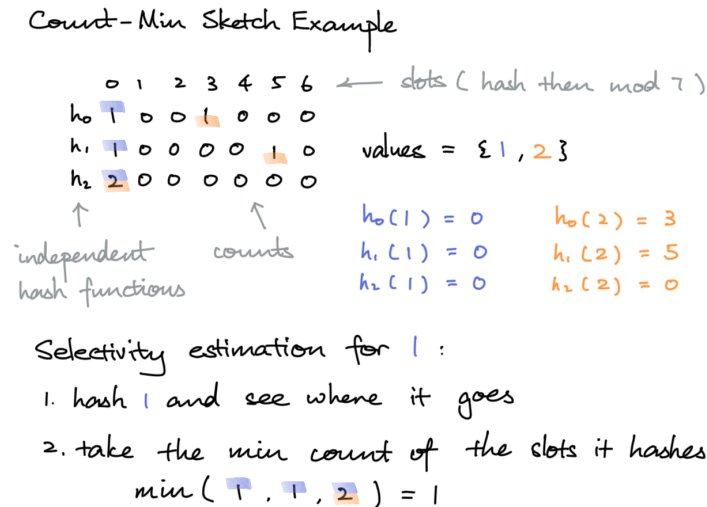


Figure 5: An example of Count-Min Sketch in action involving three hash functions and two values.

on the h_2 row. When querying for the count of, say value 1, we look for the slots 1 hashes to and take the min of the counts. Notice that even though there was a collision, it only happened at one of the three slots, making the min of the counts still correct. It will not be until a collision takes place at slot 0 for the two other hash functions that the count estimation for value 1 will become an overestimate (e.g., if we add value 3 with $h_0(3) = 0$ and value 4 with $h_1(4) = 0$). Given sufficiently spread-out slots, this is unlikely to happen.

An interactive demo is available at <https://hkorte.github.io/slides/cmsketch/#/7>.

6.3 ML-Based Methods

There exist ML-based methods for cardinality estimation, but we will not go into too much detail here. An overview of the two broad approaches is as follows:

- **Data-driven:** train models to learn the distribution of the data in the columns. This often results in more accurate results, but usually requires expensive training and storage (the model could be 10 times the size of the original table).
- **Query-driven:** take past queries and train models on them. However, a drawback is that it might not be easy to obtain sufficient queries for training.

6.4 Pessimistic Cardinality Estimation

Many cardinality estimations can both under- and over-estimate. Arguably, under-estimation is worse because the optimizer can pick a plan it thinks is cheap but is actually more expensive. When over-estimating a good plan, however, the optimizer will likely just pick other plans.

To directly deal with the under-estimation problem, recent research has produced pessimistic estimation techniques that provide a guaranteed upper bound on join cardinality.

6.4.1 A simplified motivating example

To illustrate the idea of pessimistic cardinality estimation, consider a join $R \bowtie S$. Without any other information, we can only write down a $|R \bowtie S| \leq |R| \cdot |S|$ bound. But if we are given that k upper bounds the frequency of values in S , we can improve the bound to $|R \bowtie S| \leq |R| \cdot k$. Of course, we can do better (which we will in later subsections), but the takeaway from this example is that a small amount of frequency information can help tighten our upper bound.

6.4.2 Upper bound using degree sequence

We can achieve better pessimistic estimation by computing a **degree sequence** for each column of interest. A degree sequence is defined as the sequence of frequencies of distinct values in the column sorted in descending order. For example, if a column has values $a_1, a_1, a_1, a_2, a_3, a_3$, its degree sequence is $[3, 2, 1]$, corresponding to the frequency of a_1 , a_3 , and a_2 respectively.

It can be shown that, if we take two degree sequences and line them up, the sum of the product of frequencies along each position is an upper bound for the number of output tuples if we join by the corresponding column. For example, if we want $R \bowtie S$ along column a , and the degree sequences for column a in the two relations are both $[3, 2, 1]$, we can upper bound the output size by $3 \times 3 + 2 \times 2 + 1 \times 1$.

While the full degree sequence can be large, we can always truncate the sequence by keeping the top- k frequencies and assuming the remaining values are as frequent as the k -th frequency. This way, we save on storage but are still guaranteed to never under-estimate the number of output tuples.

6.4.3 LpBound

The LpBound paper [?] takes the idea a step further by computing l_p norms to summarize the degree sequences, where l_p for data is defined by $l_p\text{-norm} = (\sum_i d_i^p)^{(1/p)}$ (d_i being the data). On the two ends, l_1 ends up equivalent to the cardinality of the relation, and l_∞ ends up equivalent to the max-degree. Between l_1 and l_∞ , we get sequences that roughly correspond to the degree sequence curve in different dimensionalities. In practice, LpBound stores the l_1 -norm, the l_∞ -norm, and a small subset of other l_p -norms in between.

Then, LpBound does some clever math where they put the l_p -norms into Shannon's inequalities as entropy values to obtain a set of equations that, when solved using an LP solver, yields a reasonably tight upper bound.

Lecture #3: The Extensible Cascades/Volcano Query Optimizer

15-721 Advanced Database Systems (Fall 2025)

<https://www.cs.cmu.edu/~15721-f25/>

Carnegie Mellon University

Notes by: Akash Nayar, Ramsey Boyce

1 Introduction

As a declarative language, SQL doesn't expect the user to define how a query is run. Rather, users can define the output they wish to obtain, while the actual method by which this data is collected and shaped into the desired output is abstracted away. With no clear explanation on how to execute a query, most DBMSs will employ a *query optimizer* to convert the input query into an actionable and sensible plan.

2 Query Optimization

Two different query plans that always produce the same exact output are considered to be “logically” equivalent. Formally, a query optimizer evaluates logically equivalent plans and attempts to select the cheapest one. Examples of logical plans are $R \bowtie S$ or simply just T . How these plans are actually conducted is defined by the physical plan, e.g., HashJoin(R, S) or TableScan(T).

Selinger's paper on the System-R optimizer marked a huge advancement in query optimization. By only examining left-deep trees, the search space was narrowed from factorial in the number of joins to exponential. System-R also employed a cost estimator to roughly yet effectively weigh different plan options. Furthermore, it employed a bottom-up approach, whereby it started at the lowest nodes in the tree, finding the cheapest ways to execute subplans such that the outputs favored higher-level operators. In a sense, by the time the optimizer got to figuring out how to join inputs R and S, it would already have candidates for the cheapest ways to obtain both R and S.

However, the System R optimizer was not easy to adapt to new operators and features in the SQL language. As such Volcano and Cascades set out to not only be “extensible,” but also demonstrated a top-down and rule-based approach to traversing the search-space.

3 Volcano/Cascades Core Concepts

3.1 Overview

“Optimizer generator”: Optimizer reads a set of rules and explores plans by matching on those rules.

Rules come in two types:

- Logical transformation: equivalent SQL, e.g., join associativity
- Implementation: logical to physical operator mapping, e.g., inner join to hash join

Expressions can be logical or physical:

- Logical: $R \bowtie S, \{R, S\} \bowtie T$
- Physical: HashJoin(TableScan(R), TableScan(S))
- A logical expression can have many corresponding physical expressions.

Expressions have logical and physical properties:

- Logical: properties of the expression regardless of the physical expression, e.g., output cardinality
- Physical: properties associated with the physical implementation, e.g., sort order, number of execution threads used

Enforcers: a class of physical operators that enforce physical properties

- Have a similar purpose to System R's interesting orders

Need some plan enumeration order, and need cost-based optimization (histograms/sketches/etc).

3.2 Memo Structure

Volcano defines the notion of a group, which is simply an abstraction for a set of logically equivalent expressions. For instance, the group $\{R, S\}$ (a join between R and S) contains subgroups $\{R\} \bowtie \{S\}$ and $\{S\} \bowtie \{R\}$. In exploring how to optimize $\{R, S\}$, the optimizer will have to traverse the tree of its subgroups and evaluate them as well.

In order not to recalculate the optimal query for a given subgroup, Volcano employs a Memo structure. Each entry in the Memo table maps a group (e.g. $\{R \bowtie S\}$) to its already computed equivalent logical and physical expressions:

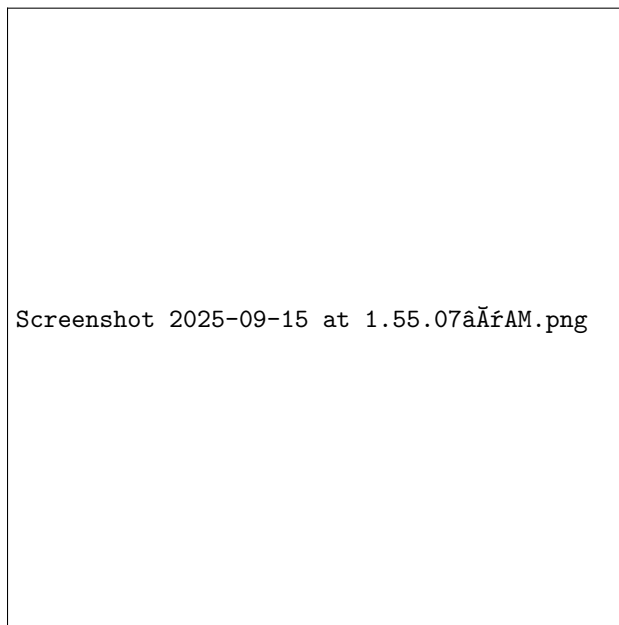


Figure 1: Memo Structure Example

Figure 1 gives an example of this structure. The first group, $\text{Join}(A, B)$, points to its already computed and costed logical and physical expressions. As such, when the optimizer sees this subgroup, perhaps while evaluating a different logically equivalent version of the same query, and it sees Group 1 as a subgroup, it can simply reuse these existing logical and physical plans.

3.3 Volcano Query Optimization

Volcano was an initial top-down query optimizer. It features a generation phase, which applies transformation rules to enumerate all logical plans, using the memo structure to avoid examining duplicates. Then, there is a cost analysis phase, which applies implementation rules to generate physical plans and picks the one with the lowest cost. This was an innovative paradigm, but was further improved in the Cascades system.

4 Cascades Query Optimization

4.1 Overview

Mix the generation and cost estimation phases.

- Volcano exhaustively searches logical space before examining physical costs
- Cascades can find a good physical plan without this potentially-costly step, handling low-resource situations better

Improved guidance abstraction.

- Cascades can efficiently reduce the search space by applying heuristics for certain join graphs
- For example, a star join graph is optimally handled by a left-deep plan
- Certain rules can be deactivated

Switch from recursive calling to a task-based model.

- Avoids potential stack overflow for big plans
- Allows more control over the search direction, e.g., derive an expression's best plan after its inputs have been derived
- Allows parallelization of optimizer
- Can control task scheduling, e.g., LIFO stacks are often used to keep track of the tasks to work on, but this can be changed to FIFO

4.2 Tasks

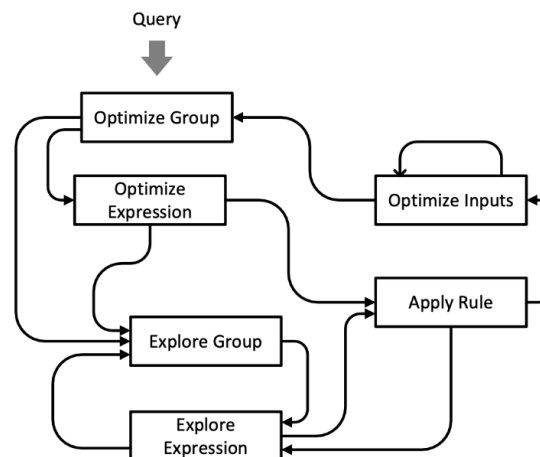


Figure 2: Cascades optimization flow

Cascades creates six different types of tasks, as shown in figure ???. In summary:

- Optimize group: generate the best physical plan for a group
- Optimize expression: generate the best physical plan for an expression, using guidance to constrain the moves taken
- Explore group: generate logical expressions for a group
- Explore expression: generate logical transformations for an expression, following the promise order and using guidance to skip some rules
- Apply rule: apply a rule to an expression, adding the results to the memo table sorted by promise
- Optimize inputs: generate the best physical plan for an expression, updating the memo with the best plans (including visited sub-expressions)

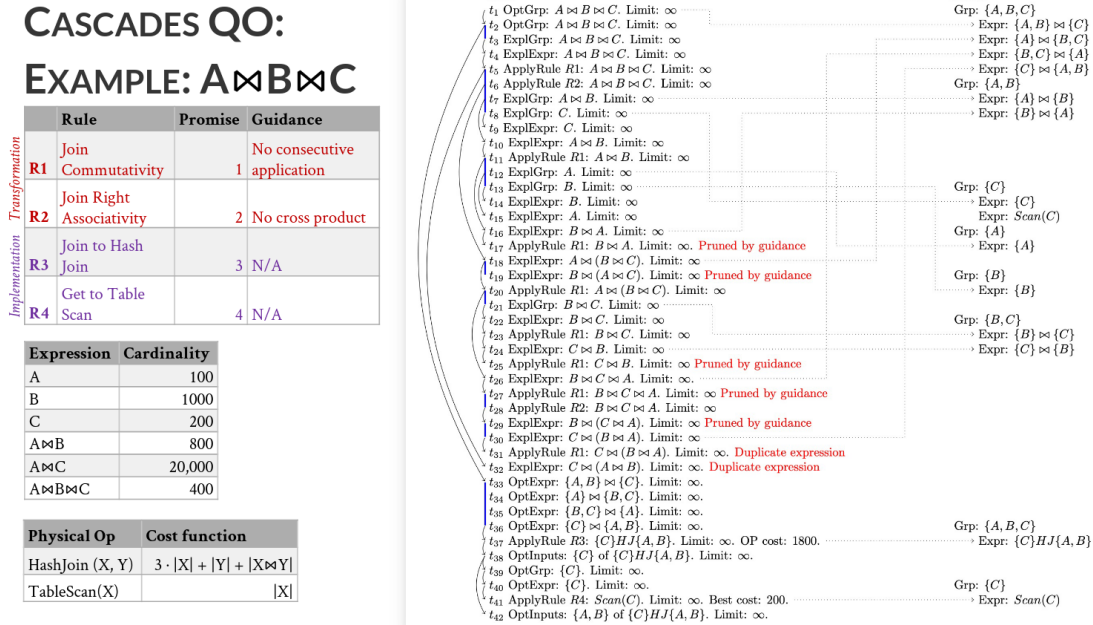


Figure 3: Example Cascades execution

4.3 Example

Figure ?? shows an example execution of Cascades using LIFO task scheduling. Note that some of the tasks are pruned by guidance (e.g. if the commuted version of the join was already considered) or pruned due to already being in the memo table.

Cascades stores a bitmap to keep track of which rules have been applied to an expression. Logical expressions only care about transformation rules and physical expressions only care about implementation rules. It's also normal to mix transformation and implementation rules, unlike Volcano.

4.4 Improvements

The search can be improved with simplification rules, which are always good to do. Examples include predicate pushdown, duplicate predicate elimination, etc.

Macro rules can be applied to transform the shape of an expression in one step. This is useful for data in a star schema.

Parallel query optimization should be done on modern high-core machines.

- The task-based system provides a framework to parallelize, but still needs to coordinate and track task dependencies.
- Processor parallelism is growing and projected to continue growing.
- Can use parallel operators (the exchange operator) and use enforcers to change the degree of parallelism.

Multi-stage optimization should be applied in modern workloads to handle both OLTP and OLAP cases.

- In OLTP systems, the optimal plan can often be simple and easy to find, and most of the query optimization should be skipped.
- To handle all cases, split the optimization into many stages and only do the full optimization in the last stage.

5 Summary

Top-down query optimization offers an alternative method compared to the Selinger bottom-up approach.

- Based on rules that transform the query.
- Also uses dynamic programming, with the memo table.
- Can use the task scheduling and guidance to change the manner in which the search space is explored.
- Both methods rely on good cost estimation.

Handling nested queries requires special transformation rules, but they are easier to add in a top-down optimizer.

Many open problems:

- Efficiency
- Extensibility
- Explanability

Can the optimization occur during query execution instead?

Lecture #4: Plan Stitch

15-721 Advanced Database Systems (Fall 2025)
<https://www.cs.cmu.edu/~15721-f25/>
Carnegie Mellon University
Notes by: Drew Ripberger

1 Wrapping up Cascades

One of the biggest differences between the Cascades style optimizer and the previous Volcano style is how Cascades tracks rules and transformations that need to be explored. Cascades replaces the deep function recursion in Volcano with a stack of Tasks. This task abstraction is a unit of work that represents doing exploration, doing a rewrite, etc. With this new method, we now also track the search space:

- Bitmap associated with every expression
- Keep a bitmap of which rules were applied so that we don't redo a previously applied rule
- We have two types of expressions, logical and physical. Logical rules care about transformations, and implementation rules care about implementations.

This has a number of benefits, including presenting a better opportunity for parallelism and avoiding the possible stack overflows that can occur in the Volcano algorithm when you have complex plans.

We can then further improve upon the cascades search with a few additions and modifications. For instance, we can introduce:

1. Simplification Rules
2. Multi-stage Optimization
3. Macros
4. Parallelism

Simplification rules introduce some basic observations of experts into the optimization. They are rules that we will *always* apply if they are seen initially. We do this because we have some prior knowledge that they are likely to be effective, and will help push the optimizer into a good path quickly. These rules can include removing duplicate predicates or pushing down simple predicates. Importantly, they take priority over the full Cascades optimization pass, because we suspect they are almost always good rules to apply, and thus help us to get a good plan early.

Along a similar idea, once we examine the initial query, we can make a conscious decision on how much optimization we should do. Theoretically, if optimization were free, we should always do it, but for simple queries that likely don't have much optimization potential, there will be little payoff for the cost of running the optimization phase. Different engines can decide on rules for how many stages of optimization should be done, depending on the query. For instance, OTLP workloads tend to have simpler queries and are more likely to be bottlenecked by indexes rather than complex joins, so in this instance, maybe you forgo the extra latency of the query optimizer. If in the end, you still aren't confident that the query is simple or you can't easily simplify the query, then do the full optimization pass.

We also have this idea of a *macro* which would combine multiple transformations for special patterns you may see in the query. This is again particularly effective because you can find a better plan quicker by combining multiple transformation steps. On top of this, we can leverage the stack of Tasks to introduce parallelism to explore the tree of options. This needs to be done carefully as some exploration steps are dependent on each other.

Cascades also introduces the idea of parallel query operators. You can model the degree of parallelism as an enforcer. Similarly, you can use the exchange operator to model distributing data across nodes.

This is still a top-down dynamic programming approach compared to Selinger's System R. Still, regardless of the optimizer scheme, good cost estimation is essential.

2 Plan Stitch

2.1 Backdrop

DBMSs can autotune based on your workload to achieve better performance. They can do this by creating indexes on commonly accessed attributes or, conversely, dropping them if their attributes are rarely accessed. They can even change the ways that we partition data.

In general, OLTP systems are a bit more complicated in terms of this index creation. They gain performance in terms of search, but suffer when deleting.

This autotuning is fantastic, but workloads aren't static, they change!

In addition to the autotuning schemes described above, we can also autotune our query plans. In particular, we can deal with performance regressions for the same queries due to changes in the query plans. We can detect performance regressions when a query we have issued before uses a new plan that results in worse performance. We do this by comparing the new plan's performance against that of all (or a subset) of the old plans. This type of regression is common in real workloads. This can happen for a number of reasons, maybe the query optimizer itself changed, or maybe the statistics changed significantly. Cloud environments provide a new opportunity to detect these types of regressions because we have so much data on the queries, however, this should also present them with more opportunities to optimize them!

The question becomes: should I use the new plan I've generated, or revert to an old plan to avoid regression? The Plan Stitch paper asks if you can instead stitch together the best parts of each plan. Therefore, we can benefit from having lots of data about our previously tried plans. Though searching through all of the previous plans can be expensive in itself.

2.2 Approach

This is basically a query optimization problem, where we are exploring the space of sub-plans of the previously used queries. The restriction is that we only use physical operators and logical expressions present in the past plans. We construct this query optimization problem using an AND-OR tree. Within this tree, the OR node says to optimize and pick the best of the children. The AND node says to optimize *all* of the children (they are all needed as input).

To start, you walk up the leaf nodes (which are access paths), stitching together equivalent expressions. The complexity comes from finding the expressions that you can unify with OR nodes. This construction is finished when you reach the root OR node. Now, the problem is the classic optimization problem. You search down the tree and make decisions at the OR nodes, deciding which method is the cheapest. However, there are some nodes for which you might not know the cost at this point. Some costs might be invalid in this configuration, or some might just be sufficiently old that they should be re-costed. Regardless, some nodes will **have** to be re-costed. Our search tree can be (and needs to be for correctness) pruned by removing any nodes associated with access paths that are no longer valid. For instance, if an index is dropped some time between an old plan and the most recent plan, we can't consider that index within the search space, even though it was valid at the time of the old plan.

This process attempts to bypass the query optimizer altogether, since it is essentially redoing some of the optimization. It does this with plan hints. Plan hints are comments or suggestions that nearly all database systems support. They allow you to annotate queries with hints or even full query plans. They can be used to specify join orders, join methods, access paths, or even degrees of parallelism. This is the mechanism used to potentially skip the stock optimization phase. Also note that the Cascades macro-rules can also be implemented in this way.

2.3 Results & Conclusion

The Plan Stitch method shows consistent performance benefits, especially in customer workloads. They found that the biggest changes that Plan Stitch was making were at the leaf level. Meaning that the key optimization that Plan Stitch was finding was the access path for scanning the data. Also, based on their specific workloads (the customer workloads and TPC-DS benchmark), they found that a few plans required stitching together more than 6 plans. This is beneficial because the number of plans included in the search

is one of the main factors in the search cost.

A related idea to this is mid-query re-optimization. It uses the same idea of using history to inform better plans. Specifically, it stores the original estimates from a query plan and then compares those estimates against what actually happened when executing the query. These differences can then be used to inform future query plans, as we have learned about the error between our estimates and reality. To extend this further, we could even pause the query execution mid-flight. If we learn more information that affects our prior assumptions used during the optimization phase, we should inform the optimizer and request that it re-optimize with this more accurate information.

To conclude, the main takeaway is to start thinking about workloads as a whole, rather than just their single component queries. These workloads often contain many repeated queries that we can take advantage of. The Plan Stitch paper outlines a specific method of how to take advantage of this repetition. The next open question is: what optimizations/adaptations can we perform *while* running the query?

Lecture #5: Architecture-Conscious Query Processing

15-721 Advanced Database Systems (Fall 2024)

<https://www.cs.cmu.edu/~15721-f24/>

Carnegie Mellon University

Prof. Jignesh Patel

Note authored by Sriya Ravi

1 Introduction

In the 1970s and 80s, database systems were primarily designed as **single-node systems**, with performance largely driven by improvements in hardware, specifically CPU processors. The rise of Moore's Law, where processor speeds doubled every two years, meant that database vendors could achieve significant performance gains "for free" - simply by recompiling their software to run on new hardware. This architectural simplicity allowed databases to evolve alongside processors, but as data volumes grew, keeping up with performance demands became increasingly challenging.

By the late 90s, **processor caches** started to appear, offering significant performance improvements for systems that could take advantage of them. Jim Gray, a pioneering figure in database systems, observed that **memory access patterns** were not optimized for this new hardware.

Sorting, which was treated as a key benchmark of performance in databases, was one area where specialized algorithms like QuickSort and replacement selection merge trees began to show substantial improvements due to their **cache-conscious designs** leveraging sequential access workloads and good spatial and temporal locality. A key optimization involved reducing memory footprint by avoiding the duplication of entire records during sorting passes. Rather than copying full records, these techniques moved only compact key-prefixes or pointers during the first sorting pass. This approach minimized the amount of data being transferred and reduced cache misses, dramatically increasing performance. For example, keeping just an 8-byte pointer to the full record instead of the entire record during sorting operations allowed the system to work within the confines of processor caches more efficiently. These changes, introduced by Gray and others, opened the door for more **architecture-conscious** data processing techniques, pushing the field to develop systems that are more mindful of the underlying hardware.

2 Modern Processor

Around the mid-2000s, advances in clock speeds stalled, with processors remaining in the 2-3 GHz range, as further increases generated excessive heat. This was due to the shrinking of transistors, which concentrated heat in smaller areas, risking overheating and melting. Modern CPUs have multiple cores, and the focus has moved away from increasing clock speeds toward optimizing for parallel execution and multi-threaded performance.

This evolution necessitates a new approach to database system design, one that leverages multi-core architectures and compute fabrics to handle ever-growing datasets efficiently. Modern DBMSs must prioritize memory-efficient execution engines, leverage sequential access patterns, and embrace architecture-conscious techniques to maximize performance on modern hardware.

2.1 Modern Processor Architecture

In modern processors, the architecture design reflects the transition in processor architecture from relying on increasing clock speeds to focusing on multi-core and heterogeneous core systems. However, not all cores can be powered simultaneously due to thermal and power constraints, leading to a division into e-Cores (efficiency cores) and p-Cores (performance cores).

- **e-Cores:** Optimized to run at lower speeds while consuming less power and generating less heat, making them ideal for workloads that don't require maximum performance.
- **p-Cores:** Deliver higher performance but generate more heat, meaning their use must be carefully scheduled to avoid damaging the chip.

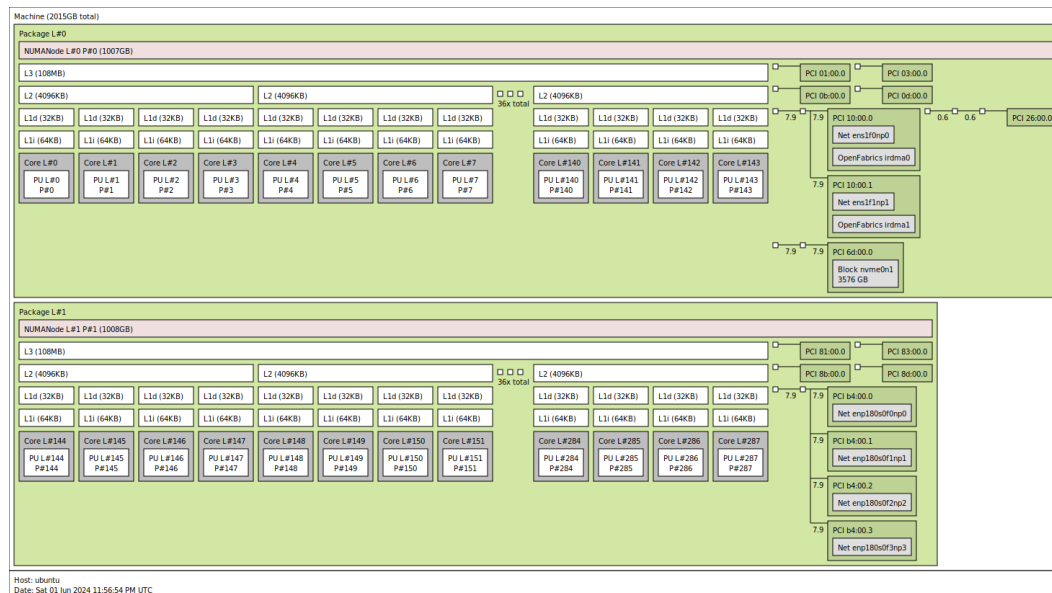


Figure 1: 2p Intel Xeon 6780E Supermicro Topology.

The **Intel Xeon 6780E**, depicted in Figure 1, employs a dual-socket architecture housing a combined 144 e-Cores. In this architecture, each core has access to the L1 caches, which are divided into the L1 instruction cache and the data cache. The L1 instruction cache ensures that frequently accessed instructions come from the cache rather than the slower DRAM, while the L1 data cache stores frequently accessed data.

Small registers and progressively larger caches provide faster access to data, but with significant performance drops as you move further down the hierarchy. Groups of four cores (**islands**) share and compete for the L2 cache - a slower but larger cache layer compared to the L1, which reduces the latency of accessing main memory (DRAM). Accessing the L1 cache is fast, but the L2 incurs a 2-3x slowdown, the L3 cache adds a 10x drop, and accessing the DRAM comes with a 100x penalty. Many databases today keep their hot data sets in memory, which is still slower than cache, but much faster than disk. However, caches are small, and managing locality—ensuring frequently accessed data stays in the faster caches—has huge performance payoffs.

New technologies, like AMD’s **chiplet architecture**, increase the L3 cache size to gigabytes by adding large caches using a new chiplet technology, but trade-offs in locality and speed remain, making the cache hierarchy a critical factor in maximizing processor performance. These cores can also run **hyperthreads**, allowing two threads to time-multiplex on the same core, sharing resources like the cache.

Note: Despite the advantages, hardware sharing, especially with caches, opens up vulnerabilities to **side-channel attacks**, where attackers exploit shared resources to infer private data. To mitigate this, Intel has incorporated hardware-based protections in its newer architectures.

Note: The hardware landscape is increasingly heterogeneous, with different processor types (e.g., GPUs, TPUs, tensor processors) coexisting. As cloud environments become more disaggregated and dense, compute power must be utilized efficiently across various processor types. Vendors like NVIDIA release processors that are more expensive but offer significant price-performance improvements for specialized tasks like generative AI. Meanwhile, older processors from previous generations often end up idle, prompting companies like Google to explore reusing them for monetizable workloads like BigQuery. This trend highlights the increasing need for databases to adapt and compile efficiently on a wide range of processors.

2.2 Translation Lookaside Buffer

In the 1960s, the industry moved away from writing programs that directly accessed physical memory, introducing the concept of virtual memory to simplify memory management and allow for virtualization. In modern cloud environments with many workloads, Direct Memory Access (DMA) becomes further impractical because memory must remain virtualized, making virtual-to-physical address translation necessary in shared, multi-tenant infrastructures. Each process operates within its own virtual address space, composed of virtual pages that must be translated to physical page addresses mapping to the DRAM.

This translation is handled by the Operating System, with hardware assistance from the **Translation Lookaside Buffer** (TLB). Without hardware support, every memory reference would require a page table lookup, but page tables are large, often gigabytes in size, and do not fit into the cache, making DRAM access for translation extremely slow. The TLB acts as a specialized **fully-associative cache**, storing mappings of a small number of frequently accessed addresses to avoid these slow lookups, working in tandem with the processor.

TLBs have limited capacity and are becoming multi-leveled, similar to L1 and L2 caches. If a TLB miss occurs as in Figure 2, it forces a DRAM access to retrieve the mapping from the page table, which is very expensive. To optimize performance, algorithms should aim to access memory in a page-friendly manner, minimizing random memory accesses. For example, accessing 64 pages at a time in sequence can greatly reduce the overhead of loading pages into the TLB. This is especially useful for data structures like hash tables or loop-based operations, where aligning memory accesses to fit within the TLB's capacity improves performance.

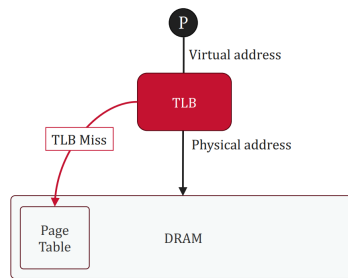


Figure 2: TLB miss requires an expensive call to Page Table in DRAM.

3 Query Execution Processing Models

3.1 Traditional Iterator Model

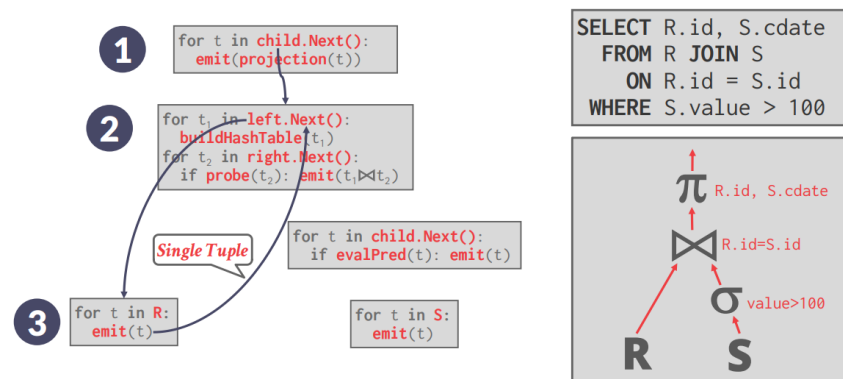


Figure 3: Iterator Model with tuple-at-a-time processing.

The **iterator model**, also known as the Volcano or Pipeline model, consists of three main phases: **Open**, **Next**, and **Close**. For instance, in a hash join, the Open step involves creating a hash table, while the Next function iterates over the child nodes to probe the hash table. Finally, the Close step deallocates memory when all records have been processed.

Query plan operators in an iterator model are easy to reason about because each operator can be implemented independently from their parent or child operators in the query plan tree as depicted in Figure 3. Each tuple is therefore processed up the plan as far as possible before the next tuple is retrieved. This is useful in disk-based systems because it allows us to fully use each tuple in memory before the next tuple or page is accessed.

However, in a traditional **tuple-at-a-time execution**, each tuple move causes significant cache inefficiency, due to constant memory movement between operators, as each tuple is moved separately through the query plan and the 64-byte cache lines of memory are often underutilized. By the time a record is moved between different operators, much of the cache may be flushed, causing frequent memory fetches and poor cache locality.

3.2 Vectorization Model

In the vectorization model, each operator, like in the iterator model, implements a Next function. However, instead of emitting one tuple at a time, operators emit batches of data as depicted in Figure 4. Therefore, the unit of transfer between operators is a **batch** rather than a tuple. The internal loop of the operator is optimized to process these batches rather than individual tuples, which reduces function call overhead. This helps maintain higher cache locality by maximizing the data resident in the L1 / L2 caches. This reduces the frequency of data movement between caches and memory, significantly enhancing performance.

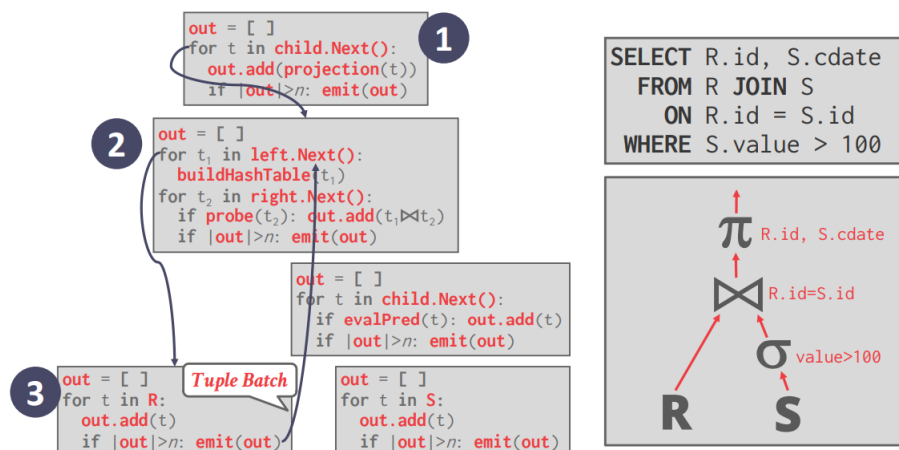


Figure 4: Vectorization Model with batch processing.

Vectorized execution, combined with SIMD, is becoming a standard practice in modern databases to handle the ever-growing need for efficient data processing, particularly for large analytical workloads. In columnar databases, the vectorization model efficiently processes batches of columns, and entire column materialization can be done either eagerly or lazily. Operators leverage **SIMD** (Single Instruction, Multiple Data) instructions, which allow multiple data elements to be processed simultaneously, improving cache locality and minimizing data movement between processor caches and memory. For instance, Intel's AVX-512 instruction set can process 512 bits of information at once, providing a massive performance boost with only minor changes in query engine code.

4 Other ways to achieve high performance on modern processors

4.1 Query Compilation

Systems like Hyper and Amazon Redshift use codegen methods to compile a query to execution code and they then run that code. For example, Hyper handles queries by generating an execution plan and breaking it down into pipelines that can stream records across. Each pipeline may then be replaced with a **custom operator**. For example, consider the inner query in Figure 5, which selects rows based on a condition, performs aggregations, and then joins the records with another relation. Instead of having multiple operators reading, writing, and copying records between function boundaries, the query compiler generates a tightly optimized, hand-coded program within pipeline boundaries. This approach minimizes data movement and creates tighter loops, enabling a single, custom operator to handle the entire query efficiently at runtime.

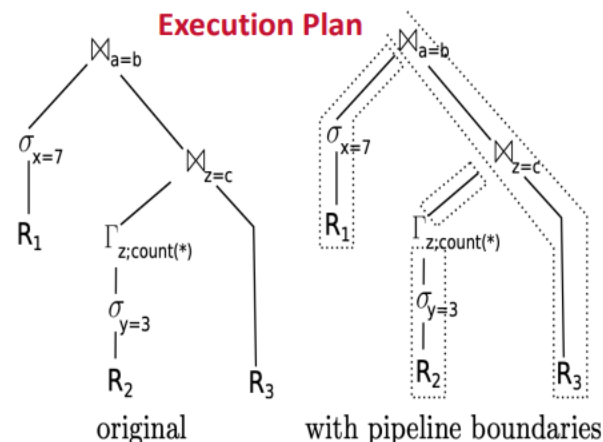


Figure 5: Execution Plan with optimized, hand-coded program within pipeline boundary.

This technique involves **code generation** (code-gen), where specific parts of the query are compiled at runtime, incurring the compiler cost during execution. Additionally, caching assembly code for frequently used query plans allows for faster execution when similar queries repeat - this is a strategy Redshift actively employs. Transforming SQL queries into intermediate forms and using tools like **LLVM** to generate optimized code offers substantial performance gains.

Note: For further optimization, systems can use a hybrid approach, generating code for specific queries upfront while also replacing certain templated common patterns (e.g., scans followed by selections or projections) for other queries when the query is run. However, maintaining both a code-generation engine and traditional vectorization execution can be challenging.

4.2 Early vs Late Materialization in Column Stores

The decision of when to materialize tuple data depends on the execution strategy chosen:

- **Eager materialization** involves projecting and materializing all required values upfront, even before they are consumed by an operator later in the query plan.
- **Lazy materialization** defers the process of materializing data. Instead of immediately producing the full set of values, it passes only position identifiers or record IDs until the consuming operator specifically requires the data. This method can significantly reduce I/O traffic and memory usage, particularly when only a small subset of the data is needed.
- **Hybrid materialization** implemented by systems like Vertica, strikes a balance between the above two strategies. Vertica employs specific rules based on real-world customer workloads as illustrated in Figure 6. It always eagerly materializes the “inner” or build side of a join, as smaller data sets are easier to handle upfront. For larger output datasets, late materialization is preferred to reduce I/O and memory overhead until the moment the data is truly required. However, if the query plan involves operations like repartitioning the result set, then early materialization is often more efficient,

as it simplifies downstream processing by avoiding delays during data transfer or transformation.

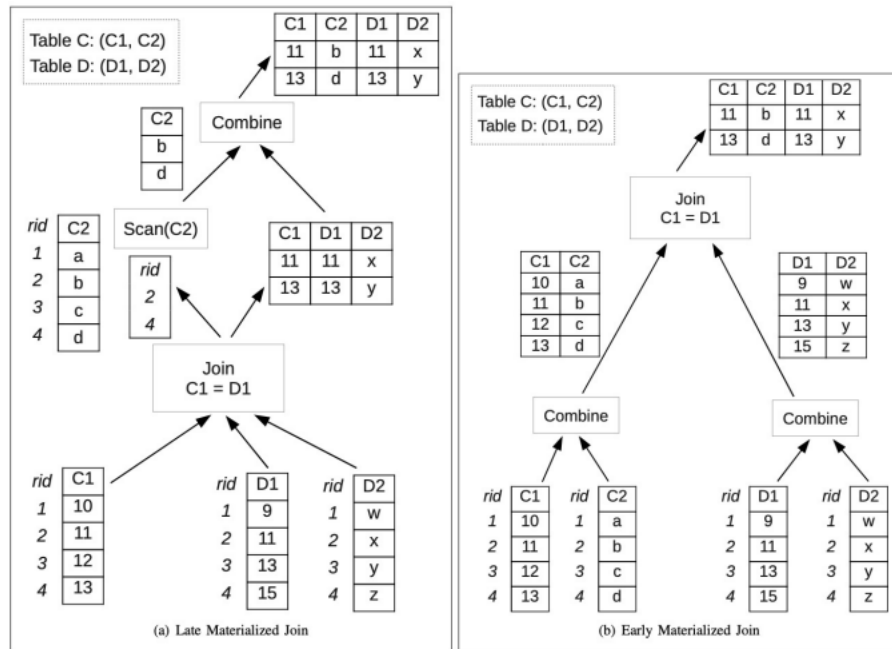


Figure 6: Late and Early Materialized joins in Vertica for query "SELECT C1, C2, D1, D2 FROM C, D WHERE C1 = D1".

5 Conclusion

Vectorized query execution has become the default approach in modern data platforms due to its ability to exploit processor cache hierarchies and efficiently manage data movement between operators. This model significantly enhances performance by keeping data in memory, making it ideal for OLAP workloads. Another key advancement in leveraging modern processors is runtime query compilation, where the query is translated into executable code. While this method can introduce a high code generation cost, it may be worthwhile for long-running queries.

Moreover, as modern systems increasingly operate in a heterogeneous hardware environment—with CPUs, GPUs, and specialized tensor processors—there is a growing need for an intermediate language or **domain-specific language** (DSL) that can translate queries across various hardware architectures. This evolution aims to balance performance across different processors, making query execution more adaptable and efficient in complex hardware ecosystems.

Lecture #6: Architecture-Conscious Join Algorithms

15-721 Advanced Database Systems (Fall 2024)

<https://www.cs.cmu.edu/~15721-f24/>

Carnegie Mellon University

Note authored by Calvin Yang

1 Introduction

Equi Joins remain one of the most important operators for database systems, yet are often one of the most expensive for analytic queries. When discussing which join algorithms are the most used today, there are two key varieties:

- **Sort-Merge Join (SMJ)**
- **Hash Join (HJ)**

The ongoing evolution of hardware architecture presents both challenges and opportunities for these join algorithms. To understand their performance characteristics, we must consider several key factors in modern CPU design:

1. Multiprocessor architectures
2. Multi-socket configurations
3. Complex memory and I/O hierarchies

These hardware advancements continually reshape our understanding of what constitutes the "optimal" join algorithm. The notion of a universally "best" algorithm is elusive, with performance characteristics shifting alongside hardware developments.

1.1 Historical Perspective on Join Algorithm Performance

To illustrate the dynamic nature of join algorithm performance, we can show several key studies evaluating the performance of various join algorithms and their findings:

Study	Key Finding
Intel (2009)	Predicted SMJ superiority over HJ with SIMD instructions; prediction did not materialize.
Wisconsin (2011)	Demonstrated effectiveness of having no partitioning phase by using a shared hash table with fast latches on single-socket CPUs. Takes advantage of cache hierarchy for expensive probe phase by having hot keys pulled to top-level processor cache. Works well for skewed data.
TUM (2012)	Found optimized SMJ outperforms HJ on Non-Uniform Memory Access (NUMA) systems.
Canadians and Swiss (2013)	Improved cache efficiency of Wisconsin HJ; noted superior performance of parallel radix hash join.
TUM (2014)	Demonstrated superiority of non-partitioning approach over parallel radix hash join.
IBM (2014)	Emphasized the importance of concise hash tables.

Table 1: Chronological overview of join algorithm performance studies

As shown by Table 1, research findings often present divergent and seemingly contradictory results. This underscores the complexity of join algorithm analysis and the need for a nuanced understanding of how these algorithms interact with evolving hardware architectures.

1.2 The Challenge of Non-Linearity

A critical aspect of understanding join algorithm performance on modern hardware is the concept of **non-linearity**. Even within a single computational node, several hardware characteristics exhibit non-linear behavior:

- Memory access latency
- Memory bandwidth
- Core-to-core communication

These non-linear relationships add layers of complexity to algorithm design and performance analysis.

2 Non-Linearity in Modern Hardware

2.1 Memory Access Latency

In modern database systems, it is crucial to optimize algorithms to minimize cache misses and TLB (Translation Lookaside Buffer) misses. The relationship between memory access patterns and latency is non-linear, as illustrated in Figure 1.

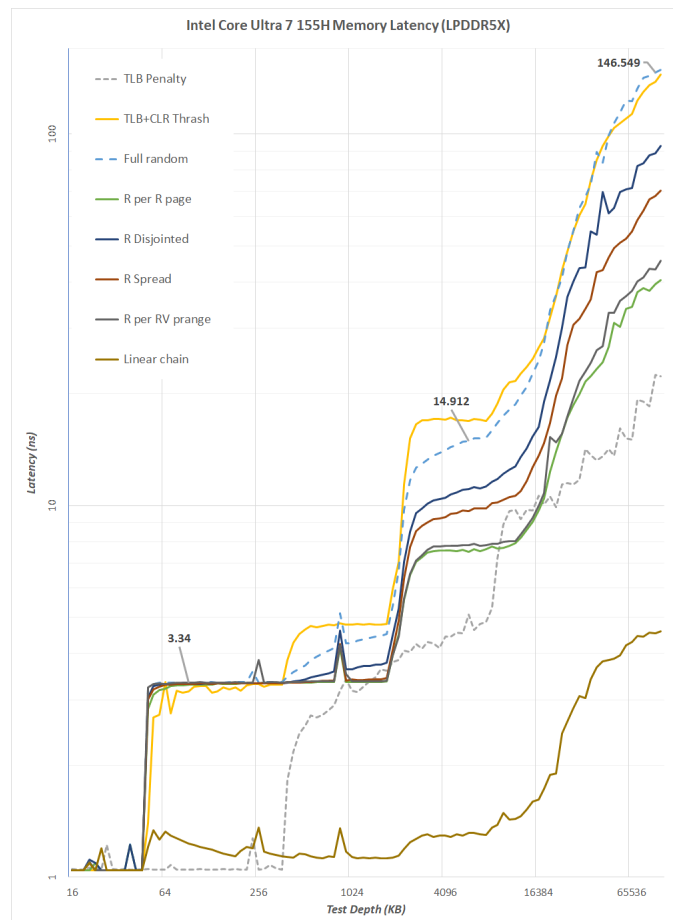


Figure 1: Non-linear memory access latency as test depth increases on an Intel Core Ultra 7 155H processor

As the test depth increases, memory accesses in the program spread to a wider range of memory space. Significant latency increases occur when hitting cache boundaries. A huge jump occurs around 64KB which indicates that the test depth is exceeding the L1 cache boundary. A similar spike occurs after 1024KB which

is the next boundary for L2 cache. By shaping algorithms to keep data in higher-level caches, we can make a substantial difference in performance by avoiding the memory latency spikes at the cache boundaries.

Additionally, it is important to pay attention to TLB behavior and design algorithms with high locality in memory accesses. Ensuring that an algorithm accesses memory locations within fewer pages helps prevent TLB misses, which can be very expensive.

2.2 Core-To-Core Latencies

Modern processors are highly parallel, introducing another dimension of non-linearity in the form of core-to-core latencies. Figures 2 and 3 illustrates this concept.

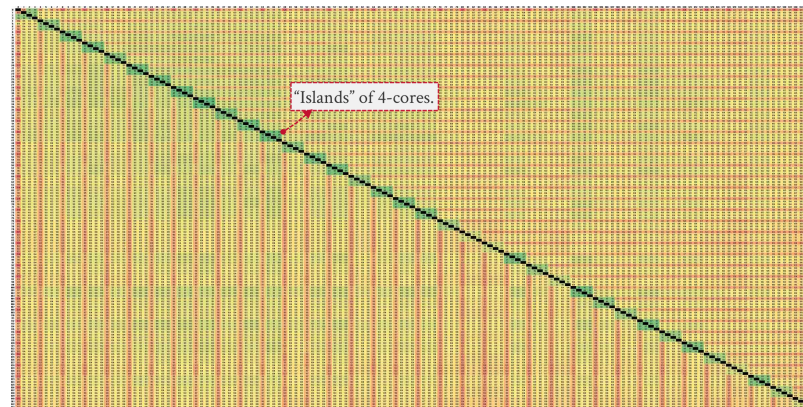


Figure 2: Non-linear core-to-core latencies on an 144 Core Intel Xeon 6780E

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
0	x	5.0	45.2	50.2	51.9	50.1	50.1	51.2	50.9	54.5	35.4	35.5	35.7	35.3	37.2	37.3	37.3	38.1	38.1	38.1	132.3	122.4
1	5.0	x	50.4	50.1	51.9	51.0	51.5	47.6	52.2	52.4	35.5	35.5	35.7	35.7	37.4	37.4	37.8	37.9	38.0	38.0	128.8	135.9
2	45.2	50.4	x	66.0	66.0	65.9	74.5	75.2	75.2	74.4	45.8	49.2	50.1	49.2	52.4	50.7	52.0	53.2	53.8	50.4	132.1	129.3
3	50.2	50.1	66.0	x	65.7	66.0	74.7	74.5	75.4	74.4	49.2	47.3	49.5	51.2	46.9	45.7	52.8	52.4	53.2	52.9	115.4	135.7
4	51.9	51.9	66.0	65.7	x	65.7	74.8	74.8	75.1	75.0	47.6	49.3	50.4	50.7	45.4	50.4	50.9	54.5	52.1	53.1	119.0	132.3
5	50.1	51.0	65.9	66.0	65.7	x	74.8	75.0	74.5	75.4	47.3	47.8	49.8	51.2	52.9	50.0	52.0	52.8	52.5	53.0	126.7	123.3
6	50.1	51.5	74.5	74.7	74.8	74.8	x	59.9	59.9	58.0	50.0	50.3	50.2	50.8	50.4	51.6	50.5	52.2	51.7	47.0	119.4	125.9
7	51.2	47.6	75.2	74.5	74.6	75.0	59.9	x	59.5	58.2	46.4	49.1	50.3	50.9	51.1	49.0	52.0	51.5	53.3	50.7	122.9	130.6
8	50.9	52.2	75.2	75.4	75.1	74.5	56.9	59.5	x	57.9	48.7	50.2	45.3	51.2	53.0	51.7	52.8	53.8	54.8	51.9	120.3	128.6
9	54.5	52.4	74.4	74.4	75.0	75.4	58.0	58.2	57.9	x	47.5	50.8	49.7	51.7	51.6	51.5	53.0	52.4	54.4	55.1	128.8	128.6
10	35.4	35.5	45.8	49.2	47.6	47.3	50.0	48.4	48.7	47.6	x	4.5	34.2	34.5	35.8	35.9	36.3	36.2	36.6	36.7	130.9	129.0
11	35.5	35.5	49.2	47.3	49.3	47.8	50.3	49.1	50.2	50.8	4.5	x	34.2	34.3	35.7	35.7	36.3	36.4	36.7	36.8	140.0	132.6
12	35.7	35.7	50.1	49.5	50.4	49.8	50.2	50.3	45.3	49.7	34.2	34.2	x	4.8	36.1	36.2	36.8	36.8	37.2	37.1	138.4	132.7
13	35.8	35.7	49.2	51.2	50.7	51.2	50.8	50.9	51.2	51.7	34.5	34.3	4.8	x	36.0	36.2	36.9	36.9	37.0	37.1	136.6	137.5
14	37.2	37.4	52.4	46.8	45.4	52.9	50.4	51.1	53.0	51.6	35.8	35.7	36.1	36.0	x	4.8	38.1	38.2	38.4	38.0	122.9	135.5
15	37.3	37.4	50.7	45.7	50.4	50.0	51.6	49.0	51.7	51.5	35.8	35.7	36.2	36.2	4.8	x	38.2	38.2	38.9	38.6	139.1	145.2
16	37.8	37.8	52.0	52.8	50.9	52.6	50.5	52.6	52.8	53.0	36.3	36.3	36.8	36.8	38.1	38.2	x	4.8	39.8	39.9	120.5	134.8
17	38.1	37.9	53.2	52.4	54.5	52.8	52.2	51.5	53.8	52.4	36.2	36.4	36.8	36.8	38.2	38.2	4.8	x	39.9	39.9	142.4	152.0
18	38.1	38.0	53.8	53.2	52.1	52.5	51.7	53.3	54.8	54.4	36.6	36.7	37.2	37.0	38.4	38.9	39.8	39.9	x	4.8	128.7	133.8
19	38.0	38.0	50.4	52.8	53.1	53.0	47.0	50.7	51.9	55.1	36.7	36.8	37.1	37.1	39.0	38.6	39.9	39.9	4.8	x	125.0	135.1
20	130.6	128.8	132.1	115.4	119.0	136.7	119.4	122.9	130.3	129.8	130.8	140.0	138.4	136.6	122.8	139.1	120.6	142.4	129.7	125.0	x	116.8
21	128.4	135.9	129.3	135.7	132.3	123.3	125.9	136.6	128.6	128.5	128.0	132.6	132.7	137.5	135.5	145.2	134.8	152.0	133.8	135.1	116.8	x

Figure 3: Non-linear core-to-core latencies on an Intel Core Ultra 7 155H

When designing algorithms, we must now consider the distance between cores if they need to communicate. Processor designers have organized cores into clusters, called "islands," typically consisting of four cores each. This architecture introduces non-linearity within the processor itself. Figure 2 shows the concept of "core islands" on a 144 core processor. Latencies between cores on the same island are much lower than between cores on separate islands.

Modern processors also have complex configurations which can affect core-to-core latencies. Figure 3 illustrates this on an Intel Core Ultra 7 155H which is a "6+8+2/22" processor. This means that it is a 16-core processor, but the specifications of the cores are as follows:

- 6 performance cores
- 8 efficiency cores
- 2 low-power cores
- Maximum of 22 threads - Hyperthreading only available on performance cores (2 threads per core)

This configuration has interesting implications for the non-linearity of core-to-core latencies. Threads 0-1 and 10-19 display better core-to-core latencies as those are the performance cores. With hyperthreading, inter-core latencies are as low as around 4.5 ns.

2.3 Socket Latencies

In systems with multiple processors on a single motherboard, there is additional non-linearity in socket-to-socket communication. While communication within a socket is relatively fast, communication between "far away" sockets can lead to higher latencies. This is due to needing to deal with multiple cache lines between sockets to communicate with a "far away" socket. Figure 4 illustrates this by showing the core-to-core latencies between different cores across multiple processors.

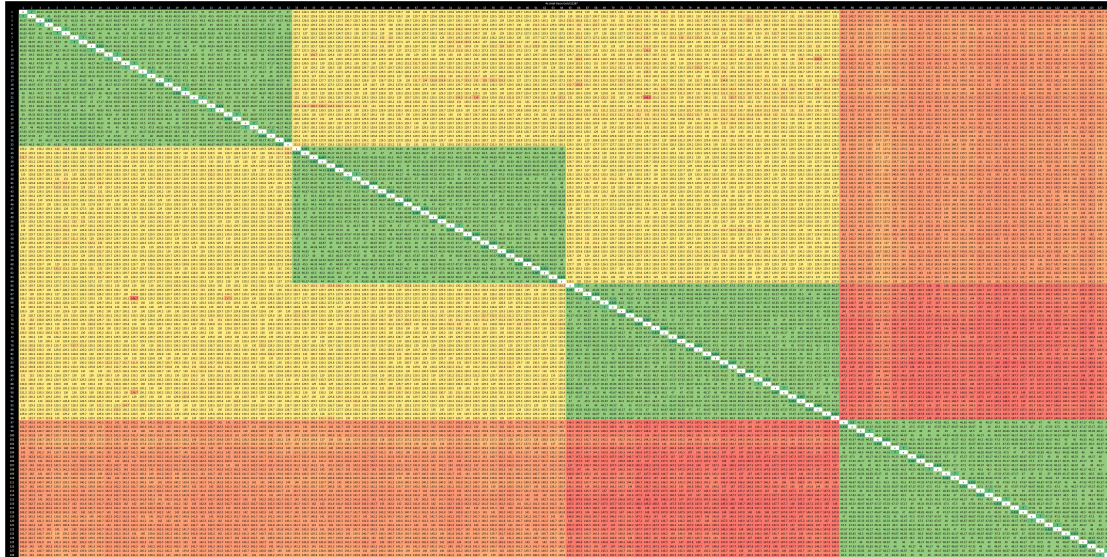


Figure 4: Non-linear core-to-core latencies on a multisocket system (Intel Skylake, 4 Sockets)

With 4 sockets, we can see four green zones which represent the core-to-core latency of cores within the same socket. We can also see that communication between sockets can vary as some sockets are "far away" in comparison to others.

Latency in core-to-core communication can also be implicit. For instance, if one thread releases a latch that is cached locally, and another thread from a different socket attempts to acquire that latch, there can be high latency as the system tries to retrieve the latch from the cache in the other socket.

3 Partitioned Hash Join Algorithms

3.1 Background

In order to understand more complex variations of the Partitioned Hash Join, we should first begin with an overview of the Partitioned Hash Join Algorithm, which consists of two phases, the partition phase and the probe phase.

3.1.1 Partioned Hash Join: Partition Phase

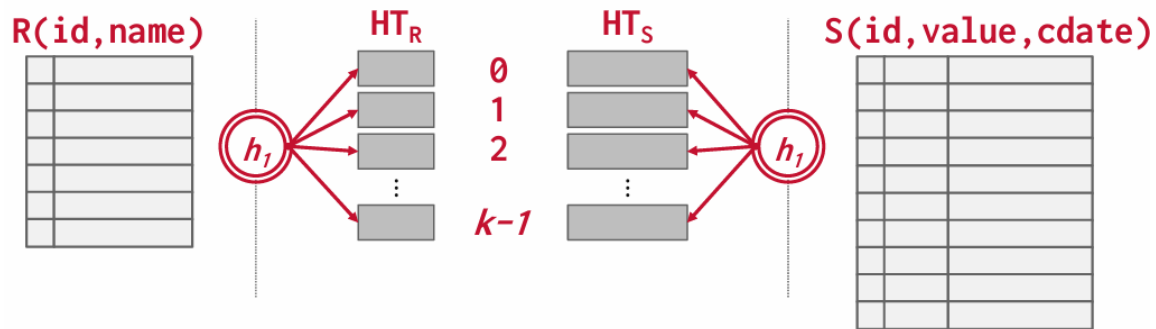


Figure 5: End of the partition phase

- Hash R into k buckets
- Hash S into k buckets using the same hash function
- Write buckets to disk when they get full

Above is a short description of the steps to perform the partition phase of the Partioned Hash Join algorithm. Figure 5 shows the outcome after performing the partition phase, where we have constructed two hash tables on R and S respectively. The next step of the partioned hash join is the probe phase.

3.1.2 Partioned Hash Join: Probe Phase

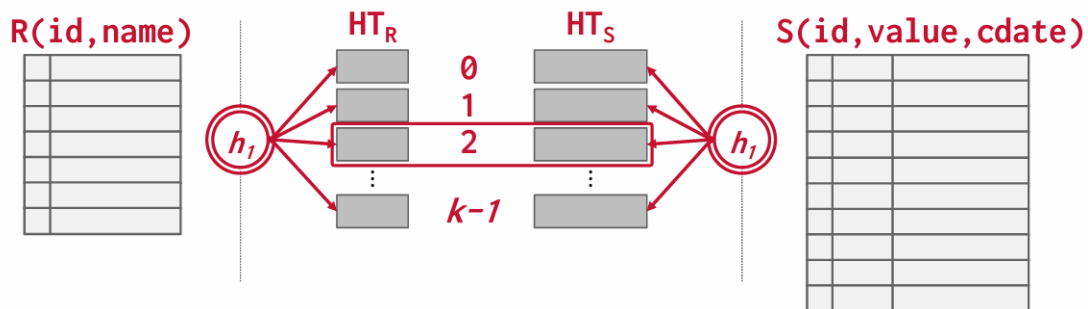


Figure 6: Probing a single pair of partitions

In the probe phase of the join, we read corresponding partitions into memory one pair at a time, and we hash join their contents. This algorithm does not take into account keeping memory accesses in the CPU cache and the limited TLB capacity, so we can improve the partioned hash join by making it more hardware aware.

3.2 Multi-Core CPU Hash Join

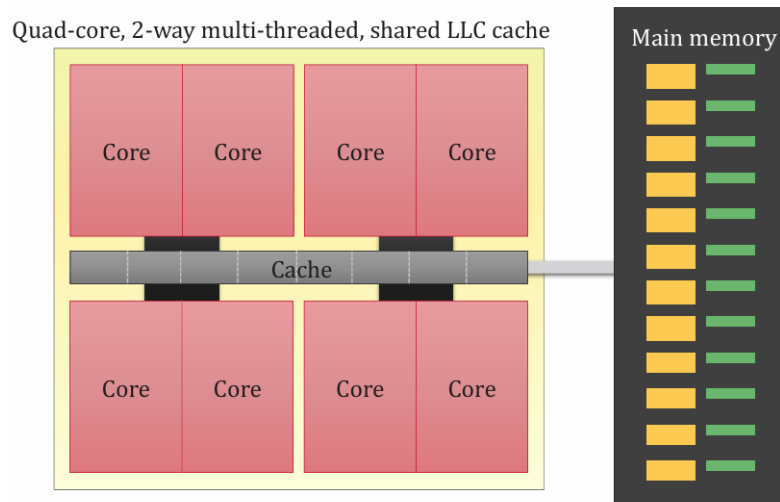


Figure 7: Creating hash table partitions which fit into processor cache

To adapt the partitioned hash join to take advantage of a multi-core CPU's cache, we can create hash table partitions such that the smallest partition fits into cache. We also need to ensure that there is enough space to stream in partitions of the other table while we are probing. Figure 7 shows how we can partition two tables for the join. The green partitions will fit in the cache while we stream the yellow partitions during the probe phase.

A limitation of this variation is that the partitioning phase can be very expensive. We want to optimize the partitioning phase such that we:

- Reduce synchronization
- Improve cache locality
- Reduce load on the TLB

3.3 No Synchronization Partitioning

Since we have multiple ways of building the hash table, we want to choose one that maximizes performance. If we try to build the hash table by using latches this can lead to higher latencies due to the latch bouncing around processor caches when different cores request a latch.

An intuitive algorithm for building a hash table without latches involves using an extra pass on the table during the partition phase to build a histogram. Each CPU scans a different region of the table and keeps a count of which rows hash to which partition.

Using the histogram, each CPU can compute the output offsets of where they should write each item in a partition. Figure 8 shows one step of having multiple CPUs writing to the partitions without synchronization under this algorithm.

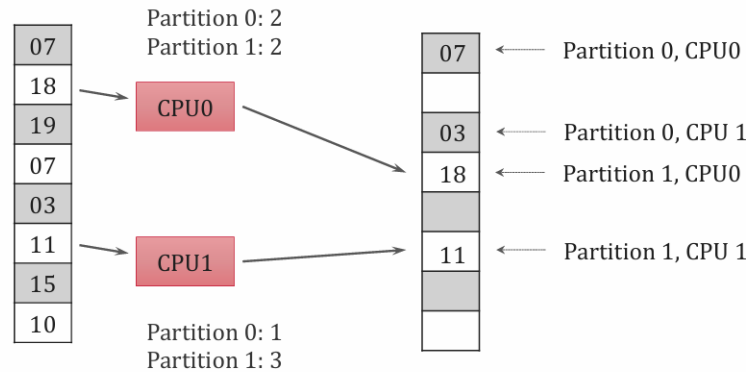


Figure 8: Partition step after creating a histogram and assigning output ranges per CPU

We can recursively repeat this partitioning algorithm until we have the target number of partitions. Doing this can be much more expensive when we have a much larger number of partitions, as we will need to write to more pages in the output. If the number of pages we hash to is greater than the number of TLB entries, then we can have very expensive TLB misses where each write to a hash bucket could require a page table access.

3.4 Radix Partitioning and Radix Join

We can solve the issue of having TLB misses when doing a no synchronization partition phase by using **Radix Partitioning**.

With radix partitioning as shown in Figure 9, we can see that we do multiple passes on the dataset and recursively partition by looking at subsequent bits. In the example below, with only 3-bit keys we perform a 2-bit pass and a 1-bit pass. In the first pass, we look at the two most significant bits and partition them based on the four possible 2-bit values. This means we are only referencing four pages at a time. In the second pass, we are only looking at the last bit, so each 2-bit partition, gets further subdivided into two smaller partitions.

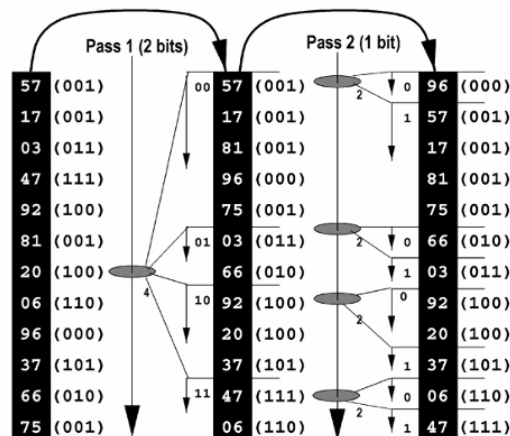


Figure 9: Two passes of Radix Partitioning on 3-bit keys

When we are doing a pass of radix partitioning, we are doing a sequential scan of the entire dataset. For each pass, we are only writing to 2^k pages where k is the # of bits used in the pass. Once we have partitioned the dataset with radix partitioning, we can implement a Radix Hash Join by using the radix partitions for a hash join shown in Figure 10

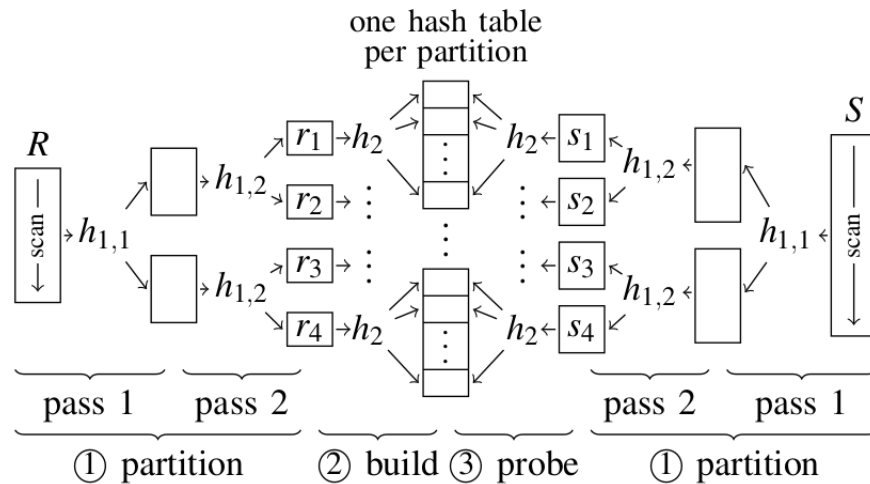


Figure 10: Three phases of Radix Join

3.5 Software Write-Combine Buffers

One other optimization that we can implement to reduce stress on the TLB when building the Hash Table is using **Software Write-Combine Buffers**.

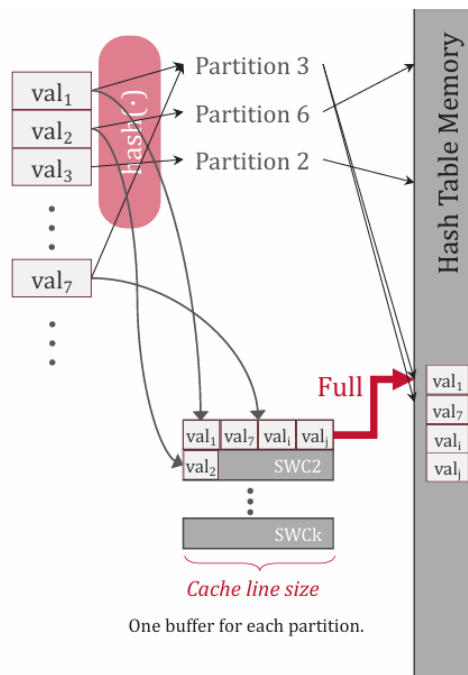


Figure 11: Using Software Write-Combine Buffers to reduce TLB misses

With a hash table that spans multiple pages in memory, we will be trying to write to different partitions which can touch many cache lines, stressing the TLB. In order to reduce the pages we are writing to, we can first write entries for a partition to a small buffer for that partition. Each buffer will be the size of one cache line, so the buffers are all in one page of memory resulting in only one TLB entry. When a buffer is

full, we can flush the cache-lined partitions to the actual hash table.

4 Conclusion

The performance of join algorithms is intricately tied to the underlying hardware architecture, and what constitutes the best join algorithm is an ever evolving question. Below are a few of the major considerations we need to make when implementing join algorithms.

In Non-Uniform Memory Access (NUMA) systems, careful attention to memory access patterns is crucial, especially for write operations. The non-linear nature of memory access in NUMA architectures can significantly impact join performance. To optimize for NUMA systems:

- Be mindful of data placement and access patterns across NUMA nodes.
- When the number of partitions significantly exceeds the number of available threads, consider implementing a NUMA-aware scheduler for joining co-partitions. This can help minimize cross-node memory accesses and improve overall performance.

Translation Lookaside Buffer (TLB) pressure also can be a significant bottleneck in join operations. To mitigate this:

- Consider using "huge" pages to reduce TLB pressure. This involves changing the default operating system page size.
- While huge pages result in fewer TLB misses, be aware that they may lead to increased internal fragmentation.

Ultimately, we need to always pay attention to hardware details and especially the memory hierarchy when considering query execution algorithms. Writing hardware-conscious algorithms can result in huge factors in terms of performance improvements.

Since hardware is always changing rapidly, we will need to periodically revisit these issues and reevaluate what we can do to improve the performance of our join algorithms.

Lecture #5: Robust Query Processing

15-721 Advanced Database Systems (Fall 2025)

<https://www.cs.cmu.edu/~15721-f25/>

Carnegie Mellon University

Notes by: Mihir Khare

1 Equi-Join Query Optimization

The high-level procedure for executing a query on a database has remained largely unchanged for decades: the raw SQL query is taken in, it gets optimized to form a physical plan, and the physical plan gets executed. Within each of these phases, there have been massive improvements to the overall structure of the system as well as its aggregate performance. Specifically within the query optimizer, key modules like join order determination, subquery unnesting, query costing, and cardinality estimation have seen a consistent research focus for the entire history of databases.

However, query optimizers are far from perfect. It has been known for decades that errors in cardinality estimation grow exponentially in the number of joins [1], and for the past decade that optimizers systematically underestimate the cardinality of tables as the number of joins grows [2]. External data like statistics and histograms can help, but even these have limited efficacy for tables that are deep within the query plan.

In recent years, there has been a big shift in databases with the idea of separating storage from compute. Many products now ask their data platforms to operate on data that has never been seen before (perhaps in Parquet files), and therefore has very little information in terms of statistics, correlations, histograms, or anything else traditionally used to assist with optimization. There is no indication that these trends are reversing, so the question at hand is how database systems can regain performance and consistency even when the optimizer is unable to create a good query plan.

2 Star Schema

As with many problems, a first step can be to add extra constraints. With query optimization, certain query shapes have a known “best query plan”, which means the space of “bad query plans” the optimizer can create is also limited. One common query shape occurs with **star schema** databases, which consist of one large fact table and n small dimension tables, where the fact table has foreign keys on the dimension tables. An example is a large fact table containing historical sales records, and small dimension tables detailing products, warehouse locations, and customers.

For equi-join queries on a star schema database where the fact table is joined on one or more dimension tables, the best query plan will almost always be a **left-deep** query plan, where hash tables built on the dimension tables are probed in a single pipeline by the fact table. In these cases, the optimizer only has to determine best join order within this single pipeline, as the query plan shape is fixed. Though this is a much simpler problem, the question of how to select the optimal join order still remains.

3 Semi-Join Optimization

An important idea is that if the optimizer cannot determine the optimal query plan in general, then the only other place to try to regain performance and consistency will be at execution time.

In general, the most expensive part of any analytical equi-join query will be building and probing the hash tables within joins. However, if we can only probe the build table with tuples we know are going to be

present in the output of the join, then the join itself will be significantly cheaper. In essence, we want to pre-filter the **dangling tuples** from the probe table.

This is what is known as the **semi-join optimization**: pre-filtering the probe side of a join based on the join predicate and data from the build side [3]. On its own, this is not very interesting — the only reasonable way to perform a semi-join would be to use a hash table in the filter, which leaves our position basically unchanged. However, if we loosen our specification and just aim to filter out “most” of the dangling tuples, we can potentially do much better. A depiction of this can be seen in Figure 1.

Semi-join Optimization

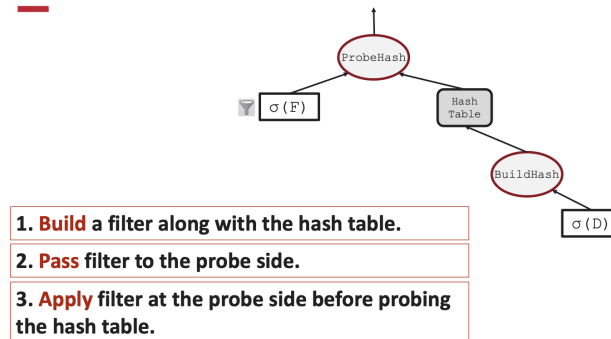


Figure 1: Semi-Join Optimization

A standard data structure for this is a **bloom filter**, which guarantees no false negatives (we don’t incorrectly filter out needed data), but can have a small proportion of false positives (which will anyways not pass the join). The false positive rate is a function of the size of the bloom filter, the number of unique values that will be added, and the number of hash functions used (which is often kept at 1). Although they are generally smaller and cheaper to build and probe than hash tables, the associated costs are not insignificant. For certain data distributions bloom filter semi-joins can greatly speed up execution, but it is important to be aware of the tradeoffs.

In certain cases, where the range of data added to the bloom filter is small, an exact bitvector can be used to improve performance and remove false positives.

4 Lookahead Information Passing

With star schema equi-joins, we have a single probe table that gets joined with many build tables — with the semi-join optimization, this means all of the bloom filters created will only reference columns present in the fact table.

Since every filter will operate on the probe table, this means that we have no need to wait for the result of a previous join before probing any bloom filters. This insight constitutes the core functionality of the **Lookahead Information Passing (LIP)** technique [4].

Lookahead Information Passing (LIP)

1. Build a filter along with hash table at each dimension table.
2. Pass all filters to fact table.
3. Apply all filters at fact table before probing any hash table.

In what order should the filters be probed?
Choose using an adaptive run-time algorithm.

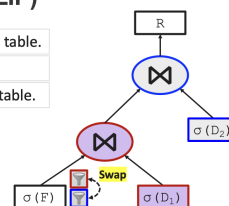


Figure 2: Lookahead Information Passing

The biggest difference between LIP and just applying the semi-join optimization to each join is adaptiveness. Specifically, if the fact table is joined with multiple dimension tables, LIP can change at runtime the order in which their filters are probed. LIP will probe the bloom filters in batches that increase in size over time, and after each batch has finished being filtered, the bloom filters are reordered by decreasing selectivity for the next batch. This means that we probe filters that get rid of the most tuples first, saving on probing costs for the later bloom filters.

In Figure 3, we can see that for some queries (e.g. 4.2), the performance with LIP is actually better than without! This is precisely due to its adaptivity in bloom filter probing, which allows it to adapt to shifts in the data distributions over the course of the query. This means that in some cases, LIP will do better than even the perfect statically-optimized query plan.

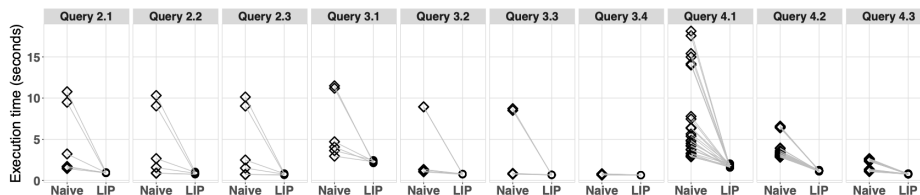


Figure 3: LIP performance comparison on SSB [4]

4.1 Robustness

Another interesting observation from Figure 3 is that the range of execution times with LIP is significantly narrower than without. This is known as **robustness**, intuitively defined as a tight upper bound on the performance difference between the best and worst execution plans for some query, and formally defined in Figure 4. On a left-deep reductive join pipeline (with independent join predicates), LIP is Θ -robust and non-LIP is θ -fragile, for a Θ typically less than θ . If the entire query does not meet the requirements for LIP, it can still be applied to any subqueries that do.

DEFINITION 1. An evaluation strategy $\mathcal{E}$ is said to be θ -fragile and Θ -robust and with respect to a plan space $\mathcal{P}$ if the maximum deviation in performance of any plan in $\mathcal{P}$ (including the worst plan $\mathcal{E}_w$) from the best one $\mathcal{E}_b$, normalized by the fact table cardinality and spread of selectivities in a query, is bounded between θ and Θ .

$$\theta \leq \frac{T(\mathcal{E}_w) - T(\mathcal{E}_b)}{(\sigma_{\max} - \sigma_{\min})|F|} \leq \Theta, \quad \sigma_{\max} \neq \sigma_{\min} \quad (9)$$

Figure 4: Formal definition of robustness [4]

4.2 LIP in SQLite3

LIP was also recently implemented in SQLite3, achieving significant performance gains on most Star Schema Benchmark queries, and roughly unchanged performance on some. The full results can be found in Figure 5. Even in cases where the theoretical requirements for robustness do not all hold, LIP has still demonstrated real world improvements on many workloads.

5 Yannakakis and Predicate Transfer

5.1 The Yannakakis Algorithm

A different approach to using the semi-join optimization to an entire query was proposed in 1981, known as the **Yannakakis Algorithm** [6], which does not require a specific query shape as LIP does. This technique guarantees the full removal of all dangling tuples from all tables, creating an equivalent query plan on what are called **Yannakakis tables**.

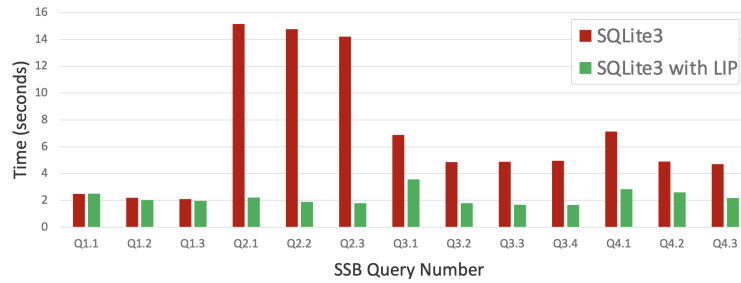


Figure 5: SQLite3 SSB performance comparison with LIP [5]

The algorithm consists of making two passes over the tables (rather than the single pass needed in LIP) — an “upwards” pass from the leaves to the arbitrarily chosen root of the join graph (reducing tables along the way), followed by a “downwards” pass from the reduced root to the leaves (further reducing tables along the way). Notably, this technique guarantees that any acyclic and conjunctive query (a select-project-join query without not-equals join predicates) can be evaluated in polynomial time with respect to the size of the database. A visualization of this algorithm is shown in Figure 6.

The Yannakakis Algorithm [VLDB'81]

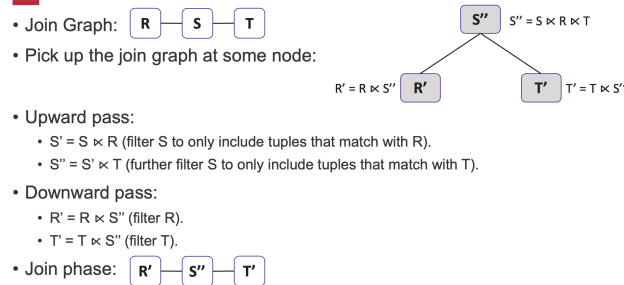


Figure 6: Yannakakis Algorithm

However, this approach didn’t see much adoption in the coming decades, as the cost of performing a second pass on the table data (often reading from disk) was impractical in many settings. It was only quite recently that it was revived, with some major improvements.

5.2 Predicate Transfer

Predicate Transfer is a variant of the semi-join optimization that is based on the Yannakakis Algorithm, with two key differences: the transfer schedule is not based on a join tree, and bloom filters are used for the semi-joins [7]. This results in a much more practical execution strategy, especially for queries where all tables can reside in memory.

The transfer schedule is created with the understanding that smaller tables generally create more selective filters than larger tables. Therefore, it is best to first create bloom filters on smaller tables and pass them to larger tables in the “forwards” pass, to reduce filter creation and probing costs on the “backwards” pass. Specifically, the transfer schedule starts at the smallest table, and then goes outwards. This algorithm, called **Small2Large**, is depicted in Figure 7.

Once the transfer schedule is created, the semi-join phase begins, in which the filters are constructed and tables are reduced as specified in the schedule. Following this is the join phase, which as in the Yannakakis Algorithm will be efficient regardless of the join order (as long as any joins follow a single edge in the join graph) [6].

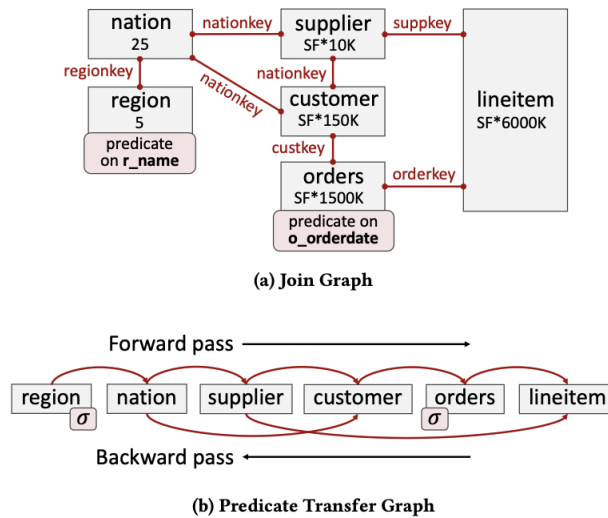


Figure 7: Predicate Transfer schedule on TPC-H Query 5 [7]

6 Robust Predicate Transfer

A drawback of Predicate Transfer is that its construction of the forwards and backwards passes does not fully reduce the tables in all acyclic queries. As shown in Figure 8, because Predicate Transfer’s transfer graph is not a tree, the transfer schedule can miss the flow of information between two tables that could reduce each other. Without a full reduction, we do not get the polynomial runtime guaranteed by the pure Yannakakis algorithm. To improve upon this, **Robust Predicate Transfer** (RPT) extends Predicate Transfer in 2 key ways [8].

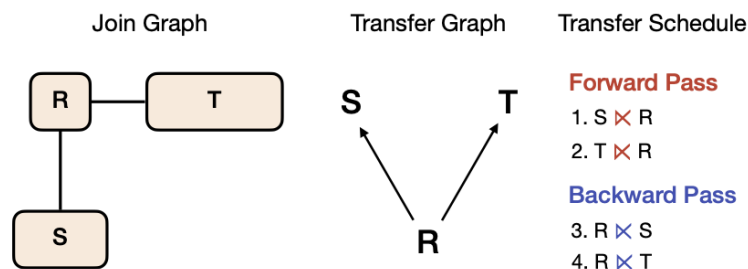


Figure 8: Partial reduction case of Predicate Transfer [8]

The first main contribution of RPT is a new method for creating the transfer schedule, called **LargestRoot**. Rather than the forward pass transferring from the smallest table outwards, RPT first transforms the join graph into a join tree rooted at the largest node, and then proceeds with a forwards leaf-to-root pass and backwards root-to-leaf pass as in the Yannakakis Algorithm. As with Predicate Transfer, this schedule means that smaller tables (which are more likely to have more selective predicates) reduce the larger tables first, reducing overall bloom filter construction costs. This not only guarantees a full reduction of the tables, but also improves upon the Yannakakis Algorithm with its support for cyclic queries.

The Yannakakis Algorithm says that in order to guarantee polynomial complexity for evaluating the query, the joins should be executed bottom-up based on the edges present in the join tree [6]. However, this prevents the optimizer from finding join orders that may have a better constant factor complexity than the Yannakakis order. Specifically, the optimizer could choose to join two tables that *indirectly* share a join column, which would not be present as an edge in the join tree.

In order to guarantee that the optimizer’s chosen join order meets the robustness requirements, RPT also introduces the **SafeSubjoin** algorithm to determine if the size of each intermediate result is bounded by the size of the final output. The algorithm is simple, and says that a subjoin is safe if its join tree is part of some maximum spanning tree of the entire join. This matches the understanding from the Yannakakis Algorithm, where as long as all the joins are possible to occur in some join tree, it is safe for the execution to proceed.

One example of an acyclic join where a candidate subjoin is unsafe is the natural join $R(a, b, c) \bowtie S(a, b) \bowtie T(b, c)$, where R , S , and T are as shown in Figure 9. Here, the final output of the joins will have size n , but if join $S(a, b) \bowtie T(b, c)$ is done first, this intermediate product will have size n^2 ! Fortunately, SafeSubjoin catches this: no join tree can have $S - T$ as an edge, and so this query plan would be marked as unsafe.

a	b	c	a	b	b	c
1	1	1	1	1	1	1
2	1	2	2	1	1	2
...	...	...	...	...	...	...
n	1	n	n	1	1	n

Figure 9: Example of data causing an unsafe subjoin

If the join graph is cyclic, then RPT provides no guarantees in any case.

6.1 RPT in DuckDB

RPT was also implemented in DuckDB v0.9.2 [8], and more recently in v1.3.0. The transfer schedule is found using LargestRoot, which informs the placement of the **CreateBF** and **ProbeBF** operators (with their logical counterparts **LogicalCreateBF** and **LogicalProbeBF**). These operators are always linked, such that each ProbeBF has access to the filter made in its corresponding CreateBF. CreateBF acts as the end of one pipeline and the start of another pipeline, while ProbeBF is an intermediate operation. An example query plan is shown in Figure 11.

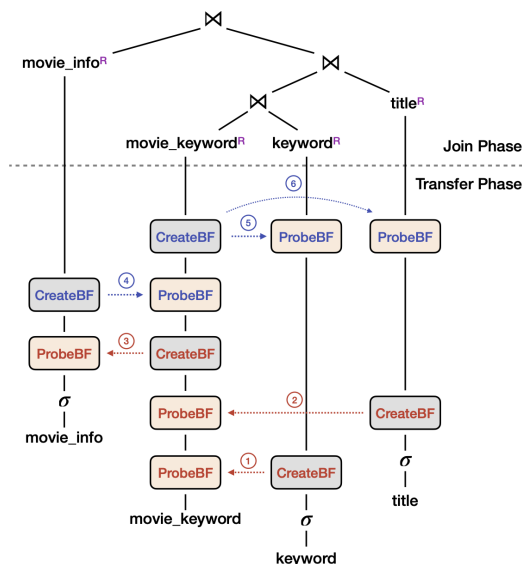


Figure 10: Query plan for JOB 3a with RPT [8]

RPT shows significant performance improvements compared with standard DuckDB, as seen in Figure 11. Something to note is that on TPC-H Q5, the performance and robustness is not nearly as good as with the other queries. This is because the query is cyclic (denoted by the red number), and therefore we don’t expect RPT to help on every input query plan.

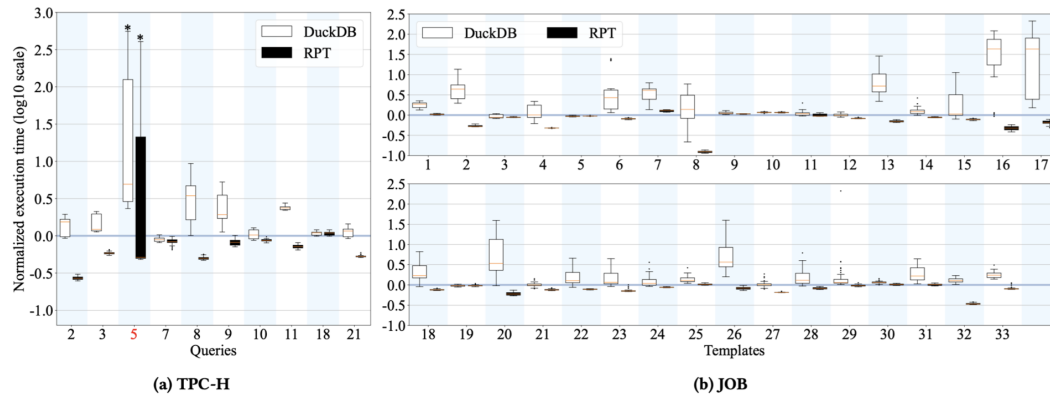


Figure 11: Performance of RPT with random bushy plans on TPC-H and JOB [8]

The RPT implementation also restricts the optimizer to exploring only left-deep plans, as it is believed that there isn't a significant difference between left-deep and bushy plan performance when using RPT.

7 References

- [1] Yannis E. Ioannidis and Stavros Christodoulakis. On the propagation of errors in the size of join results. *SIGMOD Rec.*, 20(2):268–277, April 1991. ISSN 0163-5808. doi: 10.1145/119995.115835. URL <https://doi.org/10.1145/119995.115835>.
- [2] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. How good are query optimizers, really? *Proc. VLDB Endow.*, 9(3):204–215, November 2015. ISSN 2150-8097. doi: 10.14778/2850583.2850594. URL <https://doi.org/10.14778/2850583.2850594>.
- [3] Philip A. Bernstein and Dah-Ming W. Chiu. Using semi-joins to solve relational queries. *J. ACM*, 28(1): 25–40, January 1981. ISSN 0004-5411. doi: 10.1145/322234.322238. URL <https://doi.org/10.1145/322234.322238>.
- [4] Jianqiao Zhu, Navneet Potti, Saket Saurabh, and Jignesh M. Patel. Looking ahead makes query plans robust: making the initial case with in-memory star schema data warehouse workloads. *Proc. VLDB Endow.*, 10(8):889–900, April 2017. ISSN 2150-8097. doi: 10.14778/3090163.3090167. URL <https://doi.org/10.14778/3090163.3090167>.
- [5] Kevin P. Gaffney, Martin Prammer, Larry Brasfield, D. Richard Hipp, Dan Kennedy, and Jignesh M. Patel. Sqlite: past, present, and future. *Proc. VLDB Endow.*, 15(12):3535–3547, August 2022. ISSN 2150-8097. doi: 10.14778/3554821.3554842. URL <https://doi.org/10.14778/3554821.3554842>.
- [6] Mihalis Yannakakis. Algorithms for acyclic database schemes. In *Proceedings of the Seventh International Conference on Very Large Data Bases - Volume 7*, VLDB '81, page 82–94. VLDB Endowment, 1981.
- [7] Yifei Yang, Hangdong Zhao, Xiangyao Yu, and Paraschos Koutris. Predicate transfer: Efficient pre-filtering on multi-join queries, 2023. URL <https://arxiv.org/abs/2307.15255>.
- [8] Junyi Zhao, Kai Su, Yifei Yang, Xiangyao Yu, Paraschos Koutris, and Huanchen Zhang. Debunking the myth of join ordering: Toward robust sql analytics, 2025. URL <https://arxiv.org/abs/2502.15181>.

Lecture #7: Parallel DBMS

15-721 Advanced Database Systems (Fall 2025)
<https://www.cs.cmu.edu/~15721-f25/>
Carnegie Mellon University
Notes by: Kartica Modi

1 Architecture-Conscious Query Processing - Continued

1.1 Partitioned Hash Join

In partitioned hash join, we hash the two tables into K buckets, writing the buckets to disk when they become full. Then read the corresponding partitions into the memory one pair at a time and hash join their contents. K is chosen such that each partition of the smaller table fits in memory.

In a multi-core CPU, we can partition the smaller table into partitions such that each partition fits in the cache and load all partitions in the cache. We can then bring in partitions from the larger table one by one and process them in parallel.

While partitioning, if K is large, and the hash function distributes the records uniformly over the entire address space, there could be a lot of TLB misses due to the big memory space being randomly accessed. Therefore, we need a smoother pattern to access memory.

Partition phase with no synchronization: This is a clever two-pass approach to prevent TLB thrashing in the partitioning phase.

- Scan the input once and count how many tuples will go into each partition for each CPU. This is TLB-friendly since it's sequential.
- For each CPU, using the counted tuples for each partition, compute offsets/pointers to allocate contiguous regions for each partition in the output array.
- Scan the input again, this time placing tuples into their designated partition using the precomputed pointers. Since each pointer will move sequentially, this leads to fewer TLB misses.

The tradeoff is two passes instead of one, but the benefit is a lot fewer TLB misses due to sequential memory access.

Partitioning in phases with radix partitioning: If a hash function scatters records across a large number of partitions, we again run into the issue of repeated TLB misses. To address this, radix partitioning borrows from radix sort. Instead of partitioning all at once, it uses multiple passes: in pass one, tuples are grouped by the first few bits of the hash value (e.g., 3 bits $\rightarrow$ 8 partitions). Each of these groups is then refined in a second pass using additional hash bits (e.g., another 3 bits $\rightarrow$ 8 sub-partitions each). This staged approach creates a radix-ordered layout, which ensures that tuples destined for nearby partitions are grouped together. The result is a significantly more linear memory access pattern, which reduces cache and TLB pressure at the cost of requiring extra passes.

Software Write-Combine Buffers: Another technique to reduce random memory writes is the use of software-managed write-combine buffers. Rather than writing each tuple directly to its global partition buffer, each thread writes into a small, cache-line-aligned buffer (the size of the buffer for one partition could be equal to the L2 cache line. When this buffer fills, it is flushed in one contiguous burst to the global partition.

This technique exploits the fact that the set of buffers for the K partitions is small enough to fit on one page, so even randomly accessing them is TLB efficient. Flushing the buffer to the global partition also writes to memory in the same partition, again making it TLB efficient.

NUMA Awareness and other observations: NUMA refers to non-uniform memory access. A server with multiple processors has multiple DRAMs, each attached to one processor. A processor's access to its

local DRAM is fast, but to another processor's DRAM is significantly slower. This architecture implies that software now needs to be aware of this topology and should avoid remote memory traffic whenever possible. For partitioned hash join, this means keeping partitions and their hash tables local to the cores that will process them. Another approach to reduce TLB pressure is to use huge pages. This leads to fewer TLB entries, but more internal fragmentation since pages may be underutilized.

2 Parallel Database Systems

2.1 Backdrop

When databases began to exceed the capacity of single machines, two approaches were pursued: (1) specialized hardware ("database machines"), or (2) assembly of cluster of "commodity" servers and write software to execute on this parallel hardware. Specialized hardware quickly died because it couldn't keep up with Moore's Law. Thus, shared-nothing clusters of commodity servers became more popular in the industry.

2.2 Parallel Architectures

Three main architectures were explored:

- **Shared Memory:** All processors share the same memory. This makes it easy to port a single-node DBMS, since memory/buffers are easily available from one node to another. Buffers and catalogs remain accessible to all. However, the network is expensive and doesn't scale as new nodes are added.
- **Shared Disk:** Each processor has private memory, but all share the same disk. Any node can access any data on the disk. Cheaper network than shared memory, but still hard to scale it. All memory-related data exchange components in the single-node DBMS software must be changed.
- **Shared Nothing:** Each node owns its CPU, memory, and disk. This has maximum scalability since it rides commodity trends, but now everything needs to be re-implemented - data placement, transaction coordination, recovery, etc. However, this is most adopted because of its scalability. Some key technical challenges are:
 - How to measure the effectiveness of parallel DBMS?
 - How to distribute data across nodes?
 - Coordinating transactions across nodes.
 - Fault tolerance in case of node failures.

Modern cloud platforms resemble shared disk (e.g., S3) with technologies like CXL adding elements of shared memory.

2.3 Scaleup and Speedup

- **Speedup:** Run the same job on a small vs. a large system. Speedup is the ratio of runtimes. The ideal speed-up is linear (doubling nodes halves runtime).
- **Scaleup:** Increase both data size and system size proportionally. The ideal scale-up time means that the runtime stays constant. This has two flavours - transactional (spread transactions across nodes) and batch (large analytical queries).

Startup costs (to start and coordinate parallel machinery), interference (resource contention), and skew (uneven job sizes) all prevent perfect curves.

2.4 Partitioned vs. Pipelined Parallelism

Two forms of parallelism exist:

- **Partitioned parallelism:** Divide data across nodes using a partitioning scheme and run the same operator independently. This is the backbone of scaling for scans and joins, and most of the scaling benefits come from this. Exchange operators are used to implement this.
- **Pipelined parallelism:** Operators stream tuples as soon as they are produced, without waiting for complete input. This allows for overlapping execution, lowering latency.

2.5 The Exchange Operator

To support parallelism cleanly, DBMSs introduce a special operator called **exchange**. It sits between operators in the query plan and redistributes tuples across nodes. It takes n input streams and produces m output streams. It splits the input by some specified hash function. Therefore, it can:

- Partition: Hash-partition tuples across nodes by join key.
- Merge: Gather multiple streams into one. ($m = 1$)
- Broadcast: Send each input tuple to all nodes, resulting in a broadcast join.

The exchange operator essentially abstracts away data movement so that other operators (e.g., join, aggregate) don't need to implement custom logic.

2.6 Broadcast Joins

In a broadcast join, instead of repartitioning both relations, the smaller relation R is replicated to every node. Each node then joins R with its local partition of the larger relation S .

This is particularly effective when R is tiny and S is massive: broadcasting avoids repartitioning S , saving expensive network transfers.

Lecture #8: OLAP and Indexing

15-721 Advanced Database Systems (Fall 2025)

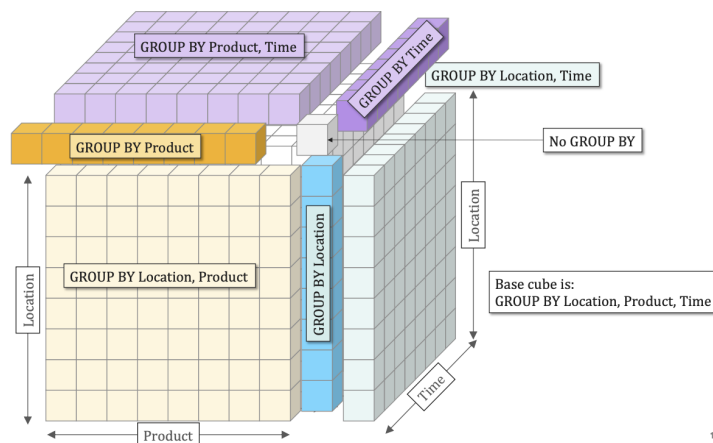
<https://www.cs.cmu.edu/~15721-f25/>

Carnegie Mellon University

Notes by: Ruiqi Wang

1 OLAP Databases

OLAP databases tend to organize data in a star or snowflake schema. In an OLAP data cube, each axis (dimension) of the cube corresponds to an attribute you can group or filter by (in this example, Product, Location, and Time). Each base cube represents an aggregate measure for a specific combination of GROUP BY Location, Product, Time.



The cube can also be sliced or diced, where slicing is fixing some dimensions to specific values, and dicing is to select a smaller portion of a cube. E.g.

```
SELECT SUM(sale_amount) AS total_sales
FROM Orders
WHERE time_of_sale BETWEEN CURRENT_DATE - INTERVAL '7 days' AND CURRENT_DATE
      AND product.category IN ('apparel', 'shoes')
      AND location IN ('PA', 'OH')
```

Output of these OLAP SQL constructs will abuse NULLs by semantically overloading it to indicate a lower dimension value.

2 Bit Indexing

One of the key operations in OLAP is to compute aggregates on these cubes, so it's important to have efficient aggregate evaluation. One way to improve the performance is to leverage the intra-cycle parallelism.

Bitmap Indices for B+ Trees

Sometimes, the search key in B+ Tree is not unique. In this case, we can store record id list corresponding to the keys in the leaf nodes. Although we can have compression scheme to reduce the size of the list, it may still be the case a list cannot fit in one page.

List of record ids usually cost 4-8 bytes per record id. An alternative representation could be a bitmap where we have 1 bit per record in the table, where it represents each distinct attribute value (and null value) as a bit vector (bitmap) over all records.

Example: For a column Gender with values M, F, two bitmaps are created:

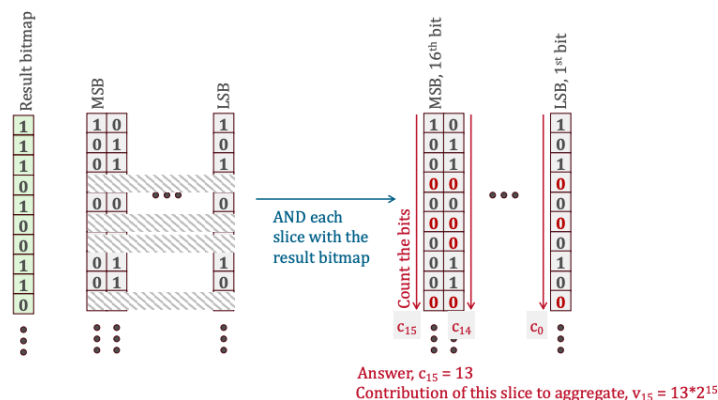
- BM_M : bit = 1 if record has M, else 0
- BM_F : bit = 1 if record has F, else 0
- BM_{null} : bit = 1 if record is null, else 0

Then we can perform AND, OR, NOT operations on the bitmaps to answer queries, which is very fast in hardware with SIMD instructions.

Note that this representation needs one bit for each distinct value for each record in the table. So if the number of bits needed to represent the record id is smaller than the number of distinct values, then bitmap is more efficient than record id list, which is usually the case for low-cardinality attributes.

Bitwise Aggregate Computation

Bit-sliced Index is constructed by simply vertically slice the native representation. As an example, if we have a 16-bit value column, we can create 16 bitmaps, one for each bit position. This enables parallel bitwise across all rows for each bit position.



As an example, in order to calculate the SUM of a column satisfying a predicate (e.g. `WHERE value < CONSTANT`)

1. Determine which rows satisfy the query condition. This can be done by performing a bitwise comparison of each record's bits with the constant's bits from MSB to LSB. After all bits are processed, the result bitmap has 1 for each row that meets the predicate, and 0 otherwise.
2. Perform AND for each slice with result bitmap and this filters the slice down to only qualifying rows.
3. Count the '1' bits in each bit slice to determine the contribution of this particular slice to aggregate value.

Fast bit-counting

1. Naively: Loop over each bit and count the number of 1s
2. Lookup array: Precompute the number of 1s for each byte (256 values) and store it in an array. Then, for each byte in the bitmap, lookup the number of 1s in the array and sum them up. Sacrificing cache space for CPU cycles.
3. POPCNT: Use the hardware instruction in modern CPUs to count the number of 1s in a word
4. SIMD: Use SIMD instructions to process multiple words in parallel.

Data Layout

- Row-store store all the attributes of a record together. Good for OLTP workloads
- Column-store organizes the data "vertically" by columns, significantly faster for analytics
- Bit store slices column further into bits, and stores each bit slice as a bitmap

Each memory word (e.g. 64 bits) is essentially an array of bits that the CPU can operate on simultaneously. In one clock cycle, a processor can add two 64-bit numbers, i.e. 64 parallel bit-level operations within that single cycle. So bit-based storage exploits this fine-grained, bit-level parallelism to achieve significant performance gains without needing extra cores or threads.

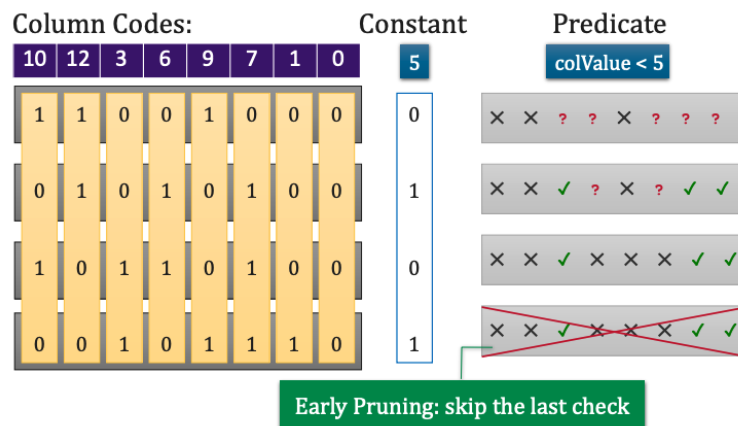
BitWeaving

- BitWeaving-H: Horizontal layout, where each row is stored in a word.
- BitWeaving-V: Vertical layout, where each column is stored in a word.

Word size is dependent on the underlying hardware architecture (e.g. 32-bit register, SIMD register, row in a DRAM subarray, etc.)

Early Pruning for BitWeaving-V

At each bit position, the comparison can determine which rows can be **definitively** accepted (✓) or rejected (×), and which ones still require checking at lower bits (?). If, at any point, the comparison outcome is already determined (in this case, when a higher bit of the column value exceeds the 5's bit), the system can prune those rows early and skip further checks. And we can stop the comparison as soon as all rows are either accepted or rejected.



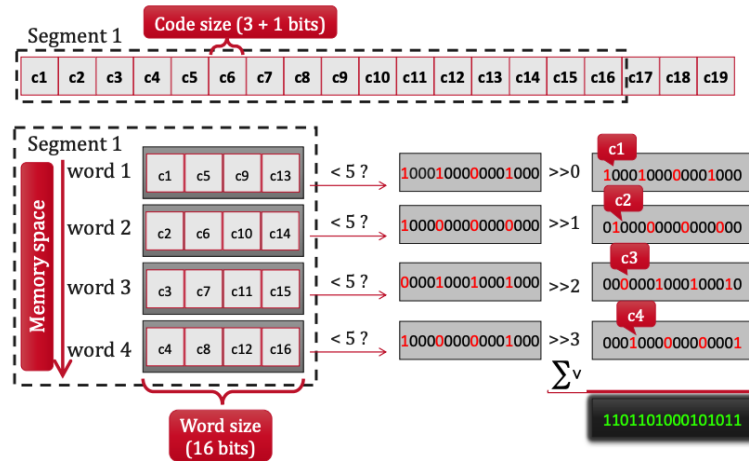
The pruning probability can be modeled as follows, where w is the word size, and f is the fill factor:

$$P(b) = (1 - (\frac{1}{2})^b)^{w \cdot f}$$

This implies that in a 32 bits system, 8 bits can prune $\approx 80\%$ of the tuples. The performance benefit is algorithmic and not hardware-dependent, it comes from the encoding's binary structure itself, which allows many comparisons to be resolved using only the higher-order bits, enabling high performance regardless of the underlying hardware.

BitWeaving-H

Each data segment contains a sequence of codes encoding data value, each represented with a fixed number of bits. The system can arrange these codes into machine words, and within each word, multiple codes can be packed together (four in this example). Instead of testing each value individually, the CPU evaluates several codes at once, taking advantage of its ability to process entire words per instruction.



The comparison < 5 is applied in parallel across all codes using bitwise shifts and masks. Each word produces a partial result vector indicating which codes satisfy the predicate. The system then combines (sums) these partial results into a final result bit vector that identifies all qualifying tuples.

Materialized Views for OLAP

Another approach to speed up aggregations is to precompute subsets of the cube that capture common groupings and aggregations. Fully materializing every possible grouping gives fast query performance but becomes costly in storage and maintenance. So the system can selectively materialize a subset of these views based on workload patterns and query frequency, allowing other aggregations to be computed on demand from those pre-computed/materialized ones.

Lecture #9: Granularities of Locks and Degrees of Consistency

15-721 Advanced Database Systems (Fall 2025)

<https://www.cs.cmu.edu/~15721-f25//>

Carnegie Mellon University

Notes by: Yueyan Zhang

1 Introduction

1.1 Background

Transactions in databases guarantee the ACID properties: Atomicity, Consistency, Isolation, and Durability (ACID). Concurrency control ensures multiple users can read and write simultaneously without violating correctness. Schedules describe the execution order of operations, and a schedule is *serializable* if it is equivalent to some serial execution.

Two central trade-offs motivate this paper:

- **Concurrency vs. Locking Overhead:** Finer-grained locks increase concurrency but require more locks.
- **Consistency vs. Performance:** Higher isolation levels reduce anomalies but limit throughput.

1.2 Key Challenges

The key challenge is to design a locking protocol that balances the number of locks acquired (overhead) and the concurrency allowed by the protocol (throughput). Naïve approaches (e.g., database-level or column-level locks only) either hurt concurrency or add too much overhead. The solution is to allow different granularities of locks to coexist.

2 Granularities of Locking

To address the trade-off between concurrency and overhead, the proposed solution introduces hierarchical locking. The insight here is that when a transaction locks a higher-level node (e.g., a table), all of its descendants (tuples and fields) are implicitly locked as well. It models resources as DAGs, acquires locks in a disciplined top-down manner, and employs **intention locks** to indicate finer-grained locking below. A lock-compatibility matrix then balances the number of locks acquired with the level of granularity, ensuring both correctness and high transaction throughput.

2.1 Intention Locks

Traditional locks only allow Shared (S) and Exclusive (X) modes. **Intention Locks** is introduced to make hierarchical locking practical:

- **Intention-Shared (IS):** Indicates explicit locking at a lower level with S locks. It shows the intent to get S lock(s) at finer granularity.
- **Intention-Exclusive (IX):** Indicates explicit locking at a lower level with X locks. It shows the intent to get X lock(s) at finer granularity.
- **Shared + Intention-Exclusive (SIX):** The subtree rooted at that node is locked explicitly in S mode, while explicit locking is being done at a lower level with X locks.

As shown in Figure ??, the compatibility matrix specifies which locks can coexist on the same resource. The rows correspond to the locks held by transaction T_1 , while the columns represent the locks requested



Figure 1: Compatibility Matrix



Figure 2: Partial Ordering of the Lock Modes

by transaction T_2 . A checkmark ($\checkmark$) means the two locks are compatible and can be granted concurrently, whereas a cross ($\times$) indicates a conflict. From the matrix, we observe that IS and IX are mutually compatible as they only declare an intent for finer-grained locks; S locks are compatible with other S or IS locks but not with IX, SIX, or X; SIX is more restrictive, allowing compatibility only with IS; and X is the most restrictive, conflicting with all other modes. This scheme ensures correctness while maximizing concurrency when possible.

2.2 Locking Protocol

The locking protocol is designed to be well-formed and ensures that locks are always acquired in a top-down manner through the resource hierarchy. As illustrated in Figure ??, this protocol provides a partial ordering of lock modes, where higher modes such as X are the most restrictive, and IS the least. Each transaction first obtains an appropriate lock at a higher-level node before proceeding to lower levels. This requirement guarantees that other transactions are aware of the intention to lock at finer granularity, thereby preventing conflicting operations.

Locks are always released in the opposite direction—either gradually from leaf to root, or all at once at the end of the transaction. Importantly, a transaction must acquire some non-intention lock (S, X, or SIX)



Figure 3: Hierarchical Locking Example

within the hierarchy; a transaction that only holds intention locks is not meaningful, since intention locks merely signal future locking at finer levels.

Example. Consider two transactions: T_1 reads the balance of Alice’s account, while T_2 increases Bob’s account by 1%. According to the locking protocol, T_1 must acquire a shared lock (S) on Tuple 1. Before doing so, it first obtains an intention-shared lock (IS) on Table R, indicating that a finer-grained shared lock will follow. Similarly, T_2 needs to update Tuple n , so it acquires an exclusive lock (X) on that tuple. To do this, it first places an intention-exclusive lock (IX) on Table R.

As shown in Figure ??, the IS and IX locks on Table R are compatible, so both transactions can proceed concurrently. Conflicts are then checked only at the tuple level: since T_1 and T_2 access different tuples (Alice and Bob), no blocking occurs. This prevents unnecessary conflicts at higher levels of the hierarchy, thereby enabling greater concurrency.

2.3 Dynamic Lock Graph

In practice, the resource hierarchy is not static but rather a dynamic graph, since data may move during a transaction. For example, an update in a B-tree index may relocate a record from one page to another, effectively changing its position in the resource graph. This creates a challenge: once the record moves, the transaction may no longer hold the appropriate locks on the new ancestors in the hierarchy.

The solution is that, before moving data, both the old and the new locations must be locked in X mode. At the same time, the well-formed protocol must still be preserved, meaning that intention locks are acquired top-down to signal the traversal path. In this way, when another transaction traverses the structure, it can correctly detect that the resource is locked and avoid conflicting operations.

3 Lock Schedules and Upgrades

A single resource may be subject to multiple locks at the same time. To ensure fairness, the system must schedule lock requests so that transactions do not wait indefinitely. The protocol also defines how current and new lock modes are reconciled, based on the partial ordering of lock modes, to maintain correctness and

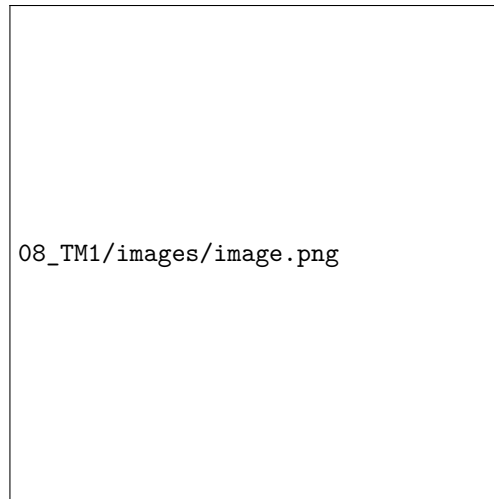


Figure 4: Deadlock Case

avoid starvation.

3.1 Deadlocks

With hierarchical locking in place, the system may encounter deadlocks. This occurs when two or more transactions wait on each other's locks in a cycle, preventing any of them from proceeding. As illustrated in Figure ??, T_1 holds a shared lock on Resource 1 and requests an exclusive lock on Resource 2, while T_2 holds a shared lock on Resource 2 and requests an exclusive lock on Resource 1. Neither transaction can make progress, and the wait-for graph shows a cycle between T_1 and T_2 , indicating a deadlock.

To resolve this, the system must either prevent or detect deadlocks:

- **Deadlock prevention:** Use strategies that avoid cycles in the wait-for graph, such as enforcing a global ordering on lock acquisition or aborting transactions when a conflict might cause a cycle.
- **Deadlock detection:** Periodically construct and examine the wait-for graph. If a cycle is found, select a victim transaction (e.g., the oldest or newest) and abort it to break the deadlock. The victim can then be restarted, possibly after a short delay.

3.2 Two Phase Locking (2PL)

Two-Phase Locking (2PL) is one of the most fundamental protocols to ensure conflict-serializable schedules in database systems. As shown in Figure ??, the central idea is that each transaction operates in two distinct phases:

- **Growing phase:** The transaction may acquire locks but cannot release any.
- **Shrinking phase:** Once the first lock is released, the transaction cannot acquire any new locks.

This discipline guarantees serializability by preventing cyclic dependencies in lock acquisition. However, plain 2PL does not prevent all anomalies. In particular, it allows **cascading aborts**: if one transaction aborts after another has read its uncommitted data, the dependent transaction must also abort. This leads to wasted work and reduced system efficiency.

To address this, **Strict Two-Phase Locking** was introduced. Under strict 2PL, a transaction holds all its locks until it either commits or aborts. Locks are only released at the end of the transaction. This eliminates cascading aborts, since no other transaction can observe uncommitted writes, and ensures that the system only produces conflict-serializable schedules.

3.3 Phantom Problem

Even under two-phase locking, there remains **phantom problem**. This occurs when a transaction executes the same query multiple times and obtains different results because another transaction has inserted (or

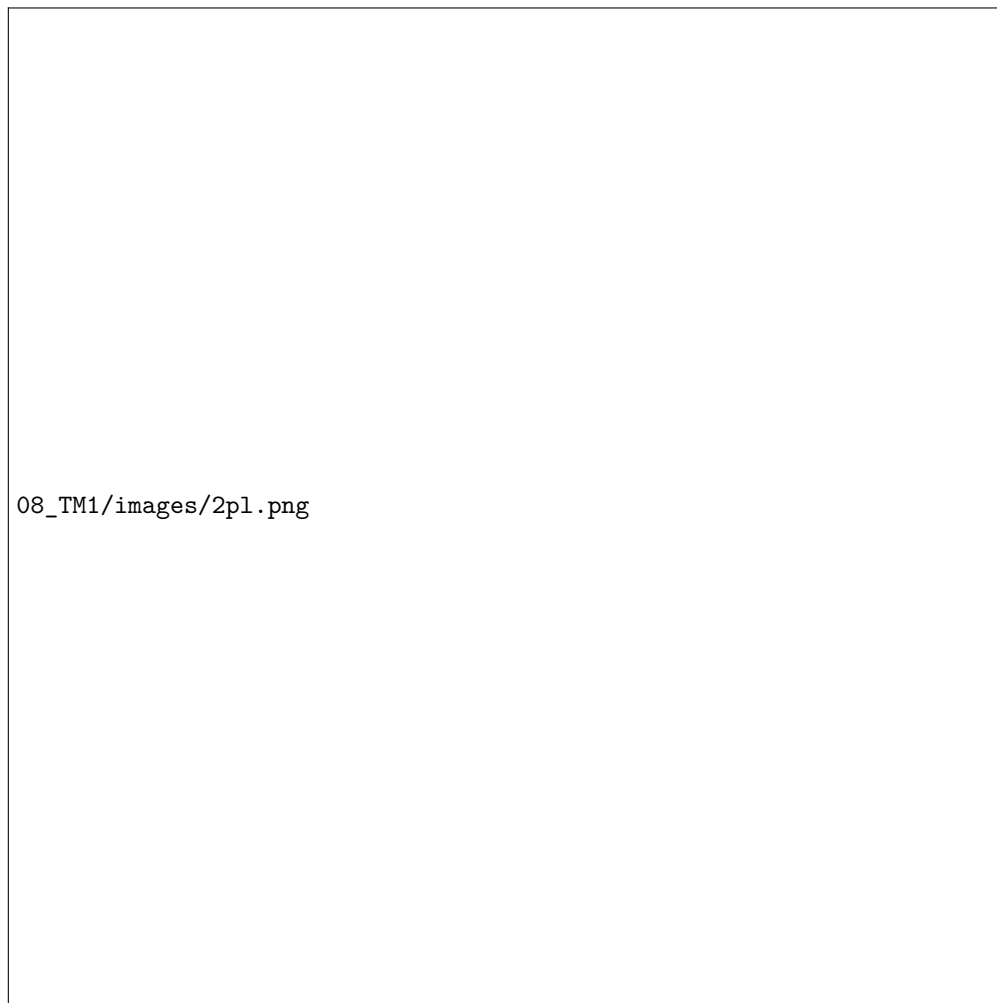


Figure 5: Two Phase Locking



Figure 6: Phantom Problem

deleted) tuples that satisfy the query predicate.

As shown in Figure ??, transaction T_1 executes a query to count the number of people with `status='lit'` and obtains the result 99. Meanwhile, transaction T_2 inserts a new tuple with `status='lit'` and commits. When T_1 re-executes the same query, the result changes to 100. From T_1 's perspective, this violates repeatable read semantics, even though both transactions are individually correct.

This anomaly arises because tuple-level locks alone cannot protect against the appearance of new tuples matching a predicate. The solution is to use **predicate locking** (or index-range locking), which locks not only the currently existing records but also the logical range defined by the query predicate.

4 Isolation Levels

The discussion of two-phase locking and the phantom problem highlights that not all anomalies are eliminated by basic locking protocols. To systematically capture the trade-off between performance and consistency, the SQL standard defines four isolation levels, each providing different guarantees against problems such as dirty reads, non-repeatable reads, and phantoms.

4.1 Anomalies Prevented at Each Level

The SQL standard levels correspond to the following protection guarantees:

Isolation Level	Dirty Read	Unrepeatable Read	Phantom
Serializable	No	No	No
Repeatable Read	No	No	Maybe
Read Committed	No	Maybe	Maybe
Read Uncommitted	Maybe	Maybe	Maybe

4.2 Locking Implementations

The levels can also be understood in terms of locking mechanisms:

- **Serializable:** Obtain all locks first; includes index locks, plus strong strict 2PL.
- **Repeatable Read:** Same as above, but no index locks.
- **Read Committed:** Same as above, but shared (S) locks are released immediately.
- **Read Uncommitted:** Same as above, but allows dirty reads (no S locks).

4.3 Database System Defaults

Different DBMSs adopt different defaults and maximum supported levels:

DBMS	Default	Maximum
Actian Ingres	Serializable	Serializable
IBM DB2	Cursor Stability	Serializable
CockroachDB	Serializable	Serializable
Google Spanner	Strict Serializable	Strict Serializable
MS SQL Server	Read Committed	Serializable
MySQL	Repeatable Reads	Serializable
Oracle	Read Committed	Snapshot Isolation
PostgreSQL	Read Committed	Serializable
SAP HANA	Read Committed	Serializable
VoltDB	Serializable	Serializable
YugaByte	Snapshot Isolation	Serializable

5 Summary

This paper established a foundational framework for concurrency control in database systems. By introducing intention locks, it provided a systematic way to balance concurrency against locking overhead, and influenced the SQL standard.

For a long time, many database platforms relied primarily on locking-based protocols for concurrency control. However, alternative approaches such as Optimistic Concurrency Control (OCC) and Multiversion Concurrency Control (MVCC) have since emerged. MVCC, in particular, creates new versions of data upon updates rather than updating in place, and can be combined with locking or OCC to provide better performance–consistency trade-offs.

Today, modern systems employ a wide variety of concurrency control mechanisms, each offering different balances between the degree of consistency and system performance. The legacy of this work is that it set the direction for decades of database design, shaping how we reason about locks, schedules, isolation levels, and correctness in transactional systems.

Lecture #10: Optimistic Concurrency Control

15-721 Advanced Database Systems (Fall 2025)

<https://www.cs.cmu.edu/~15721-f25/>

Carnegie Mellon University

Notes by: Roy Huang

1 Motivation: Observations About Locking-Based Concurrency Control (CC)

Locking-based protocols, while providing correctness, often present inherent disadvantages. Locking is fundamentally **pessimistic**.

- **Overhead for Reads:** Even read-only queries are slowed down because they must acquire read locks. This is expensive, especially in transactional systems where 90% of queries might be readers.
- **Deadlock Management:** Locking requires methods for deadlock detection or prevention, which is expensive (e.g., assembling a wait-for graph or using protocols like wound-wait).
- **Disk Residency/Hot Objects:** If a database holds a lock on an object residing on disk (secondary memory), performance suffers greatly, especially if the object is “hot” (frequently accessed, e.g., the root of a tree).
- **Long Lock Hold Times:** Locks must be held until the end-of-transaction (assuming serializable mode/strict 2PL), lowering concurrency.
- **Paying Overhead Always:** Locking is needed only in the worst case, even for writers, but we always pay the locking overhead.

The core question motivating Optimistic Concurrency Control (OCC) is: *Can we be optimistic and more efficient in the common case of non-conflicting transactions and read-heavy transactional workloads?*

1.1 The Optimistic Approach

OCC protocols rely mainly on **transaction backup** (abort and restart) as a control mechanism, hoping that conflicts will not occur. This is good in environments where conflicts are rare.

- **Wasted Work:** In OCC, when a conflict occurs, the abort is wasted work. Pessimistic protocols would have stopped conflicting transactions earlier.
- **Read Set Maintenance:** OCC transactions must maintain read sets (the objects they read). This check is pushed to the validation phase.
- **Starvation:** OCC must still worry about starvation for long-running transactions. A solution for starving transactions is to restart them after a few retries in “exclusive” mode (which is essentially like grabbing a lock).

2 Transaction Model: Three Phases

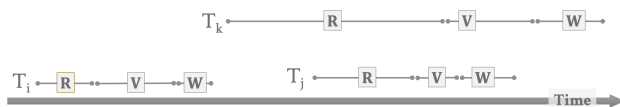


Figure 1: Examples of transactions and the three validation phases

The OCC transaction model consists of three phases:

1. **Read Phase:** Transactions read and write objects, making local copies. Reads are completely unrestricted (unless returning a result, which is subject to validation). All writes occur on local copies of nodes to be modified.
2. **Validation Phase:** Check for serializable schedule-related anomalies. This is the critical step to make OCC work.
3. **Write Phase:** If validation succeeds, the changes are safe, and the local objects are made permanent/global.

2.1 Goal: Serial Equivalence

The goal of OCC is to guarantee that the final state of the database (d_{final}) is the same as if the transactions were applied serially in some specific permutation π .

$$d_{final} = T_{\pi(n)} \circ T_{\pi(n-1)} \circ \dots \circ T_{\pi(1)}(d_{initial})$$

Determining this permutation π to maximize concurrency is the problem OCC solves.

3 Preliminaries: Object Interface

The underlying system provides procedures for manipulating objects. An object can be anything: a table, a page, a record, etc..

3.1 Basic Object Procedures

- **create():** Create a new object and return its name.
- **delete(n):** Delete object n .
- **read(n, i):** Read item i of object n (return the value).
- **write(n, i, v):** Write value v as item i of object n .

3.2 Support Procedures

- **copy(n):** Create a new object that is a copy of object n and return its name.
- **exchange(n1, n2):** Exchange the names (pointers/references) of two objects.

3.3 Read and Write Phase Functions (Transactional Interface)

Transactions are required to use the prefix t : **tcreate**, **tdelete**, **tread**, and **twrite**.

- **tcreate:** Creates a “Private” new object that is not yet visible to others.
- **twrite:**
 - Writes directly to new objects the transaction has created (if n is in the create set).
 - Otherwise, it writes to the transaction’s private/local copy.
 - If it is the first write to an existing object, it performs a **Copy-on-write**, creating a new local copy ($m := \text{copy}(n)$) and adding n to the **write set**.
- **tread:**
 - Remembers what was read, adding n to the **read set**.
 - If n is in the **write set**, the transaction reads from its private write set (the local copy).
 - Otherwise, it reads from the global object n .
- **tend:** If validation succeeds, it makes changes permanent in the “global” database. The changes are made permanent by using the **exchange** operation for every object n in the **write set**. This process involves a pointer switch, but note that the object now resides in a new location, potentially losing physical ordering or clustering properties. Cleanup of local state (read, write, create, delete sets) occurs afterward.

4 Validation: Assigning Transaction Numbers (TN)

Assigning the transaction number (TN) is a critical decision as it sets the serialization order (π).

- **Timing:** Assigning the TN when the transaction enters the system is not optimistic. Transaction numbers can be assigned at the end of the read phase (more optimistic), or delayed even further into the validation phase.
- **Mechanism:** A global counter (using atomic increment) assigns TNs, but this can be a bottleneck. Hardware clocks, such as the Intel RDTSC instruction (which retrieves the CPU cycles since the counter was last reset), offer an alternative.

5 Validation Conditions (The Three Checks)

Validation ensures that the chosen serialization order (T_i comes before T_j if $t(i) < t(j)$) is valid. If any one of the three conditions below holds for all transactions T_i preceding the transaction T_j being validated, T_j is deemed safe to commit relative to T_i .



Figure 2: Summary of OCC Validation Checks

5.1 Condition 1 (Trivial Case)

- **Condition:** T_i completes its write phase before T_j starts its read phase.
- **Result:** No conflict, as all of T_i 's actions happen entirely before T_j 's actions. This is pure serial execution by definition.

5.2 Condition 2 (Partial Overlap)

- **Condition:** The write set of T_i does not intersect the read set of T_j :

$$WriteSet(T_i) \cap ReadSet(T_j) = \emptyset$$

AND T_i completes its write phase before T_j starts its write phase.

- **Checks Covered:** This ensures that T_i 's writes did not affect T_j 's reads. Since T_i finishes writing before T_j starts writing, T_j does not overwrite T_i .

5.3 Condition 3 (Maximum Overlap)

- **Condition:** The write set of T_i does not intersect the read set or the write set of T_j :

$$WriteSet(T_i) \cap ReadSet(T_j) = \emptyset \quad \text{AND} \quad WriteSet(T_i) \cap WriteSet(T_j) = \emptyset$$

AND T_i completes its read phase before T_j completes its read phase.

- **Note on TN Assignment:** The requirement that T_i completes its read phase before T_j completes its read phase (if $t(i) < t(j)$) is automatically satisfied if TNs are assigned at the end of the read phase.

If a transaction T_j fails to meet any of these three conditions with a prior transaction T_i , it must abort and restart.

6 Flavors of Validation

The two flavors of OCC validation are Serial and Parallel.

6.1 Serial Validation (Cases 1 and 2)

Serial validation only relies on Conditions 1 and 2, which implies that write phases must be serial. This is achieved by placing the assignment of the TN, validation, and the subsequent write phase all inside a **critical section**.

- **Process:**
 1. Start TN (*start tn*) is read in `tbegin`.
 2. Transaction reaches `tend` and enters the critical section.
 3. A tentative TN (*finish tn*) is assigned.
 4. Validation loops over all committed transactions t from *start tn* + 1 to *finish tn*, checking if $WriteSet(T_t) \cap ReadSet(T_j) = \emptyset$ (Condition 2).
 5. If valid, the write phase executes, the TN is officially incremented and assigned, and cleanup occurs.
 6. If validation fails, the transaction is aborted and restarted, and the tentative TN is freed (not assigned).
- **Applicability:** Serial validation is suitable if there is just one CPU/core and the writes are “fast”.

6.1.1 Serial Validation: Improved Version

To improve efficiency, validation checks can be moved outside the critical section to minimize blocking.

- Before entering the critical section, *tnc* is read (assigned to *mid tn*). Checks against transactions committed between *start tn* + 1 and *mid tn* are performed outside the critical section.
- Only transactions that committed during the short window while the validation was running need to be checked inside the critical section.
- This iterative process can be repeated to further reduce the work inside the critical section.

6.1.2 Read-Only Queries

Read-only queries do not need to be assigned a TN or enter the critical section. They only need to read *tnc* at the end of the read phase (as *finish tn*) and check Condition 2 (intersection with write sets of committed transactions).

6.2 Parallel Validation (Cases 1, 2, and 3)

Parallel validation uses all three validation conditions (1, 2, and 3), allowing greater concurrency, especially when multiple CPUs are present.

- **Active Set:** A set of transactions **active** is maintained globally, listing transactions that have completed their read phase but have not completed their write phase.
- **Critical Section (Start):** The transaction enters a critical section to grab a copy of the currently **active** set (**finish active**) and adds its own ID to the global **active** set. Entry to this first critical section is equivalent to being assigned a “tentative” TN.
- **Validation (Outside CS):** The transaction performs Condition 2 checks against transactions t from *start tn* + 1 to *finish tn* (backward validation).
- **Condition 3 Check:** The transaction then checks Condition 3 against every transaction i in the **finish active** set.
- **Critical Section (End) and Write Phase:** If validation succeeds, the transaction enters a second critical section to officially assign its TN, remove itself from the **active** set, and then executes the write phase and cleanup **outside** the critical section. The writing is safe because transactions that conflict with the committed transaction (which is now done writing) will be blocked/failed due to the **active** set status.

7 Workload Impact and Outlook

7.1 Conflict and Performance

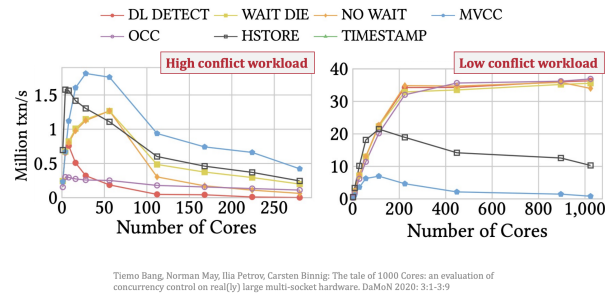


Figure 3: Throughput of various concurrency control algorithms on TPC-C

- OCC works **great** if the workload has **low conflict** or is read-heavy (e.g., updating different records or query-dominant systems).
- If contention is high, OCC results in lots of **wasted work** and rollbacks. Pessimistic locking may be superior in high-contention environments.
- For applications involving very large tree-structured indexes (like B-trees), OCC is favorable because the size of read and write sets is bounded by a small constant (e.g., 4 for a B-tree of depth 5 with 200 million keys), ensuring validation is fast. Furthermore, for B-trees, the read phase is typically orders of magnitude longer than the validation and write phases, often involving secondary memory access, making OCC suitable.

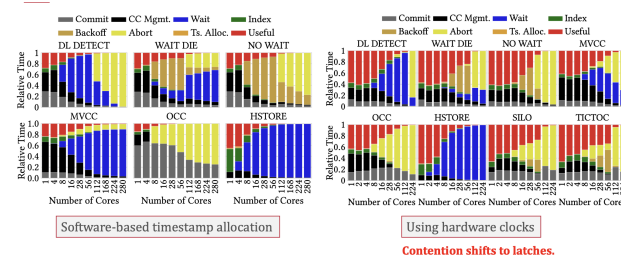


Figure 4: Time breakdown for TPC-C

7.2 Concurrency Control Mixes

OCC and Locking are two alternative ways to check/prevent conflicts in concurrent transactions.

- MVCC + Locking:** Used in systems like PostgreSQL. Readers use a snapshot, while writers create a new version and grab a write lock to stop other writers.
- MVCC + OCC:** Used in systems like Hekaton. Readers use a snapshot, and writers use an OCC validation phase.

7.3 Scaling OLTP

- Many OLTP workloads fit on a single large machine (e.g., boxes with 28 CPUs, 1568 logical cores, and 20 TB of DRAM).
- Unlike analytic workloads, adding more nodes (distributed/geo-replication) generally **reduces performance** in an OLTP system due to increased synchronization overhead. Performance can decrease quadratically with the number of nodes in the worst case.

Lecture #11: Memory-Optimized OLTP—Hekaton

15-721 Advanced Database Systems (Fall 2025)

<https://www.cs.cmu.edu/~15721-f25//>

Carnegie Mellon University

Notes by: Sirui Huang

1 Background

Today, compared to OLAP systems, many OLTP databases are smaller in size, often fitting entirely in memory. When processing their workloads, OLTP databases often have to manage high contention to synchronize concurrent transactions. Especially in distributed settings, scaling out can be costly due to more contention and network overhead. Therefore, OLTP database vendors often choose to store the entire workload in memory to reduce latency and improve throughput.

Based on this understanding, researchers at Microsoft studied SQL Server’s transaction processing performance and defined several metrics to analyze the bottlenecks in OLTP systems:

- CPI: Cycles per instruction, measures how many CPU cycles are needed to execute one instruction; it translates to how many transactions can be processed in a given time.
- SF: Scalability factor, measures how well the database’s concurrency control (CC) mechanism (design and implementation) scales with more threads or similar resources; it translates to how well the system handles concurrent behavior and utilizes parallel resources (e.g. CPU cores).
- IR: Instructions required, measures how many instructions are needed to run a given transaction; it translates to how much work the system needs to do at transaction granularity.

SQL Server measured less than 1.6 CPI on common benchmarks and 1.89 SF on TPC-E benchmark, indicating that SQL Server is already efficient in utilizing CPU cycles and scaling with more threads. This suggests that further improvements in transaction speedup should focus on reducing IR, the number of instructions for each transaction. This inspired the design of Hekaton, a memory-optimized OLTP engine in SQL Server.

2 Hekaton

Hekaton is a memory-optimized OLTP engine sitting atop SQL Server as an optional extension for in-memory transaction processing. Based on its memory-focused design, Hekaton reimagines several components of a traditional OLTP database system, including data structures, concurrency control, and code generation.

2.1 Data Structures

Hekaton uses in-memory data structures to eliminate buffer pool overhead. This includes utilizing an in-memory hash table as its main storage structure and building range indices like Bw-trees on top of it to accelerate data accesses. Indices will not be logged and would be rebuilt from scratch during recovery. Hekaton also applies checkpointing and logging techniques for its main table to support recovery in case of failures.

2.2 Concurrency Control

All data structures in Hekaton are designed to be latch-free to avoid latching overhead. Hekaton also uses an optimistic multi-version concurrency control (MVCC) mechanism to manage concurrent transactions.

2.3 Code Generation

Finally, Hekaton uses a code generation scheme to compile statements and stored procedures into some native code at runtime. This allows Hekaton to convert query execution plans into efficient machine code, thus reducing the number of instructions required.

All three components work together to reduce IR. Based on the types of workloads SQL Server targets, Hekaton does not partition data across multiple cores. Hekaton is integrated into SQL Server; for users to leverage Hekaton, they need to explicitly mark tables as in-memory when creating them.

3 Storage Structure

Hekaton uses a combination of a main hash table and Bw-tree indices to store data in memory. The main table is a hash table that stores all records. Each record has multiple versions to support MVCC. Bw-trees are built on top of the main table to accelerate data accesses. Each index entry in a Bw-tree points to a record in the main table.

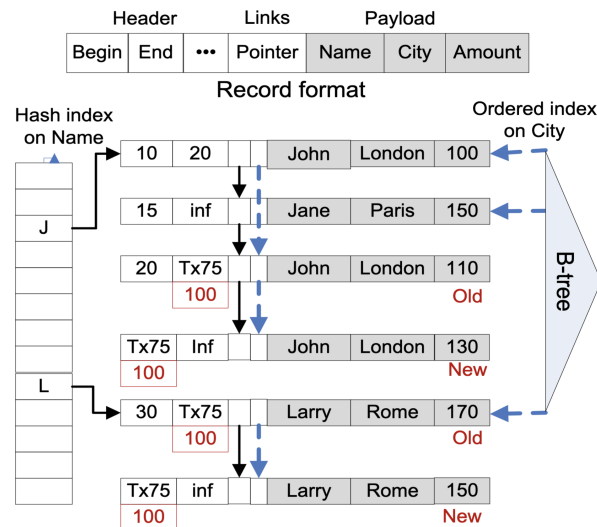


Figure 1: Hekaton storage structure

3.1 Record Format

Each record in Hekaton consists of three parts: Header, Links, and Payload. The Header contains metadata about the record, in particular transaction timestamps to support MVCC. The Links part contains pointers to the previous and next versions of the record for the hash table and indices for preserving version chains. The Payload part contains the actual fields of the record.

3.2 Hash Table

Hekaton's main table is a hash table that maps keys to records. Each bucket in the hash table contains a pointer to the head of a linked list of records that hash to the same bucket. The linked list is formed by its set of pointers in the Links part of each record. The linked list is sorted by transaction timestamps to support MVCC.

When a transaction modifies a record, it creates a new version of it and updates the Links part of the record to point to the new version. Before the transaction ends (commits or aborts), it writes a transaction timestamp as the end timestamp of the previous version and as the start timestamp of the new version. These transaction timestamps are different from the actual timestamps assigned when the transaction ends and serve as placeholders. Other transactions can then use these timestamps to determine which version

of the record they can access based on their own transaction timestamps. After the transaction ends, the actual timestamps are written in the place of the transaction timestamps.

3.3 Bw-tree Indices

Bw-trees are used as secondary indices on top of the main hash table. They are latch-free B+ trees that use delta records to represent changes and applies atomic updates using compare-and-swap (CAS) operations. They support rapid concurrent accesses and modifications to allow transactions to read and write data efficiently.

Each index entry in a Bw-tree points to a record in the main table. Each Bw-tree preserves its version chains using its designated sets of pointers in the Links part of each record.

The Bw-trees also maintain a log-structured page storage. Modifications to pages are carried out lazily by keeping track of them in append-only logs. Periodically, a background process consolidates the logs into new pages to reduce the number of indirections needed to access data. This technique helps reduce Bw-tree maintenance overhead for each page modification.

4 Code Generation

Hekaton maintains a mini compiler that generates native code for query execution plans and stored procedures at runtime. T-SQL stored procedures, the procedural programming language used in SQL Server, is first parsed into an intermediate representation called MAT (Memory-Optimized Assembly Tree) in the SQL engine. MAT is then translated into PIT (Platform-Independent Tree) in the code generation phase.

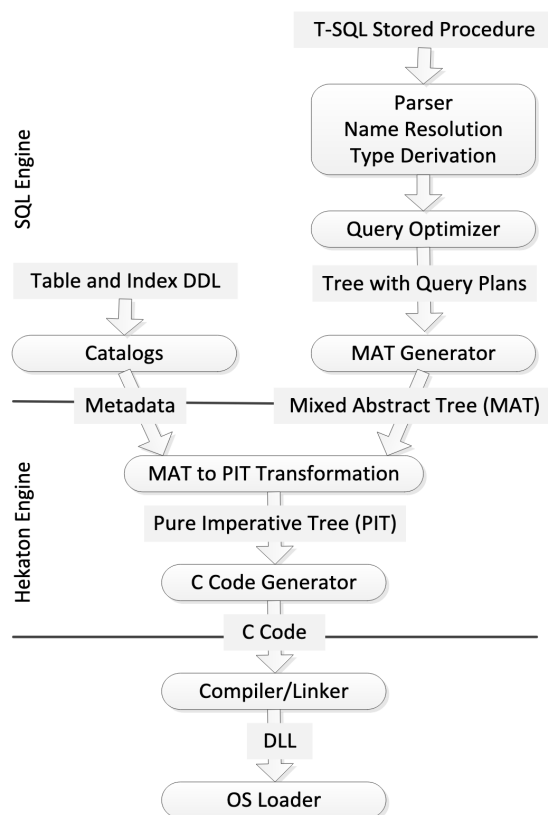


Figure 2: Hekaton compiler procedures

PIT is a platform-independent representation of the execution plan. It abstracts away low-level details like

CPU architecture and memory layout. Finally, PIT is translated into C code, which is then compiled into native machine code using a C compiler. The generated machine code is then executed directly by the CPU to process queries and stored procedures.

Table creation in Hekaton also requires code generation. Since records are opaque to the compiler, the compiler needs to provide the engine with customized callback functions for each table (e.g., `compareRecords()`).

The compiler also needs to handle several other challenges, such as type mismatch between T-SQL and C-types, null-handling for operations like outerjoins, and managing different exception semantics (e.g. division by zero).

Finally, at the time of implementation, a compiled stored procedure cannot access regular (non in-memory) tables. Hekaton compensates for this by supporting Query Interop, operators for accessing data in Hekaton tables usable in interpreted SQL Server query plans.

5 Concurrency Control

Hekaton uses an optimistic MVCC mechanism to manage concurrent transactions. This scheme supports multiple isolation levels, which also includes snapshot isolation.

5.1 Comparison with other Concurrency Control Mechanisms

5.1.1 Comparing Memory-optimized Engine with Traditional Disk-oriented Engine

To make sense of how optimistic MVCC works in Hekaton, we first compare the traditional disk-oriented engine with Hekaton's memory-optimized engine and examine the tradeoffs. In a traditional disk-oriented engine:

- Disk-friendly data structures (e.g., pages and B+ trees) are used to minimize disk I/O.
- Applies frequent context switching to hide disk latency.
- Threads wait on latches to protect shared data structures.
- Locks are used to ensure isolation between transactions.
- Critical sections (i.e., code segments susceptible to race conditions) span thousands of instructions, impeding scalability.

Meanwhile, for a memory-optimized engine like Hekaton:

- Memory-friendly data structures (e.g., hash tables and Bw-trees) are applied to optimize for in-memory access patterns on individual records.
- Minimizes context switching (usually 1, at most 2) since memory access is fast.
- No latches are used; data structures are designed to be latch-free.
- There are no locks, and transactions do not block each other.
- Transactions can span many instructions, but only 1 critical section: incrementing atomic counter.

5.1.2 Comparing Hekaton with H-Store

Now, we compare Hekaton against H-Store, another concurrency control mechanism based on the assumption that few transactions will communicate across partitions. H-Store displays mainly the following characteristics:

- Mainly scales out.
- Communication across partitions can be expensive.
- Only one CPU is in charge of a partition and accesses records on it at a time.
- Transactions that access multiple partitions are coordinated using a two-phase commit protocol.
- Specialized for partitionable workloads.

Hekaton, on the other hand, displays the following characteristics:

- Mainly scales up.
- Workload data is shared coherently in memory.
- Any number of CPUs can access a single record concurrently.
- Isolation is enforced through transactions validating their read and write operations.
- General-purpose OLTP engine; no partitioning required.

5.1.3 Comparing Single Version Concurrency Control with MVCC

Finally, let's compare single version concurrency control with MVCC. Here are the characteristics of single version CC:

- The traditional approach, optimal for handling memory-resident data.
- Each record has only one version.
- Locks are acquired on access and released after transaction ends.

Meanwhile, MVCC displays the following characteristics:

- Each record can have multiple versions.
- Need to identify which version to read based on transaction timestamps.
- Garbage collection is needed to clean up old versions.

Comparing the two and considering both approaches in in-memory settings, we find that MVCC would be more suitable for read-heavy workloads with long-running transactions. We consider two common operations: reads and updates.

- Reads: In single version CC, reads need to acquire locks before reading, which adds up to one memory write and one memory read per record. In MVCC, due to the append-based log structure, only a memory read is needed to find the record.
- Updates: In single version CC, updates is in place, so only one memory write is needed. In MVCC, updates create a new version and also update the previous version's end timestamp, leading to two memory writes.

From the above analysis, we see that MVCC is more efficient for reads (1 memory read vs. 1 memory read + 1 memory write) but less efficient for updates (2 memory writes vs. 1 memory write). It is worth noting that in single version CC, readers need to write to memory to acquire locks. Since in most transaction workloads, even update-heavy ones, reads dominate updates, MVCC is generally much more efficient.

Note: For the discussion above, we are assuming reads are mainly on the latest version of records. When we consider historical reads, more memory reads might be needed to traverse version chains in MVCC, leading to more complex tradeoffs.

MVCC still has extra advantages, such as its record versioning which provides isolation between readers and writers, thus avoiding blocking and deadlocks. A typical example is when a transaction tries to read a record that is being updated by another transaction. In single version CC, the reader would block until the writer finishes and releases the lock. In MVCC, the reader can simply read the previous version of the record without blocking if it is sufficient for its isolation level and its demand.

5.2 Achieving Snapshot Isolation with Optimistic MVCC

We now explain how Hekaton's optimistic MVCC mechanism achieves **snapshot isolation**. Each transaction has two unique timestamps: a start timestamp assigned when it begins and an end timestamp assigned when it ends (commits or aborts). To achieve snapshot isolation, a transaction reads its records at the time of its start timestamp and writes its changes at the time of its end timestamp.

This protocol also automatically achieves **read-committed** isolation, since a transaction only reads committed records with timestamps earlier than its start timestamp. Writes will always be valid since only one writer can modify a record at a time, and the newest version will always be at the end of the version chain. However, this protocol is **not serializable** or **repeatable read** isolation, since a transaction may read non-committed records created by other concurrent transactions.

First, to achieve **repeatable read** isolation, Hekaton requires transactions to validate their reads before committing. During validation, a transaction checks if any record in its read set has been modified by other transactions since its start timestamp. If so, the transaction aborts. Otherwise, the transaction can proceed to commit its changes. This validation ensures that transactions appear to execute in a serial order, thus achieving repeatable read isolation. **Serializable** isolation can then be achieved by detecting phantom reads using index-range scans with the same predicate during validation.

In summary, in terms of validation:

- Read Committed: no validation needed.
- Repeatable Read: validate reads do not have new versions before commit.
- Serializable isolation: detect phantoms using index-range scans using the same predicate.

Combined with our comparisons before, we see that Hekaton’s optimistic MVCC:

- Uses no latches or locks and is thus highly scalable.
- Isolates readers from writers, making reads efficient.
- Supports all common isolation levels.
- Relieves deadlock issues since there are no locks.

5.3 Transaction States

The lifecycle of a transaction in Hekaton can be represented as the following state machine:

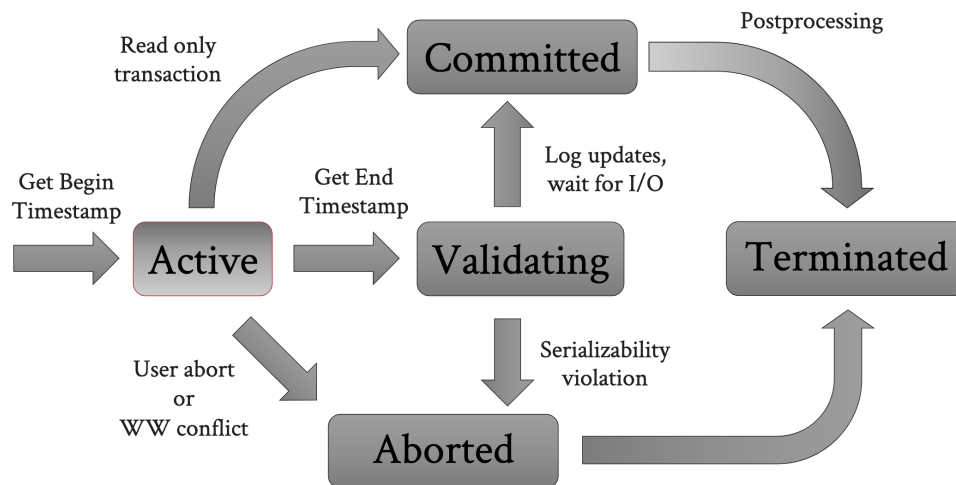


Figure 3: Hekaton transaction states

In particular, the **Active** state indicates the start of a transaction, during which the reads are performed. In the case of read-only transactions, the transaction can directly move to the **Committed** state after finishing its reads. The postprocessing step after the **Committed** state will be for cleaning up data structures constructed (e.g. writing timestamps) during the transaction.

5.3.1 Transaction Map

Based on the states, a **transaction map** will be maintained to track the status of each transaction. This map will include the transaction ID, its current state, and its start and end timestamps. The transaction map is globally visible to all transactions and is used during validation to check the status of transactions that have modified records in the read set. Whether a record is visible is thus dependent on both the timestamps in its header and the state written in the transaction map.

We now go through the steps of a successful update transaction and track how the transaction map changes. In the **Active** state:

- The transaction obtains a unique transaction ID and a start timestamp, each from an atomic counter designated for either.
- The transaction writes its transaction ID and a start timestamp to the transaction map. It also writes the state as **Active**.
- The transaction performs its reads and writes, creating new record versions as needed. The end timestamps of the previous versions and the start timestamps of the new versions are set to its transaction ID as placeholders.

In the **Validating** state:

- The transaction obtains a unique end timestamp from the atomic counter.
- The transaction writes its state to **Validating** and records its end timestamp in the transaction map.
- It then validates its reads based on its isolation level.

After validation succeeds, the transaction moves to the **Committed** state:

- The transaction logs the updates and waits for I/O to complete.
- The transaction writes its state to **Committed** in the transaction map.

Finally, while moving to the **Terminated** state:

- The transaction updates the end timestamps of the previous versions and the start timestamps of the new versions with its end timestamp during postprocessing.
- The transaction cleans up its entries in the transaction map.

5.3.2 Write-write Conflicts

With the above protocol, write-write conflicts are automatically handled. When a transaction creates a new version of a record, it will use its transaction ID as the placeholder for the end timestamp of the previous version. If another transaction tries to create a new version of the same record concurrently, it will see that the end timestamp of the previous version is already set to a transaction ID (not infinity). The second transaction will then abort since it cannot create a new version of a record that is being modified by another transaction.

5.3.3 Write-read Conflicts

However, what if a reader transaction finds a record whose end timestamp is set to a transaction ID? In this case, the reader transaction will check the transaction map to see the state of the writer transaction. The four cases are:

- If the writer is **Active** or **Aborted**, the reader will not read the record. **Active** means the version is not yet committed, while **Aborted** means the version will not be committed.
- If the writer is **Committed**, the reader will then decide whether to read the record based on the end timestamp of the transaction writing it.
- If the writer is **Validating**, the reader will optimistically assume the transaction will commit and read the record. It will then come back during validation to check if the writer did commit. This is because validation can take time, and blocking readers during this period could hurt performance.

The **Validating** case above introduces commit dependencies between transactions. To handle this, transactions can maintain a counter to track how many other transactions they depend on to commit. These counters are initialized to the number of dependent transactions when reading, and decremented when a dependent transaction commits (this can be implemented using registered callbacks/signals design pattern). Only when the counter reaches zero, the transaction can proceed to commit. If any dependent transaction aborts, the transaction must also abort, which unfortunately can lead to cascading aborts. The counter technique transforms multiple wait-for relationships into a single one, thus reducing the complexity of tracking dependencies.

6 Evaluation

In this section, Hekaton is evaluated on a machine with 2-socket, 6-core Xeon X5650 with 48GB RAM. The evaluation mainly compares optimistic multi-version concurrency control (MV/O) with single-version concurrency control (1V).

The TATP benchmark models a telecom application, which contains 4 tables, 7 transactions, and sized for 20M subscribers. The benchmark has little contention and consists mostly of short transactions (5 ops / txn).

Limiting all transactions to run under **Serializable** isolation, MVCC performs a bit worse than single-version CC at low concurrency, due to the extra overhead of maintaining multiple versions:

- MV/O: 3,121,494 txns/sec
- 1V: 4,220,119 txns/sec

However, as concurrency increases, MVCC scales better due to its non-blocking nature, while single-version CC suffers from increased latching and locking contention:

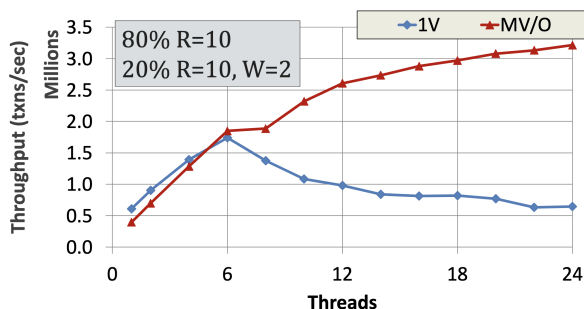


Figure 4: Hekaton contention evaluation results

As shown in the diagram, MV/O shows an approximate 2.0x throughput improvement when increasing the number of threads from 6 to 24. It is also about 5.0x as fast as 1V at 24 threads. This shows MV/O's robustness to contention and its soundness even without throttling.

Finally, we look at the impact of long-running read-only transactions on short update transactions. In this experiment, we change the number of each type of transaction while keeping the total number of threads at 24. Here are the results:

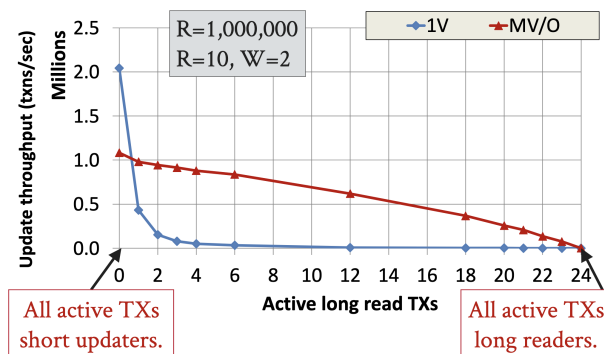


Figure 5: Hekaton long-running reader evaluation results

The diagram shows that when no long-running readers are present, 1V is 1.9x faster than MV/O due to MV/O's extra overhead. However, even with one long-running reader, MV/O outperforms 1V by 2.3x, which

shows that MV/O does not suffer from reader-writer contention. As the number of long-running readers increases, MV/O's performance degrades gracefully, while 1V's performance drops sharply due to increased contention.

Lecture #13: Consensus on Transaction Commit

15-721 Advanced Database Systems (Fall 2025)

<https://www.cs.cmu.edu/~15721-f25//>

Carnegie Mellon University

Notes by: Mingqi Lu

1 Introduction

In a single-node database system, committing a transaction usually involves forcing the write-ahead log (WAL) to stable storage so that the effects of the transaction are guaranteed to persist even after failures. In a distributed database, however, a transaction typically spans multiple resource managers (RMs) that execute on different machines. In such an environment, a single local flush is insufficient to ensure that all participants reach the same decision on whether to commit or abort. Therefore, a distributed transaction commit protocol is required to coordinate these independent RMs and guarantee that all of them make a consistent decision. The protocol must ensure that either every participant commits or every participant aborts (atomicity), that once a decision is recorded it remains stable and durable even after failures.

RM state machine. An RM starts in *working*. After local checks and a forced log record, it may enter *prepared*. Only when **every** participant is prepared may the system commit; otherwise it must abort (Fig. 1).

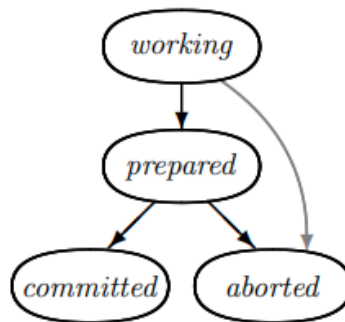


Figure 1: RM state transitions.

2 Two Phase Commit

Two-Phase Commit (2PC) relies on a single coordinator, the **Transaction Manager (TM)**, to collect votes and decide commit/abort. Each RM forces its local state to stable storage before sending messages while in that state so it can recover after crashes.

2.1 Classic 2PC

Phase 1: Prepare.

1. TM broadcasts PREPARE to all RMs.
2. Each RM: (i) decides YES (can commit) or NO (must abort); (ii) forces its vote to its log; (iii) replies to TM.
3. TM collects votes. If any NO or a timeout occurs, TM aborts; if all YES, TM may commit.

Phase 2: Commit/Abort.

1. TM forces the decision (commit/abort) to stable storage, then broadcasts the decision.
2. Each RM forces the received decision, applies local changes/undo as appropriate, and sends ACK. TM releases transaction metadata after all ACKs.

Message complexity. With N RMs: prepare $(N-1)$, votes $(N-1)$, decision $(N-1) \Rightarrow$ about $3N-3$ messages. If the initiating RM is not the TM, add two direct messages for start/finish notification. A standard timeline is shown in Fig. 2.

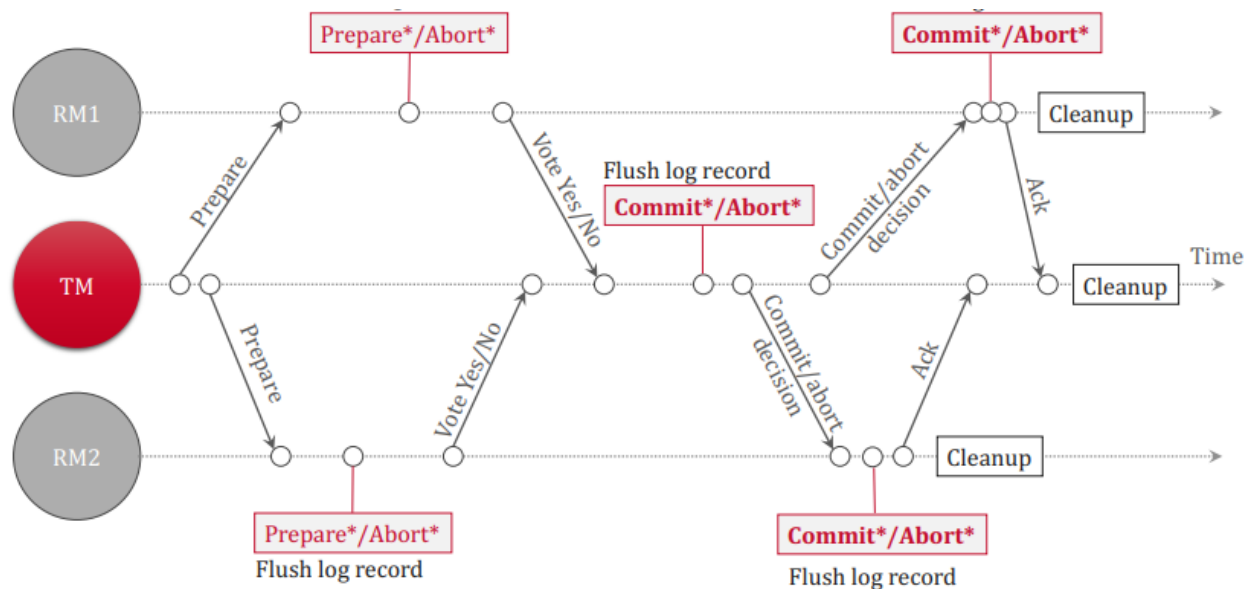


Figure 2: Classic 2PC timeline

Failure behavior and blocking. 2PC is **coordinator-centric**. If the TM crashes before broadcasting a forced decision, participants are stuck waiting, even if all had voted YES. This can cause a blocking problem: 2PC can halt until the TM recovers.

2.2 Presumed Abort 2PC

Presumed Abort reduces work on the **abort** path by treating abort as the default outcome of a transaction. This design significantly cuts down the number of forced log writes and network messages needed during abort-heavy workloads.

- **Logging:** No explicit ABORT record is forced to disk. During recovery, if an RM finds no PREPARE or COMMIT record in its stable log, it can safely assume that the transaction was aborted. This eliminates unnecessary log writes in the frequent case where transactions fail or are canceled.
- **Messaging:** The TM does not need to broadcast abort messages to RMs that already voted NO during the prepare phase, since those RMs have already decided to abort. This reduces the number of messages exchanged in the abort case.
- **Read-only RMs:** RMs that only perform reads can drop out immediately after the prepare phase by voting READ. They do not log any prepare or commit records and do not participate in the second phase, further reducing coordination cost.

These optimizations reduce both the I/O overhead and the number of messages when many transactions abort or when read-only participants are common. Fig. 3 highlights the behavioral differences of the PA protocol.

2.3 Presumed Commit 2PC

Presumed Commit reduces work on the **commit** path by treating commit as the default outcome. This optimization is effective for workloads where most transactions commit successfully, as it minimizes the number of forced log writes and acknowledgment messages.

- **Logging:** RMs do not need to force PREPARE or COMMIT records to disk, since the system assumes that transactions will commit by default. However, to ensure correct recovery, the TM must write a collecting record before broadcasting the prepare messages to keep track of which RMs might have prepared. This record allows the TM to determine which participants were involved if it needs to abort the transaction after a crash.
- **Messaging:** During the second phase, RMs do not send explicit ACKs back to the TM after receiving the commit decision, because commit is presumed. Once an RM receives the decision, it can directly commit locally and perform cleanup without waiting for further coordination, reducing both latency and message count.

PC is particularly efficient when most transactions commit and failures are rare, as it avoids unnecessary logging and communication on the common path.

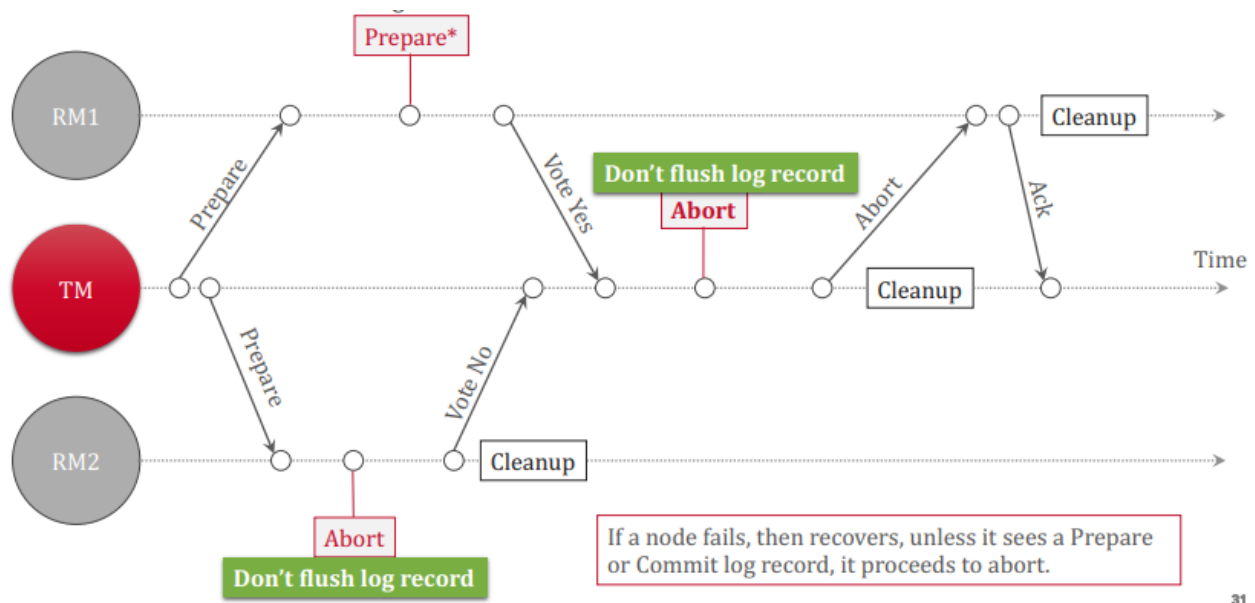


Figure 3: Timeline of the 2 Phase Commit PA protocol.

3 Distributed Consensus Protocol

This section places atomic commit into the broader context of distributed consensus and clarifies how 2PC, Paxos, and Raft differ. CAP serves as a way to reason about coordination trade-offs in practice.

3.1 2PC vs. General Consensus

Two-Phase Commit (2PC) solves a specialized agreement problem: decide commit or abort for a single transaction, driven by a single coordinator (TM). It ensures atomicity but can block if the coordinator fails at the wrong time; participants may be stuck until the TM recovers. In contrast, consensus protocols (e.g., Paxos and Raft) solve a more general problem: agree on an arbitrary value despite failures. They guarantee safety under crash faults and can make progress when leadership is stable and a quorum of nodes is alive.

3.2 Comparison between 2PC, Paxos, and Raft

- **2PC:** simple and efficient on the happy path but blocks if the TM fails after prepare until it recovers.

- **Paxos:** non-blocking when a majority is alive and there is a long enough period without further failures.
- **Raft:** similar guarantees as Paxos but with fewer node roles and the additional constraint that only up-to-date nodes can lead.

3.3 CAP Theorem

CAP states that under a network partition a system must choose between **consistency** and **availability**; it cannot simultaneously guarantee consistency, availability, and partition tolerance. PACELC refines this by noting that even without partitions there is a trade-off between **latency** and **consistency**.

4 Conclusion

Distributed transaction commit protocols evolve from the classic Two-Phase Commit (2PC) design toward more resilient and efficient schemes inspired by distributed consensus. Classic 2PC remains a simple and practical mechanism to ensure atomic commitment but suffers from blocking when the coordinator fails. The Presumed Abort (PA) and Presumed Commit (PC) variants refine this by optimizing the common execution paths. PA reduces abort-path logging and messaging overhead while allowing read-only participants to exit early, and PC trims commit-path costs through reduced acknowledgments and selective logging. Consensus-based protocols such as Paxos and Raft generalize the agreement problem, enabling systems like Paxos Commit to tolerate coordinator failures and ensure progress as long as a majority of nodes remain available. These advancements reflect a broader trend in distributed systems toward balancing efficiency, fault tolerance, and scalability.

Lecture #14: Analytics at Scale: Dremel and BigQuery

15-721 Advanced Database Systems (Fall 2025)

<https://www.cs.cmu.edu/~15721-f25//>

Carnegie Mellon University

Notes by: Ahmed Chaudhry

1 Analytics at Scale: Why Hadoop Was Problematic

In the early 2000s, as Google experienced massive data growth, it became apparent that existing commercial database systems could not support the scale required for both transactional (OLTP) and analytical (OLAP) workloads. To overcome these limitations, Google began developing its own internal systems.

One of its early and influential innovations was MapReduce, a framework designed for massively parallel data processing tasks, including SQL-like queries, ETL (Extract, Transform, Load) operations such as web-page scraping, and early machine learning computations. The MapReduce paradigm—dividing computation into map and reduce phases—was revolutionary and inspired the open-source system Hadoop.

However, despite Hadoop’s popularity in the external world, Google soon recognized significant limitations in this approach for analytical workloads. Internally, MapReduce was considered “a terrible way to do parallel processing” for several reasons:

- **Lack of Declarative Interface:** MapReduce required users to write imperative programs to define computations. This was less productive and more error-prone than declarative query languages such as SQL.
- **No Query Optimization:** Because MapReduce lacked a declarative interface, the system could not automatically optimize execution. Programmers had to define the entire execution DAG (Directed Acyclic Graph) manually, which often led to inefficient execution plans.

As a result, Google quickly transitioned toward SQL and database-style query processing. This shift improved both programmer productivity and system efficiency. Over time, even many NoSQL systems began to embrace SQL, demonstrating the enduring value of declarative query languages.

2 PIPE SQL: Addressing Query Optimizer Challenges with CTEs

While Google has fully embraced ANSI SQL for its cloud platforms, it continues to innovate in SQL syntax to improve developer usability. One such innovation is PIPE SQL.

Traditionally, developers writing complex SQL queries use Common Table Expressions (CTEs)—defined with the `WITH` clause—to organize intermediate results. CTEs make deeply nested queries more readable and maintainable by linearizing the query structure and defining named temporary result sets that can be referenced later.

Although CTEs improve human readability, they sometimes present challenges for Query Optimizers (QO). The linear and modular structure that benefits developers may not always yield efficient query plans, as optimizers can find it difficult to globally optimize multiple nested CTEs.

PIPE SQL was proposed to address this issue by providing a more natural, data-flow-oriented syntax. Instead of starting with `SELECT` and later specifying `FROM`, PIPE SQL often begins with `FROM`, mirroring the intuitive way humans think about how data flows through transformations. It introduces operators like `EXTEND` to make each transformation step explicit.

Key characteristics of PIPE SQL include:

- **Syntactical Sugar:** PIPE SQL does not add new computational capabilities; it reorganizes existing SQL constructs into a clearer, pipeline-like form.
- **Enhanced Readability and Writability:** Its explicit linear flow makes complex queries easier to follow and construct, minimizing the back-and-forth common in nested CTE structures.

PIPE SQL is actively used internally at Google and has also been adopted by major data platforms such as Databricks, Snowflake, and Google BigQuery.

3 Dremel: A Revolutionary Data System

Dremel is one of Google’s most groundbreaking data systems and has had a profound influence on modern analytical data platforms. It received a prestigious 10-year best paper award, highlighting its foundational impact. Dremel was developed to process Google’s massive analytical workloads and introduced several major innovations.

3.1 Native Support for Nested Data Structures:

Many of Google’s internal datasets, particularly those stored in Protocol Buffers (Protobufs), are deeply nested—for example, an order containing customer details, an array of order items, and each item containing multiple nested attributes. Traditional relational databases struggled with such structures, often requiring flattening into wide, redundant tables. Dremel, in contrast, provides first-class support for these hierarchical schemas directly.

3.2 Columnar Representation for Nested Data:

Dremel introduced a method to represent nested data in a columnar format, enabling queries to access only the necessary fields and thereby minimizing I/O. It uses two key metadata constructs:

- **Repetition Level (R):** Indicates when a value repeats within a list or array (e.g., 0 for a new list, 1 for subsequent items). This allows the system to reconstruct nested lists.
- **Definition Level (D):** Indicates the nesting depth of a field and whether a value is present or null at that level. This enables efficient handling of optional and null fields.

This “shredding” technique became the foundation for open-source formats like Parquet.

3.3 Path Expressions and Nested Output:

Dremel’s SQL supports querying nested data through path expressions (e.g., `name.language.code`), treating nested records as labeled trees. It also allows outputting nested results using the `WITHIN` clause, letting developers consume structured data directly without additional transformations.

3.4 Disaggregated Storage Architecture:

Dremel pioneered separating storage and compute into independent layers. Data resides in distributed storage systems like Colossus (formerly GFS), while query execution happens in compute clusters.

- **Benefits:**
 1. Independent scaling of compute and storage resources.
 2. Cost efficiency through specialized resource allocation.
 3. Division of responsibility, allowing dedicated teams to manage storage (replication, fault tolerance) separately from query processing.
- **Initial Challenges:** Early versions suffered a ten-fold slowdown due to frequent metadata lookups on distributed storage. This issue was later mitigated through algorithmic tuning.

This disaggregated model—where storage acts as a managed service—is now the standard architecture for modern cloud databases such as AWS S3, Google Cloud Storage, and Azure Blob Storage.

3.5 In-Situ Analysis (Early Lakehouse Concept):

Dremel introduced the concept of in-place data processing, allowing multiple systems (SQL engines, graph databases, ML frameworks) to access the same shared data in a self-describing format such as Capacitor

(which later influenced Parquet). This eliminated the need for redundant data copies, a precursor to the modern Lakehouse architecture.

4 Lower Tail Latency and Adaptive Query Optimization

Tail latency refers to the latency experienced by a small percentage of operations—typically the slowest 1% or 0.1%—in highly parallel or distributed systems. Even if average latency is low, high tail latency can dominate overall completion time in large-scale jobs that depend on many parallel subtasks.

Dremel’s Approach to Reducing Tail Latency: Dremel’s query optimizer is not heavily rule-based or cost-based like traditional relational optimizers. Instead, it relies on massive parallelism, fault tolerance, and adaptive query processing—largely enabled by its in-memory shuffle service—to achieve high performance and scalability.

4.1 In-Memory Shuffle Service:

In traditional parallel databases, intermediate data exchange between query stages is tightly coupled: producer tasks must remain active until all consumer tasks have consumed their data. This design ties the lifetime of producers to the slowest consumer, increasing tail latency.

Dremel’s shuffle service breaks this dependency. Producers send their intermediate data to the shuffle service, which buffers it in memory. Once the data is sent, producers can terminate and release resources. The shuffle service later delivers the data asynchronously to consumers.

Benefits:

- **Reduced Tail Latency:** Decoupling producers and consumers prevents slow workers from delaying others.
- **Dynamic Resource Management:** The shuffle service creates natural “pause points” between stages, enabling dynamic decisions about resource allocation.
- **Straggler Mitigation:** Slow or failed workers can be detected and their work reassigned without restarting the entire stage.
- **Adaptive Parallelism:** Based on observed data distribution, Dremel can adjust the number of workers for subsequent stages or repartition data if skew is detected.

4.2 Adaptive Join Strategies:

One of the key applications of adaptive processing in Dremel is join optimization.

- **Broadcast Join:** A broadcast join replicates a small table across all nodes processing a larger table. Each node can then perform the join locally, avoiding the costly network shuffle of the larger table.
- **Dynamic Join Selection:** After the initial scan and shuffle phase, Dremel can estimate table sizes and distributions. If one table is significantly smaller, the system can dynamically switch from a repartition (hash) join to a broadcast join, improving performance.
- **Dynamic Repartitioning:** If skewed data is detected—where one partition becomes disproportionately large—Dremel can split it further and distribute it across additional workers, maintaining load balance and reducing straggler effects.

These adaptive strategies, enabled by Dremel’s shuffle service, make the system highly efficient, fault-tolerant, and resilient at massive scales, all while maintaining low tail latency even without employing complex static query optimization techniques.

Lecture #15: Amazon Redshift

15-721 Advanced Database Systems (Fall 2025)

<https://www.cs.cmu.edu/~15721-f25//>
Carnegie Mellon University
Notes by: Aditya Ramesh

1 Introduction

Redshift was launched in February 2013 as Amazon's OLAP DBaaS flagship product in response to the growing demand for fast, user friendly, and cost effective solution for a scalable data warehouse solution. In their market analysis, Amazon found a *data analysis gap* where most data in an enterprise is "dark data": data that is collected but not analyzed. In the early 2000s, companies became more data driven and the amount of data collected was growing exponentially, doubling ever 20 months. In the discussions with customers, Amazon found the following causes for the analysis gap:

1. **Cost** - large up-front expense was hard to justify the unclear value of these datasets
2. **Complexity** - Heavy IT involvement and hardware support required made it complicated for many businesses that lacked the required knowledge in their organization
3. **Performance** - Growing the data warehouse without negatively impacting the query performance was a difficult problem
4. **Rigidity** - Most data were unstructured and mutate over time, increasing the overall complexity of the database design.

Amazon Retail itself faced these problems, adding increased demand in-house. The team placed the following as design goals to achieve their above goals:

1. **Minimize cost of experimentation** - Large amount of data contains noise that it not useful for the company. Understanding which part of the data and how to analyze the data to get business insight requires extensive experimentation.
2. **Minimize time to first report** - This metric is important for customer retention and new customer experience as Amazon applies the lessons used from its consumer segment to its database services.
3. **Minimize administration** - Minimizing the necessity for the user to handle backup, cluster creation, patching, and resizing reduces the complexity for its users
4. **Minimizing scaling concerns** - Parallelizing and handling scaling while keeping the price scaling linearly or sublinearly with dataset growth decreases cost for the user.
5. **Minimizing tuning** - Reducing the number of knobs that has to handle reduces cost and complexity for the end user.

2 Architecture

2.1 Data Plane

The **data plane** layer manages the data storage and SQL execution. Amazon started with an database engine licensed from ParAccel, a software company that provided a database management system to provide analytics into business intelligence. The storage and compute are distributed over multiple nodes, with a single leader node managing the compute nodes. The leader node connects with the client, parses requests, and generate the optimized query plan that are sent to the compute nodes, which handles the actual query processing and data manipulation that is stored against local data.

The data in a cluster is partitioned at the node level and at the core level within each node. The user can set the distribution method in settings. Within each slice, the data is stored as linked columns. In 2013, data

blocks were replicated in the database instance and within S3, both being written to synchronously, along with an asynchronous write to a secondary copy in S3. Query processing involves query plan generation followed by a compilation to C++ and machine code, where the cost of compilation is amortized by the multiple compute nodes executing parallelly.

2.2 Control Plane

The **control plane** is a separate plane that helps the DBMS deploy new database engine bits, aggregate metrics, generate events, manage logs, and monitoring errors. This plane is located off-instance and handles maintenance tasks based on the incoming telemetry or by request from the user, either through an API or a console.

3 Amazon Redshift Revisted ---

3.1 Code Generation

Redshift generates compiled binary from C++ code based on the query plan and schema executed. The generated code keeps the working set as close to the CPU as possible to reduce memory latency. Amazon defines each compiled file as a *segment*, which contains a pipeline of operators called *steps*. The code is generated such that the tuple is kept in the registers until the tuple is sent through network, materialized into main memory, or flushed to disk by processing each tuple in a pipeline.

3.1.1 Push-based Vectorized Query Processing

Since acquiring the key and value of a tuple requires a pull-based fetcher, this results in retaining many states for each column scanned in the registers, filling the entire register file if many columns are read. Branch misprediction from a selective predicate also stalls the pipeline and decompression algorithms getting inlined decreases the performance of the scans. To mitigate this, Redshift added a pre-compiled SIMD-vectorized scan that covers all data types and its encoding and compression schemes, where the output is stored on the stack. The reduced register pressure along with the alleviated compilation requirements lead to order of magnitude faster compilations and performance.

3.1.2 Hiding latency with Prefetching

Since the pipeline is mostly push based, Redshift suffers from memory stalls with large hash tables. Amazon handles this by letting the memory stalls occur and hiding the latency with prefetching. A circular buffer is placed in L1 cache and prefetches and pushes each tuple that arrives into the buffer. This is only required if the hash table is too large to fit in the CPU cache. If the size is known beforehand or is small enough, then the prefetching code is not generated.

3.1.3 Compilation

Compilation adds an extra layer and latency to query execution. When identical and similar queries are executed, the compiled segments are reused. The incurred cost of a new query is minimized by using compute services outside the Redshift cluster by parallelizing and distributing the compilation workload.

3.1.4 Runtime Adaptation

The execution engine makes runtime improvements for joins by collecting statistics such as the size of the join to size the bloom filter accordingly, making it larger to handle more selective probing or smaller if it is not needed. It also maintains multiple bloom filters of varying sizes that it can swap between to adapt to changing patterns within the data.

3.1.5 Advanced Query Accelerator (AQUA)

AQUA utilizes FPGAs and ASICs to accelerate compression and encryption and to execute filters and aggregation with high throughput. The need for AQUA arised to avoid a network bottleneck and acts as a functional interface, not as a storage interface.

3.2 Scaling Storage

Amazon Redshift defines its storage layer to include local storage to cloud object storage and all data lifecycle operations such as commits, caching, restore, replication, and disaster recovery. The storage has undergone major changes since the 2015 paper:

- **Durability and Availability** - the storage layer is built on Amazon S3, utilizing the durability and disaster tolerance of S3 for Redshift
- **Scalability** - Amazon S3 allows for easy scalability as existing solutions are repurposed for Redshift
- **Performance** - Prefetching and dynamically sizing algorithms and data structures based on the size of the in-memory caches improve the performance of Redshift.

3.2.1 Redshift Managed Storage (RMS)

The RMS has been designed for durability of 99.999999999% and 99.99% availability over a given year over multiple availability zones. Amazon designed its own file format to store data on SSDs, spilling into Amazon's S3 servers, where a backup is also stored. Transactions are synchronously committed to Amazon S3 by RMS and follow snapshot isolation. In addition, techniques such as decoupling metadata from data allows for faster reconfiguration, minimizing the cost of experimentation and better performance.

3.2.2 Beyond Local Capacity

Redshift implements a tiered-storage cache placing cold data blocks in low-level clock caches and upgrading as the block gets hotter. During cluster reconfiguration, the tiered cache also drives rehydration, where the authors found an 80% local disk hit rate at 20% rehydration completion. Redshift also utilizes a dynamic disk cache.

3.2.3 Incremental Commits

Redshift shifted to a log-based commit protocol, only logging the difference in data since the last commit. They found a 40% commit performance improvement by converting the series of random I/O operations into a log append. They found it helped reduced cost of concurrency scaling and data sharing as they achieve transactionally consistent data by appending the log to the superblock.

3.2.4 MVCC

Previously, Redshift had a graph-based mechanism to find dependencies between transactions. This was replaced with a design based on Serial Safety Net (SSN) which now acts as a certifier on top of Snapshot Isolation, helping to guarantee strict serializability in a more memory efficient manner.

3.3 Scaling Compute

Redshift allows the customer to add and remove compute nodes with minimal data movement, prioritizing ongoing request over the repartitioning of data due to the new number of compute nodes. It effectively decouples the compute from the memory as much as possible, allowing for multiple compute nodes to handle a single data partition if needed. With this, Redshift is able to handle concurrency scaling, by adding compute nodes in the background if needed. In addition, Redshift also implemented data sharing between clusters, allowing for more parallelism on the same data partition.

3.4 Automated Tuning

One of the main goals described in the 2015 paper was minimizing tuning, complexity, and administration. The team automates the distribution of distribution and sort keys through an Automatic Table Optimization (ATO) by collecting metadata from the executed queries. A distribution key controls how the data is partitioned over each compute node. A sort key handles how the data is sorted within one compute node. Based on the metadata, the ATO generates the number of keys needed and its expected improvements. Since the recommendation ATO generates is dependent on stable patterns of queries and rearranging the data is expensive, the user is given oversight and has the ability to approve and reject the generated distribution. The ATO generates these numbers with a *distribution key advisor* and *sort key advisor*. The distribution key advisor builds a weighted graph from all the joins and selects the number of keys that reduces the total network distribution cost. The sort key advisor analyzes the selectivity of the range restricted scans and recommends the number that improves the effectiveness of zone map filtering.

Redshift also handles the flow of incoming query into a compute node. Restrict the flow too much and there is poor CPU utilization. Allow too many and resources become less effective as they are oversaturated. To find the balance, Redshift employs a machine learning model to predict a query's resource requirements and adjusts the number of concurrent executing queries based on this value. The process is handled by the Automatic Workload Manager (AutoWLM) which employs a weighted round-robin scheme for scheduling the queries. Redshift also uses materialized views for repeated queries. These views are updated by incrementally updating the filters, projection, and groups on the portions that were affected. It also manages which views to prioritize based on cost to repeat and frequency of query. With these materialized views, the optimizer may choose to rewrite the query to use the materialized views.

Lecture #16: Google Spanner

15-721 Advanced Database Systems (Fall 2025)

<https://www.cs.cmu.edu/~15721-f25/>

Carnegie Mellon University

Notes by: Yuhe Ma

1 Motivation and System Overview

1.1 Why Spanner?

Google needed a *globally distributed, transactional database* that could:

- Scale to thousands of machines and petabytes of data.
- Provide **strong consistency** and **transactions** for mission-critical services (e.g., Ads).
- Offer a familiar **SQL** interface instead of forcing every application to reason about sharding and replication.

Before Spanner:

- Sharded MySQL provided SQL but made sharding, failover, and consistency management extremely painful.
- Bigtable was scalable but had no multi-row transactions or strong ACID guarantees.

Spanner starts as a transactional, multi-version key-value store and then evolves into a full **SQL system** with:

- Relational schema, secondary indexes, and a distributed query processor.
- Support for both OLTP and some analytical workloads.

1.2 High-level Architecture

At a high level:

- Data is **sharded** into key ranges called *splits* or *tablets*.
- Each tablet is replicated via **Paxos** across multiple data centers (zones).
- Each Paxos group has a **leader** and several **followers**.
- A transaction layer uses **2-phase commit (2PC)** across tablets, and **TrueTime** to enforce external consistency.

Figure 1 shows:

- A *Paxos group* for Tablet A with replicas in three data centers, one acting as the leader.
- Writes and strongly consistent reads going through the leader.
- Followers serving snapshot reads.
- Multiple tablets (B–Z) each as their own Paxos group, with cross-tablet transactions coordinated by 2PC.

2 Time, TrueTime, and External Consistency

2.1 External Consistency

Spanner aims for **external consistency** (a form of linearizability): if transaction T_1 finishes *before* transaction T_2 begins in real time, then:

$$T_1 \prec_{\text{real-time}} T_2 \Rightarrow \text{ts}(T_1) < \text{ts}(T_2),$$

SPANNER TABLETS

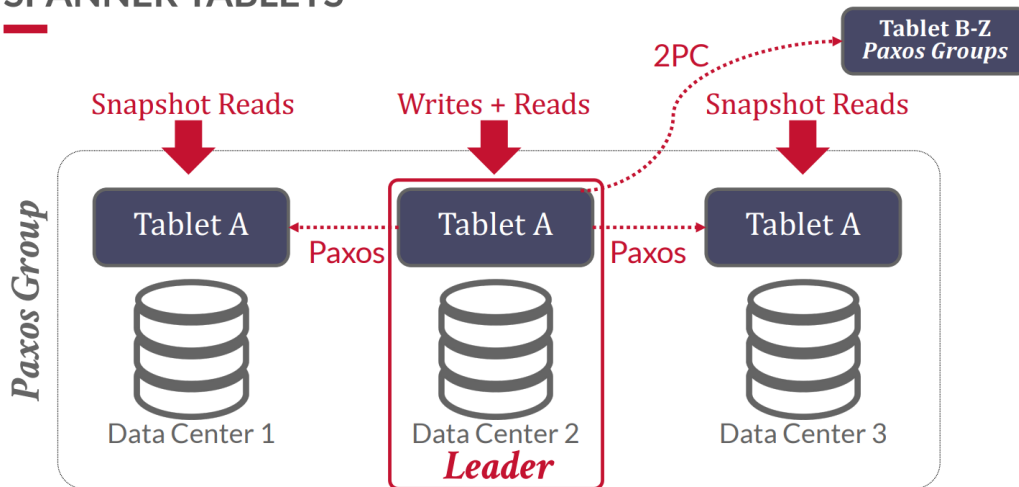


Figure 1: Spanner tablets replicated as Paxos groups across data centers, with multiple tablets (A, B–Z) coordinated using 2PC across Paxos groups.

where $ts(T)$ is the commit timestamp of transaction T .

This is stronger than just serializability: the ordering must respect the wall-clock view of an external observer.

2.2 Clock Skew Problem

With distributed nodes and imperfect clocks, naively using local physical time as timestamps can violate external consistency.

- Node 1 and Node 2 have clocks that differ by some skew.
- A transaction T_1 commits on Node 1 at local time 6.
- A later transaction T_2 commits on Node 2 at local time 4.

In real time, T_1 happens before T_2 , but their timestamps appear as:

$$ts(T_1) = 6, \quad ts(T_2) = 4,$$

which violates $T_1 \prec T_2 \Rightarrow ts(T_1) < ts(T_2)$.

In Figure 2, each node has its own time axis. The figure overlays the lifetimes of T_1 and T_2 on each axis, and shows that due to skew, a later real-time transaction can end up with an earlier timestamp on a different node.

2.3 TrueTime API and Time Intervals

To fix this, Spanner introduces the **TrueTime** API. Instead of returning a single time, TrueTime returns an interval of possible real times:

$$TT.now() = [t_{\text{earliest}}, t_{\text{latest}}] = [t - \epsilon, t + \epsilon],$$

where:

- t is the local clock's best estimate of the current time.
- ϵ is the current bound on clock uncertainty.

Figure 3 visualizes:

- A horizontal time line.
- A bracket $[t_{\text{earliest}}, t_{\text{latest}}]$ indicating the interval where the actual real time must lie.
- The point labeled `now()` sits conceptually in the center, but the system only knows the entire interval.

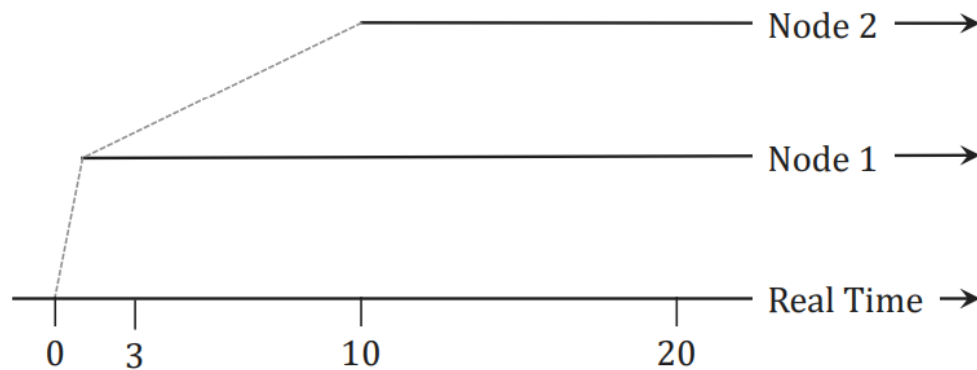


Figure 2: Clock skew example: Node 1 and Node 2 have misaligned clocks, so real-time order and timestamp order can disagree.

What time is it? Guaranteed to be in this range,

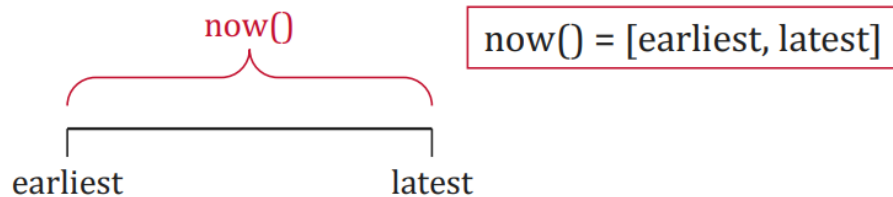


Figure 3: TrueTime interval: `TT.now()` returns a range $[t - \epsilon, t + \epsilon]$ that contains the real time, plus a commit-wait protocol to ensure external consistency.

2.4 Commit Timestamps and Commit Wait

For a transaction, Spanner chooses a commit timestamp s such that:

$$s \in [t - \epsilon, t + \epsilon],$$

and then enforces a **commit wait**:

Wait until `TT.now().earliest` $> s$.

Intuition:

- By the time commit wait finishes, real time has certainly moved past s on all machines.
- Any transaction that begins after seeing the result of this commit must call `TT.now()`, which will return a range strictly greater than s .

Thus, for any two transactions T_1 and T_2 where T_1 finishes before T_2 begins in real time, their timestamps satisfy:

$$\text{ts}(T_1) < \text{ts}(T_2),$$

ensuring external consistency.

3 Data Partitioning, Replication, and Transactions

3.1 Tablets and Paxos Groups

Spanner partitions data into **tablets**, each responsible for a key range. Each tablet is managed by a **Paxos group**:

- One leader and multiple followers across zones.
- Leaders handle all read–write transactions.
- Followers keep up via the replicated Paxos log.

Figure 1 shows:

- Tablet A’s replicas in three data centers joined by arrows labeled *Paxos*.
- The leader replica annotated with “Writes + Reads”.
- Followers annotated with “Snapshot Reads” as they can serve consistent reads at a given timestamp.
- Additional tablets (B–Z), each forming their own Paxos groups.

3.2 Concurrency Control and MVCC

Within a Paxos group:

- Spanner uses **strict two-phase locking (2PL)** for read–write transactions.
- Updates are versioned by commit timestamps, giving a multi-version data model.
- Read-only transactions can read a consistent snapshot at a timestamp t without acquiring locks.

The MVCC model:

- Each logical row has multiple versions (value, ts).
- To read at timestamp t , Spanner selects the version with the highest $ts \leq t$.

3.3 Two-Phase Commit across Tablets

For a transaction that touches multiple tablets:

- A **transaction manager** acts as the 2PC coordinator.
- Each involved tablet (Paxos group) is a 2PC participant.

The protocol:

1. *Prepare* phase: each participant validates and *promises* to commit at timestamp s .
2. *Commit* phase: after all participants are ready, the coordinator decides to commit or abort and logs this decision.

The integration with TrueTime:

- The commit timestamp s is chosen to respect TrueTime bounds.
- Commit wait ensures that the final commit is externally consistent even across multiple tablets.

4 SQL Layer and Distributed Query Processing

4.1 From KV Store to SQL System

The Spanner SQL paper describes how Spanner evolved into a full SQL system:

- A common SQL dialect shared with systems like F1 and BigQuery.
- Support for relational schemas, secondary indexes, and joins.
- A distributed query optimizer that transforms a relational algebra (RA) tree into an execution plan.

4.2 Partitioned Execution and Distributed Union

Core idea: execute as much work as possible *close to the data*.

- Conceptually, a logical scan $\text{Scan}(T)$ is rewritten as a distributed union over shards:

$$\text{Scan}(T) \approx \text{DistributedUnion}_i(\text{Scan}(T_i)),$$

where T_i are the shards (tablets) containing disjoint key ranges of T .

- Simple operations such as filters, projections, and some aggregates can be pushed below `DistributedUnion` if they are *partitionable*.

This yields a plan where:

- Each shard runs its local subplan (scan, filter, partial aggregate).
- The root node merges partial results (e.g., sums, counts, top- k).

4.3 Batched Apply Joins and DistributedApply

For joins based on keys of independently sharded tables, Spanner uses a **batched Apply join**, implemented via `DistributedApply`.

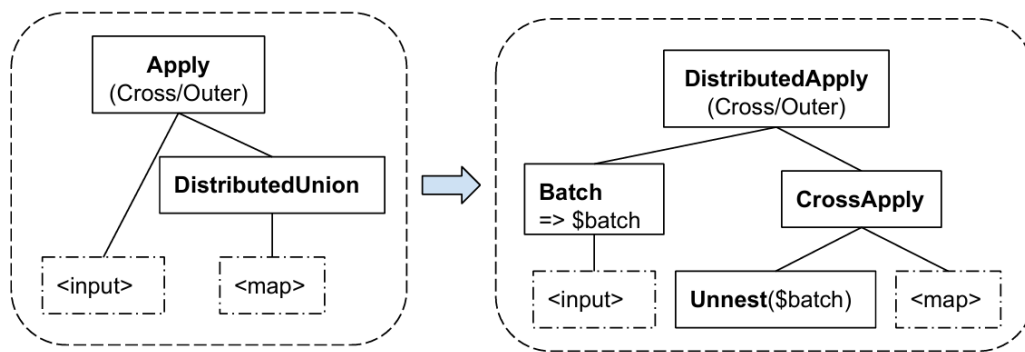


Figure 4: Batched Apply join plan: a row-at-a-time Apply plan is transformed by batching rows and unnesting them on target shards, implemented via `DistributedApply`.

Figure 4 illustrates:

- A naïve plan where a `CrossApply/OuterApply` operator sends one lookup per input row to the remote table or index.
- An optimized plan that:
 - Batches a set of input rows into a *batch*.
 - Sends a single batched request per relevant shard using `DistributedUnion`.
 - Unnests the batch on the target shard and executes the join subtree there.

Benefits:

- Greatly reduces the number of RPCs (from one per row to one per batch per shard).
- Exploits data locality: most join work happens locally at the shards.
- Still respects the same logical semantics as the original Apply join.

4.4 TrueTime, Snapshots, and Read Performance

The SQL engine relies heavily on Spanner’s snapshot semantics:

- Snapshot reads** at timestamp t can be served from followers, increasing read throughput and reducing latency.
- Read-only transactions specify a timestamp or a staleness bound, and the query processor chooses a safe snapshot.
- Because of TrueTime, these snapshots are globally consistent: every shard’s state at timestamp t is ordered consistently with other shards.

This allows:

- **Parallel scans** across many tablets, each serving snapshot data.
- Complex read-only queries (joins, aggregates) over a consistent, globally ordered snapshot without coordination at read time.

Summary

Spanner is a globally distributed, multi-version, synchronously replicated database that provides external consistency by integrating a tightly controlled time API with traditional transactional mechanisms. Each write transaction is assigned a commit timestamp chosen from a bounded TrueTime interval $TT.now() = [t - \epsilon, t + \epsilon]$, where ϵ is a dynamically maintained uncertainty bound derived from GPS and atomic-clock-backed time masters. The system then performs a commit wait, delaying visibility of the transaction until the lower bound of TrueTime has advanced past the chosen timestamp, i.e., until $TT.now().earliest > s$. This ensures that if transaction T_1 completes before T_2 begins in real time, then $commit_{ts}(T_1) < commit_{ts}(T_2)$, yielding external consistency. Concurrency control combines strict two-phase locking for read-write transactions with multi-version storage: each key maintains a timeline of versions indexed by commit timestamps, enabling lock-free snapshot reads at an arbitrary timestamp T that see exactly the effects of all transactions with $commit_{ts} \leq T$.

Physically, Spanner partitions tables into key-range shards (tablets), each managed by a Paxos group with one elected leader and several followers distributed across data centers. All writes and strong reads for a tablet are funneled through the leader and replicated via Multi-Paxos; followers serve snapshot reads once they have applied all log entries up to the relevant snapshot timestamp. Cross-tablet transactions run two-phase commit over the participating leaders, with TrueTime providing globally consistent commit timestamps. On top of this storage and replication layer, Spanner implements a distributed SQL engine. Logical scans are rewritten into distributed unions over shards, $Scan(T) \approx DistributedUnion_i(Scan(T_i))$, and the optimizer pushes filters, projections, and many aggregations below the union when they are partitionable. Correlated joins and index-to-base-table lookups are executed via batched operators such as `DistributedApply`, which accumulate input tuples, group them by target shards, and issue batched remote subqueries to avoid per-row RPCs. Range extraction, based on compile-time rewrites into correlated self-joins and runtime filter trees, derives distribution, seek, and lock ranges for multi-column primary keys, pruning shards and key intervals aggressively. Finally, a blockwise-columnar storage format (Ressi) replaces SSTables, combining an LSM organization with PAX-style blocks and typed vectors to improve scan performance and support hybrid OLTP/OLAP workloads without sacrificing the transactional and timing guarantees.